

Gloam

A self-hosted, account-free 3D virtual tabletop for one private D&D group — written to be built end to end by Claude Code.



OWNER Xini (host & admin)

BUILDER Claude Code · Opus 5.5

DATE 27 September 2026

SCOPE 24 features · 224 acceptance criteria · 16 build phases

0. Read me first (for Xini)

This document is written to *Claude Code*. Everything after this page is addressed to the builder ("you"). This page is for you, the human.

What this is. A complete spec for Gloam: a web app you run on your own PC. Your friends open a link, type an invite code, wait in a lobby until you let them in, and then play on a shared 3D board: maps, 3D minis or hand-drawn standees, fog of war with real light and darkvision, a Divinity-style movement line with a movement budget, 3D dice everyone can see, character sheets that cope with homebrew, spells with area templates and effects, initiative, HP and conditions, sound, undo, and autosave. No accounts, no subscriptions, nothing to sign up for.

How to use it.

1. Install on your PC: **Node.js 24 LTS**, **Git**, and enable pnpm with `corepack enable`. Later (only when you want friends to join from outside your house) install **cloudflared** — the app shows you the exact command for your OS.
2. Create an empty folder, run `git init`, create a `docs` folder and save the Markdown version of this document there as `docs/SPEC.md`. Give Claude Code the Markdown file, not the PDF: it can search and re-read Markdown far more easily.
3. Open Claude Code in that folder. Pick Opus 5.5 and set effort with `/effort high` (use `/effort xhigh` for the vision, movement and spells phases if you want maximum care). Stay in auto permission mode.
4. Paste the **kickoff prompt from Appendix A**. Then set the **goal from Appendix A** with `/goal`. Claude Code will research first, then build in phases, testing and screenshotting its own work as it goes.
5. Expect a long build (many hours, many turns). You can stop and resume at any time; progress lives in `docs/PROGRESS.md` and `docs/FEATURES.json`, and every feature is committed to git.

What it costs to run. Nothing monthly. The app, database and files all live on your PC. Cloudflare "quick tunnels" need no account at all (the link changes every time you open the table). If you want the same link every week, create one free Cloudflare account and add a domain you own (about £8–10 a year); the app supports both.

Two things worth knowing before you start.

- **Units.** D&D speeds are in feet: a typical character moves **30 ft (about 9 m)** per turn, not 30 m. Gloam works in feet internally and has a metres toggle that uses the official convention of 5 ft = 1.5 m, so 30 ft shows as 9 m.
- **"All the spells".** The rules content that anyone may legally ship is the SRD 5.2.1: **339 spells**, all 15 conditions, and the core rules, under a Creative Commons licence. Spells that exist only in the Player's Handbook are Wizards of the Coast's copyrighted text, so the app cannot ship them — but you can add any spell your group uses through the homebrew builder or the JSON import (for example, by asking Claude to convert a list), and it stays in your own database.

Where things are in this document. Part A explains the mission and architecture. Part B specifies every feature with numbered acceptance criteria. Part C is the engineering design. Part D is the visual and sound design. Part E covers D&D rules research and content. Part F is the build plan. The appendices hold ready-to-use files (kickoff prompt, CLAUDE.md, subagents, settings, icon source, import formats).

Contents

Part A — Orientation	5
1. Mission and quality bar	5
2. Product principles	5
3. Architecture at a glance	6
4. How you work (instructions for Claude Code)	7
Part B — Product	9
5. Glossary	9
6. Roles and permissions	10
7. Core journeys	11
8. Feature specifications	12
8.1 F01 — Table lifecycle and the doorway (HOST)	12
8.2 F02 — Joining, identity, lobby and approval (AUTH)	13
8.3 F03 — Campaigns and scenes (SCN)	14
8.4 F04 — The board and camera (BRD)	15
8.5 F05 — Tokens and minis (TOK)	16
8.6 F06 — Movement and measurement (MOV)	17
8.7 F07 — Walls, doors and zones (WAL)	19
8.8 F08 — Vision, light and fog of war (VIS)	20
8.9 F09 — Dice (DICE)	21
8.10 F10 — Character sheets (SHEET)	22
8.11 F11 — HP, damage, conditions and death (HP)	24
8.12 F12 — Combat and initiative (CMB)	25
8.13 F13 — Spells, abilities and effects (SPL)	26
8.14 F14 — Undo, redo and history (UNDO)	28
8.15 F15 — Persistence, saves and recovery (PER)	28
8.16 F16 — Assets and uploads (AST)	28
8.17 F17 — Audio and music (AUD)	29
8.18 F18 — Table flavour: emotes, pings, handouts and the log (FUN)	30
8.19 F19 — DM control panel (DMP)	30
8.20 F20 — Admin console (ADM)	31
8.21 F21 — Responsive and touch (RSP)	32
8.22 F22 — Settings and accessibility (A11Y)	32
8.23 F23 — Local API and MCP server (API)	33
8.24 F24 — First run and the demo campaign (DEMO)	33
8.25 Cross-cutting acceptance criteria	34
Part C — Engineering	35
9. Stack and versions	35
10. Repository layout	37
11. Runtime, configuration and the data directory	39
12. Data model and database	40
13. Realtime protocol	43
14. Commands, permissions and history	48
15. Vision and lighting engine	49
16. Movement and pathfinding engine	52
17. Areas of effect and targeting	53
18. Dice engine	55

19. Rules engine and house rules	57
20. Persistence, snapshots and backups	59
21. Asset pipeline	60
22. Security model	61
23. Client architecture	63
24. Rendering and VFX	64
25. Audio engine	66
26. Local API and MCP server	67
Part D — Design system	69
27. Art direction and tokens	69
28. Components	71
29. Screens and wireframes	71
30. Icons	74
31. Sound design	75
Part E — Rules and content	77
32. Research phase (Phase 0 tasks)	77
33. Content pipeline (SRD 5.2.1)	78
34. Verified rules reference (SRD 5.2.1)	79
Part F — Delivery	85
35. Phase plan	85
36. Testing and verification	86
37. Performance budgets	87
38. Definition of done	88
39. Host runbook (becomes <code>docs/HOSTING.md</code>)	88
40. Risks and fallbacks	88
41. Out of scope (don't build)	89
Appendices	90
Appendix A — Kickoff prompt and goal	90
Appendix B — <code>CLAUDE.md</code>	90
Appendix C — Subagents (<code>.claude/agents/*.md</code>)	91
Appendix D — <code>.claude/settings.json</code> and hook scripts	92
Appendix E — <code>docs/FEATURES.json</code> and tools	93
Appendix F — Import formats and AI conversion prompts	94
Appendix G — Icon reference drawings	96
Appendix H — Keyboard and gesture map	100
Appendix I — Attribution and trademarks	101
Appendix J — Configuration reference	101

Part A — Orientation

1. Mission and quality bar

Build **Gloam**, a self-hosted, account-free, real-time 3D virtual tabletop (VTT) for a single private Dungeons-&-Dragons-style group. It runs as **one Node.js process on the host's own computer**. Friends join from any browser (desktop, tablet or phone) through a Cloudflare Tunnel link that exists only while the host has "opened the table".

The host (Xini) is the **Admin**. The Admin appoints a **DM**. Everyone else is a **Player** (or a **Spectator**). The DM has total creative freedom. Players are held to the rules by default, and the DM can lift any restriction for any character at any moment.

The quality bar is a crafted indie game, not a CRUD app. When a player drags their mini, a glowing path should route around walls, show "20 ft · 10 left", turn red past their limit, and warn them they are about to provoke an opportunity attack. When someone rolls, real 3D dice should clatter across the screen and land on the number the server decided. When the party walks into a dark crypt, the torch should flicker, the darkvision character should see the room in grey, and the human next to them should see nothing. Aim for that feeling everywhere. Go beyond the basics: include the small interactions, feedback, animation and sound that make a tabletop feel alive.

Reference points (study the *interaction patterns*, never copy assets or trade dress):

Reference	What to learn from it
Divinity: Original Sin 2, Baldur's Gate 3	Movement path preview with cost and remaining budget, opportunity-attack warnings on the path, turn-order portrait strip, readable combat, satisfying hit feedback
Foundry VTT	Rigour of walls, doors, light sources, senses and fog-of-war exploration
Owlbear Rodeo	How little friction joining and fog painting can have
TaleSpire	Tactile 3D minis on a physical-feeling table, lighting mood

2. Product principles

These are the constitution. When anything else in this document is unclear, decide in the way that best satisfies these principles, record the decision in `docs/DECISIONS.md`, and continue.

- **P1 — Zero accounts, zero SaaS.** At runtime Gloam talks to nothing on the internet. No CDNs, no Google Fonts, no analytics, no error reporting, no AI APIs, no auth providers. Every font, model decoder, texture and sound is bundled or generated locally. The only outside component is the optional `cloudflared` process that acts as the doorway. This is enforced by the Content-Security-Policy (`connect-src 'self'`) and by tests.
- **P2 — The app offers; nobody is forced.** Every automation (movement limits, rolls, saving throws, damage, initiative, death saves, conditions, concentration) is a *suggestion with a manual path and a Skip*. The DM can skip, edit or override anything. Anywhere a roll is requested, a player may instead type the result of a physical die.
- **P3 — The DM is sovereign, the Admin is omnipotent, Players follow the rules unless the DM lifts them.** The DM can place any unit with any values, change any number, and grant house-rule exceptions per character. The Admin can do everything the DM can, plus run the server, doorway, people and data.
- **P4 — The server is the referee.** Every change is validated on the server. Information that a player's character cannot perceive never reaches that player's browser; hiding an *entity* (a token, a hidden or secret wall, a note, a roll, a sheet) only visually is a bug. The one deliberate exception is map art: every admitted player downloads the whole map image, so fog hides the map itself only visually (§15.8).
- **P5 — Nothing is lost.** Every accepted change is written to SQLite before it is broadcast. Snapshots, backups and undo sit on top of that.

- **P6 — The host's PC is the host, and only when the host says so.** The server listens on `127.0.0.1`. Friends can reach it only through the tunnel while the table is open. Closing the table closes the door.
- **P7 — Everywhere.** Desktop first, but fully usable with touch on tablets and phones.
- **P8 — Beautiful by default.** A committed art direction (Part D), cinematic lighting, purposeful motion, synthesized sound. No generic "AI app" look.

3. Architecture at a glance

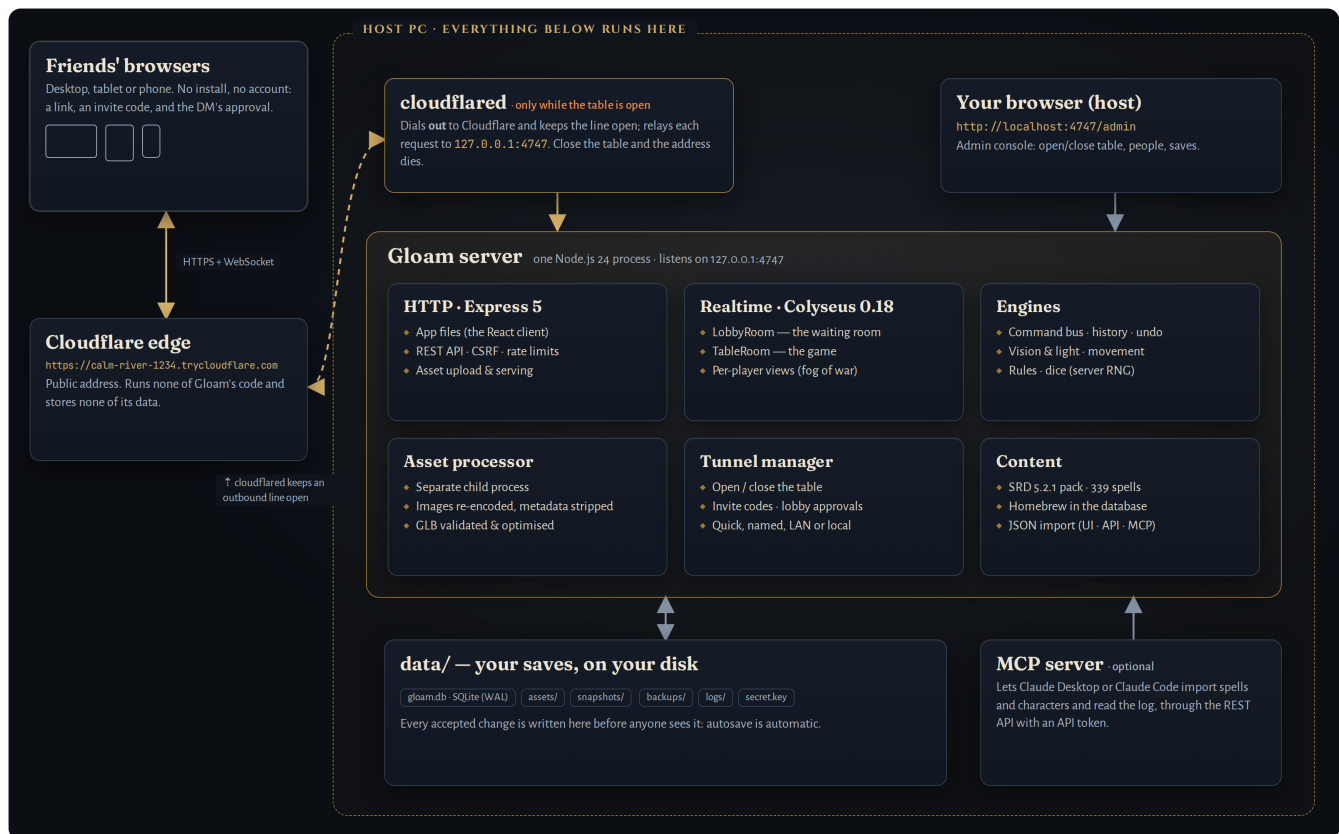


Figure 1 — Gloam architecture. One Node process on the host PC; Cloudflare Tunnel is only a doorway.

What Cloudflare Tunnel does (and doesn't). `cloudflared` is a small program on the host PC. When the Admin clicks *Open table*, Gloam starts it. It dials *out* to Cloudflare and keeps that line open; Cloudflare gives it a public HTTPS address. When a friend visits that address, Cloudflare relays the request down the open line to `127.0.0.1:4747`. Cloudflare runs none of Gloam's code and stores none of its data; the PC does all the work. No port forwarding is needed, and the home IP address is never exposed. When the Admin clicks *Close table* (or the PC sleeps), the line drops and the address stops working.

- **Quick tunnel** (default): no Cloudflare account; a random `*.trycloudflare.com` address each time the table opens. Documented limits: at most 200 in-flight requests, no Server-Sent Events, no uptime guarantee. WebSockets work. It will not start if `~/cloudflared/config.yaml` exists.
- **Named tunnel** (optional): one free Cloudflare account plus a domain on Cloudflare gives a stable address (for example `table.example.com`). Gloam runs it with a tunnel token stored in its settings.
- **LAN mode** (optional): for players in the same house, the server can listen on the local network instead (`http://<pc-ip>:4747`).

The life of one token move (the pattern every feature follows):

1. Player drags their mini. The browser computes a preview path (shared pathfinding code) and shows the path line, cost and budget. Every 66 ms it sends a small `move.preview` message; the server relays it only to players who can currently see that token.
2. On release, the browser sends `move.commit` with the path.

3. The server checks permission (owner? their turn? movement locked?), re-computes the path cost against the *true* walls (including walls the player can't see), and clamps or rejects if over budget.
4. The server applies the change, writes the new position, the budget and a history entry to SQLite in **one transaction**, then updates the Colyseus state.
5. The vision engine recomputes what each player can see. Players who can now see a monster get it added to their view; players who lost sight get it removed. Players who see only part of the move get the path clipped to the part they could see.
6. Every browser animates the mini along the path. The mover's fog of war updates as they walk.

4. How you work (instructions for Claude Code)

4.1 Read order and source of truth

- Read this entire document once before writing code. Then work through the phases in §35 in order.
- Before starting each phase, re-read that phase's section, the feature sections it references, and the engineering sections it touches. This document is the source of truth; your memory of it after compaction is not.
- If this document conflicts with a library's current documentation, the library wins for API details and this document wins for behaviour. Record the difference in `docs/DECISIONS.md`.

4.2 Autonomy

- **Do not ask Xini questions.** When the spec is silent or ambiguous, choose the option most consistent with the principles in §2, write one line in `docs/DECISIONS.md` (date, decision, reason, alternatives rejected) and carry on.
- Stop only when (a) you need something only a human can provide (for example, a Cloudflare tunnel token), (b) an action would be destructive outside this repository, or (c) two principles genuinely conflict and the choice is irreversible. In every other case, keep going.
- **Standing instruction about how turns end.** A message with no tool call ends your turn, and the work stops until someone asks you to continue. Do not end turns in any of these four ways while work remains: (1) a long summary that announces the next step but does not start it; (2) an offer to continue "unless you'd prefer otherwise"; (3) a list of decisions for Xini when none of them blocks the rest of the work; (4) deciding this is a good place to report because the turn was long or a milestone is done. Status notes and recommendations are welcome, but put them in the same message as your next tool call and carry on with whatever does not depend on an answer. This does not override the need for care with destructive actions.
- **Time matters.** Don't spend time that can be avoided. Finishing features end to end beats polishing one feature forever.

4.3 State files (create these in Phase 0)

File	Purpose	Rules
<code>docs/SPEC.md</code>	This document	Read-only for you, except appending clarifications to Appendix J's errata section
<code>docs/FEATURES.json</code>	One entry per acceptance criterion (AC-...) extracted from this spec by <code>tools/extract-features.mjs</code>	Only set <code>"passes": true</code> with an <code>evidence</code> string (test name, command output file, or screenshot path). Never delete or reword entries. If an AC is wrong, add <code>"disputed": "<reason>"</code> and log it in <code>DECISIONS.md</code>
<code>docs/PROGRESS.md</code>	Session log	Append after every work block: what was done, what's next, blockers, the output of <code>pnpm features:status</code>
<code>docs/DECISIONS.md</code>	Decisions and deviations	One line per decision
<code>docs/research/*.md</code>	Phase 0 research outputs	Cite sources (URL + page)

- **Commits.** Commit after each completed group of acceptance criteria, with messages like `P3 MOV-02/03/08: routed path preview, difficult terrain, wall collisions`. Never commit with failing `pnpm check`.

- **No stubs in shipped paths.** No TODO placeholders, dead buttons or fake data in merged work. If something is deferred, it is absent from the UI and listed in PROGRESS.md.
- **Scope.** Build what this document specifies. Don't add features it doesn't ask for (see §41 for explicit exclusions); spend spare effort on quality instead.

4.4 Verification loop (how "done" is decided)

Nothing counts as done because it looks done. For every acceptance criterion, produce evidence:

1. **Unit tests** (Vitest) for all pure logic in `packages/shared`: geometry, visibility, pathfinding, AoE, dice, rules math, schema validation.
2. **Integration tests** for the server: spin up the server in-process on a random port with a temp data dir, connect test clients with `@colyseus/sdk`, send commands, assert state and persistence.
3. **End-to-end tests** (Playwright) with several browser contexts at once (Admin/DM + Player A + Player B) for the journeys in §7.
4. **Screenshots.** `pnpm shots` renders the key screens at 1440×900, 1024×768 and 390×844 into `artifacts/screens/<phase>/`. After each phase, dispatch the `visual-critic` subagent (Appendix C) to review them against Part D. Fix what it flags that affects the spec's requirements; ignore pure taste disagreements. At most two critic rounds per phase.
5. **Independent review.** At the end of Phases 4, 9 and 15, dispatch `rules-auditor` and `security-reviewer` (Appendix C). They report; you fix.

A separate agent grades the work because the agent that built something is a poor judge of it. Don't add vague "double-check everything" passes on top; the gates above are the checks.

4.5 Subagents: when and when not

- **Use subagents** for Phase 0 research (in parallel), for independent review (visual-critic, rules-auditor, security-reviewer), and for isolated leaf tasks that touch no shared code (for example, drafting the sound recipes or icon variants).
- **Don't split tightly coupled systems across parallel agents.** Movement, vision, commands and the room state must have a single owner: you, working sequentially. Parallel agents working on coupled game systems have been observed to *add* defects.

4.6 Context hygiene

- After any compaction, re-read `CLAUDE.md`, the tail of `docs/PROGRESS.md`, and the current phase in §35 before continuing.
- Keep large outputs out of context: write test logs and command output to files under `artifacts/` and search them.
- Your context window will be compacted automatically as it fills, so don't stop early to save tokens; save state to the files above instead.

4.7 External downloads

Phase 0 downloads a few things once (the official SRD PDF, a pinned Foundry dnd5e data tag, npm packages). If a download fails, retry once, then implement the fallback in §40 and continue. Runtime code must never download anything.

Part B — Product

5. Glossary

Term	Meaning
Admin	The host (Xini). Exactly one Admin account. Can do everything. Logs in with a password set at first run.
DM	Dungeon Master. Assigned per campaign by the Admin (co-DMs allowed). Full creative control of the game. The Admin may appoint themself.
Player	An admitted person who controls one or more characters.
Spectator	An admitted person who watches. Sees the union of all players' vision; cannot act except emotes and pings.
Pending	Someone who entered a valid invite code and is waiting in the lobby for approval. Sees nothing of the table.
Profile	A person's identity at this table: display name, colour, optional PIN, recognised devices. Not an account with any service.
Table	The running game server. "Open" (doorway up, invite code active) or "Closed".
Session	One opening of the table, from Open to Close. Numbered; the campaign log groups entries by session.
Campaign	A saved game world: scenes, characters, homebrew, logs, settings. The table runs one campaign at a time.
Scene	One battle map or location: map, walls, lights, zones, tokens, fog. One scene is <i>active</i> for players; DMs may prep others.
Map	A scene's floor: an uploaded image, an uploaded 3D model, or a procedural floor.
Token	Anything placed on the board that represents a creature or object: a 3D mini, a standee (upright 2D card) or a coin (flat 2D disc).
Actor	The data behind a token: a <i>Character</i> (has a sheet, owned by a player), an <i>NPC</i> (sheet owned by the DM) or a <i>Unit</i> (a token with inline stats and no sheet).
Sheet	A character sheet: structured core fields used by automation plus free-form custom blocks for homebrew.
Asset	An uploaded file after validation: image, 3D model (GLB) or audio.
Wall	A line segment with blocking properties (movement, sight, light). Doors, windows, curtains, invisible walls and secret doors are wall kinds.
Zone	A polygon with meaning: difficult terrain, water, hazard, impassable, or a labelled area.
Light	A light source (free-standing or carried by a token) with bright and dim radii.
Effect	A persistent spell or ability area on the board (Fog Cloud, Darkness, Spirit Guardians...) with a duration and triggers.
Fog (reveal mask)	Areas the DM has painted as known to players.
Explored memory	Areas a player's character has seen before; shown desaturated, without creatures.
Vision	What a player's character(s) can perceive right now, given walls, light and senses.
Budget	Movement remaining for a creature this turn.
Roll request	The DM asking one or more players for a check, save or attack roll.
Resolution card	The panel that walks a spell or attack through targets, rolls, damage and application.
Snapshot	A full saved copy of a campaign at a point in time.
History entry	A record of one change, with enough information to undo it.
House rule	A campaign setting that changes how the app automates a rule.

6. Roles and permissions

The matrix lives in one module, `packages/shared/src/rules/permissions.ts`: the server enforces it, and the client imports the same module only to decide which controls to show. "Own" means tokens and sheets the player owns or has been granted control of. "DM-grantable" means the DM can enable it for a specific player or character.

Capability	Admin	DM	Player	Spectator	Pending
See the table (active scene, within vision)	✓	✓ (sees all)	✓ (own vision)	✓ (party vision)	—
Open/close table, invite codes, doorway	✓	—	—	—	—
Admit / deny / kick lobby users	✓	✓ (setting, default on)	—	—	—
Ban / unban, rename profiles, API tokens, campaigns, snapshots restore, settings	✓	—	—	—	—
Assign DM role	✓	—	—	—	—
Create/edit/activate scenes, walls, lights, zones, fog	✓	✓	—	—	—
Create/delete any token, set any value, hide/reveal	✓	✓	—	—	—
Move own token (exploration)	✓	✓	✓	—	—
Move own token in combat	✓	✓	Only on own turn, within budget (DM-grantable: free movement)	—	—
Move any token, anywhere, anytime	✓	✓	—	—	—
Open/close unlocked doors within reach	✓	✓	✓	—	—
Toggle own token's light	✓	✓	✓	—	—
Roll dice (public / private-to-DM / self)	✓	✓	✓	—	—
Roll requests to others (including blind requests, where the roller doesn't see the result)	✓	✓	—	—	—
Edit own sheet	✓	✓	✓ unless locked (then propose changes)	—	—
Edit any sheet, lock/unlock sheets	✓	✓	—	—	—
Cast spells / attack from own sheet	✓	✓	✓	—	—
Apply damage/healing/conditions to others	✓	✓	Only via resolution cards the DM confirms (DM-grantable: direct)	—	—
Start/stop combat, set initiative, reorder	✓	✓	—	—	—
End own turn	✓	✓	✓	—	—
Upload assets	✓ (auto-approved)	✓ (auto-approved)	✓ (approval queue, 200 MB quota)	—	✓ (character art images only, queued, 20 MB quota)
Approve/reject uploads, sheet proposals, homebrew	✓	✓	—	—	—
Create homebrew spells/items	✓	✓	Propose (DM approves)	—	—
Music & ambience control	✓	✓	—	—	—

Capability	Admin	DM	Player	Spectator	Pending
Emotes, pings, hand raise	✓	✓	✓	✓	—
Undo own actions	✓	✓	✓	—	—
Revert anyone's actions (history panel)	✓	✓	—	—	—
"Act as" any character	✓	✓	—	—	—
View as any player (preview their vision)	✓	✓	—	—	—

7. Core journeys

Each journey is also an end-to-end test (see §36).

J1 — Game night, host side. Xini runs `pnpm start`. The terminal prints the local admin link. Xini opens `http://localhost:4747/admin`, clicks **Open table**, chooses *Quick tunnel*. Within about 20 seconds the console shows `https://calm-river-1234.trycloudflare.com` and invite code `7K2QH-9XM4D`. Xini clicks **Copy Discord message** and pastes it into Discord. Knock notifications arrive as friends join; Xini admits each one. Xini (or the appointed DM) activates the "Lantern Crypt" scene.

J2 — First join, player side. Dave opens the link on his laptop, enters the code, types "Dave", picks a colour and sets a 4-digit PIN. He lands in the waiting room: a candle burns, "Waiting for the DM to let you in...". While he waits he opens the drawing pad and sketches his human fighter. The DM admits him; the screen dissolves into the board. The DM drops Dave's character at the party spawn point, using his sketch as a standee.

J3 — Returning player, new link. Next week the quick-tunnel link is different, so Dave's browser doesn't recognise the new site. He enters the new code, chooses **I've played before**, picks "Dave", types his PIN, and appears in the lobby as *Dave — returning* ✓. After admission his character, sheet and dice skin are exactly as he left them.

J4 — Exploring in the dark. The party enters an unlit crypt. Mira (an elf with 60 ft darkvision) sees the corridor in grey out to 60 ft. Dave (no darkvision, torch in hand) sees a warm 40-ft pool of light. A goblin waits 50 ft away in darkness, outside Dave's light: Mira sees it; Dave's browser has never received its existence. Mira says "there's something ahead" on Discord and pings the spot.

J5 — A combat round. The DM clicks **Start combat**, selects everyone in the room and chooses *Players roll, I roll NPCs*. Players see an initiative card; Dave rolls with the app, Mira types her physical d20. The portrait strip fills in. On Dave's turn his budget shows 30 ft; he drags, the path goes green then red past 30 ft, and he releases at 25 ft. Before attacking he realises it's a bad spot and clicks **Reset move**: he's back where he started with 30 ft again. He moves 20 ft to the goblin, attacks, steps back 5 ft (the path warns him about an opportunity attack), and clicks **End turn**.

J6 — Fireball. Mira casts Fireball at 3rd level. A 20-ft-radius sphere follows her cursor within a 150-ft range ring; it snaps red where a wall blocks line of effect. She clicks. The fireball explodes (VFX, sound), and a resolution card opens for the DM: 4 creatures caught. Two goblins' DEX saves are rolled by the DM with one click; Dave gets a save prompt and rolls a 14 vs a hidden DC 15, failing. Mira rolls 8d6 = 29. The card shows 29 for failures and 14 for successes, with Dave's fire resistance applied (14). The DM clicks **Apply**. HP bars drain with a trailing ghost bar; one goblin drops and the DM confirms *Dead*.

J7 — Dropping to 0. Dave drops to 0 HP. His token lies down, the Unconscious icon appears, and at the start of his next turn he gets a **Death Saving Throw** prompt. He rolls a natural 20 and regains 1 HP: he's conscious again (still Prone until he stands up).

J8 — Fat finger. The DM accidentally clicks *Hide all* in the fog tools and wipes the revealed map. Ctrl+Z restores it. Later the DM applies Fireball damage twice by mistake; the History panel shows both entries and one click reverts the duplicate.

J9 — PC falls asleep. Xini's PC sleeps mid-session. Players see "Connection lost — the table is asleep. Retrying...". When the PC wakes, clients reconnect automatically and the board is exactly as it was (every accepted change was already on disk). If the tunnel address changed, Xini posts the new link and players rejoin with their PINs.

J10 — Homebrew import with AI. Xini pastes a list of homebrew spells into Claude with the "AI import prompt" copied from Gloam, gets JSON back, and pastes it into **Import spells**. Gloam validates it, shows a dry-run report (12 new, 1 conflict), and imports. Or, with the MCP server connected, Xini just asks Claude to "import these spells into Gloam".

8. Feature specifications

Every feature below has an ID (`F01` ...), a code used in its acceptance criteria (`HOST` , `AUTH` ...), and numbered acceptance criteria (`AC-HOST-01` ...). `tools/extract-features.mjs` turns every line that starts with an AC ID into an entry of `docs/FEATURES.json`. Keep that format exact: a list item that starts with the ID in backticks.

8.1 F01 — Table lifecycle and the doorway (`HOST`)

Purpose. Let the Admin start the server, open the table to friends with one click, and close it again, with no accounts.

Behaviour.

- `pnpm start` launches the production server. It prints a banner: app name and version, data directory, local URL `http://localhost:4747`, table status `CLOSED`, and a **one-time admin login link** (`/admin/magic?token=...`, valid 10 minutes, single use, local-only; see §22.3).
- **First run** (no admin credential in the database): the banner instead prints a **setup link** `/setup?token=...` (valid 30 minutes, single use, local-only). The setup page asks for an admin password (≥ 12 characters, strength meter, confirmation) and then offers "Create demo campaign" (F24).
- **Admin console → Table page** shows:
 - a status card: `Closed`, `Opening...`, `Open`, `Closing...`, `Doorway reconnecting...`;
 - mode selector: *Quick tunnel* (default), *Named tunnel*, *LAN*, *Local only*;
 - **Open table** / **Close table** buttons;
 - when open: the public URL with a copy button, the invite code (large, monospace, grouped `XXXXX-XXXXX`) with copy / rotate / revoke, expiry (until closed (default) / 2 h / 4 h / 8 h), max uses (unlimited by default), a **Lock** toggle (valid codes can no longer knock), **Copy Discord message**, and counters (in lobby, admitted, spectators).
- **Open table** (quick tunnel): generate a fresh invite code; spawn `cloudflared tunnel --no-autoupdate --url http://127.0.0.1:<PORT> --metrics 127.0.0.1:<METRICS_PORT>`; get the hostname by polling `GET http://127.0.0.1:<METRICS_PORT>/quicktunnel` (returns `{"hostname": "..."}`), falling back to parsing stderr with `/https:\/\/[a-z0-9-]+\..trycloudflare\.com/`; wait until `GET /ready` on the metrics port returns 200; then set status `Open` and start session N+1 in the campaign log.
- **Named tunnel**: run `cloudflared tunnel --no-autoupdate run` with the tunnel token passed in the child process's `TUNNEL_TOKEN` environment variable (never on the command line, where other processes could read it), and use the public hostname configured in settings.
- **LAN mode** re-binds the server to `0.0.0.0` (after a confirmation dialog explaining that anyone on the network can reach the join page) and shows `http://<LAN-IP>:4747`. **Local only** opens the table without any doorway (for testing, or players on the same machine).
- **Close table**: revoke all invite codes; broadcast `table.closing`; players see a "The table is closed — thanks for playing" screen; non-admin sockets close; write a `close` snapshot; stop `cloudflared` (SIGTERM, then SIGKILL after 5 s); end the session in the log.
- If `cloudflared` exits unexpectedly while open: restart it up to 3 times with backoff (2 s, 5 s, 10 s). Show `Doorway reconnecting...`. If the quick-tunnel hostname changes, highlight the new URL and offer **Copy Discord message** again.
- If `cloudflared` is not installed (checked with `cloudflared --version`, or the path in settings), show an install card for the detected OS: macOS `brew install cloudflared`; Windows `winget install --id Cloudflare.cloudflared`; Linux: link to the Cloudflare package repository instructions, with Debian/Ubuntu commands. Include a **Re-check** button. Local and LAN modes keep working.
- If `~/cloudflared/config.yaml` exists, warn that quick tunnels won't start while it exists and suggest Named mode or renaming the file.
- Graceful shutdown on SIGINT/SIGTERM (Ctrl+C in the terminal): the same steps as Close table, then flush fog memory, close the database, and exit within 10 s.

Acceptance criteria.

- **AC-HOST-01** On first start with an empty data directory, the terminal prints a one-time setup URL; opening it from `localhost` lets the Admin set a password; the link cannot be reused; the same request carrying a `cf-ray` or `cf-connecting-ip` header is rejected with 403.
- **AC-HOST-02** The server binds `127.0.0.1` by default; LAN mode binds `0.0.0.0` only after explicit confirmation and shows a warning badge while active.
- **AC-HOST-03** Open table (quick) spawns cloudflared, obtains the `trycloudflare.com` hostname within 30 s, and shows status `Open` only after the metrics `/ready` endpoint returns 200.
- **AC-HOST-04** Close table stops cloudflared, revokes all invite codes, shows every non-admin client a "table closed" screen, disconnects them, and writes a `close` snapshot.
- **AC-HOST-05** **Copy Discord message** copies text containing the public URL and the invite code, for example: 🎮 The table is open! Join: `https://calm-river-1234.trycloudflare.com - code 7K2QH-9XM4D` (works until the table closes).
- **AC-HOST-06** With cloudflared missing, the Table page shows OS-specific install instructions and a Re-check button, and Local/LAN modes still work.
- **AC-HOST-07** SIGINT/SIGTERM performs a graceful shutdown (clients notified, snapshot written, tunnel stopped, database closed) within 10 s.
- **AC-HOST-08** Named-tunnel mode runs cloudflared with the stored token passed via `TUNNEL_TOKEN` (not `argv`) and uses the configured public hostname; the token is never shown in full again after saving (masked) and never logged.
- **AC-HOST-09** If cloudflared crashes while open, it is restarted up to 3 times, the status shows `Doorway reconnecting...`, and a changed hostname is highlighted.

8.2 F02 — Joining, identity, lobby and approval (AUTH)

Purpose. Friends get in with one code and no accounts; the Admin/DM decides who sits at the table; returning players keep their characters even when the quick-tunnel link changes.

Identity model. A **profile** = display name (2–24 characters; letters, digits, spaces, `-'_.`), a colour from the 12-colour player palette (§27.2), an optional **PIN** (4–8 digits, stored as an argon2id hash), and zero or more recognised devices (a random `gloam_dev` cookie stored hashed). Profiles are local records in SQLite; they are not accounts with any service.

Join flow (`/join`).

1. **Code step.** A segmented input for the 10-character code (Crockford base32, auto-uppercase, auto-insert the dash, paste-friendly). On submit: `POST /api/join/code`. Wrong code → a generic "That code didn't open the door" message (never reveal whether a code existed or expired).
2. **Identity step.** If the device cookie matches a profile: "Welcome back, Dave" with a **Continue** button (and "Not you?"). Otherwise two tabs: **New here** (name, colour swatches, optional PIN with a note "a PIN lets you rejoin from a new link") and **I've played before** (a list of existing profile names and colours → pick → enter PIN). Profiles without a PIN can be claimed only via device cookie or by Admin approval with a warning "unverified".
3. **Waiting room.** The browser joins the `Lobby` Colyseus room. The screen: a candle-lit door scene (shader flame, soft ambience), the text "Waiting for the DM to let you in...", and three optional activities: **Draw your character** (drawing pad, F10), **Choose your dice** (dice skin, F09), **Test sound**. Pending users receive nothing about the table.

Knock handling. The Admin and DMs (setting *DMs can admit*, default on) get a **knock card** (toast plus Approvals inbox entry) with a door-knock sound: name, colour, *new / returning* ✓ *PIN verified / returning (device) / returning (unverified)*, device label (browser and OS parsed from the User-Agent), time waiting. Actions: **Admit as Player**, **Admit as Spectator**, **Deny**, and — for the Admin only — **Ban**. On admit, the lobby client receives `admitted` and joins the table room without a page reload.

Setting: auto-admit returning players (default off). When on, PIN-verified or device-recognised returning profiles skip the prompt (the knock is still logged).

Kick and ban. Kick sends the person back to the waiting room (they may knock again). Ban blocks the profile and every device associated with it; future knocks are auto-denied with "You can't join this table". Unban from the Admin console.

Reconnection. A dropped socket (Wi-Fi blip, phone lock) reconnects through Colyseus reconnection within a 60 s window without re-approval. After that window, if the table is still open in the same session and the person was admitted in this session, their session cookie lets them rejoin directly. A new session (the table was closed and reopened) requires approval again unless auto-admit is on.

Acceptance criteria.

- **AC-AUTH-01** A wrong invite code shows a generic error; 10 wrong attempts from one client IP within 10 minutes return HTTP 429 for 10 minutes. (Client IP = `cf-connecting-ip` when the socket peer is loopback and the header is present; otherwise the socket address.)
- **AC-AUTH-02** A valid code plus identity puts the person in the waiting room within 1 s, and the Admin/DMs receive a knock card with sound.
- **AC-AUTH-03** Admit moves the person from the waiting room to the table without a reload within 1 s; Deny shows "The DM couldn't let you in right now"; Ban auto-denies all future knocks from that profile and its devices.
- **AC-AUTH-04** A returning player on the same origin is recognised by device cookie; on a new origin they can choose "I've played before", pick their profile and enter their PIN; 5 wrong PINs lock that profile's PIN entry for 15 minutes.
- **AC-AUTH-05** Kick returns the person to the waiting room; kicked or banned sockets are closed by the server within 500 ms.
- **AC-AUTH-06** Pending users' state contains only their own knock status (asserted by inspecting the lobby room state in a test).
- **AC-AUTH-07** A dropped connection reconnects automatically within 60 s without re-approval, restoring the camera position and selection.
- **AC-AUTH-08** Rotating the invite code invalidates the old code immediately; people already inside are unaffected.
- **AC-AUTH-09** With auto-admit on, PIN-verified or device-recognised returning profiles are admitted without a prompt, and the knock is still recorded in the security log.
- **AC-AUTH-10** Invite codes are 10 Crockford-base32 characters (≈ 50 bits of entropy) displayed as `XXXXX-XXXXX`, stored only as SHA-256 hashes, and die when the table closes.

8.3 F03 — Campaigns and scenes (scn)

Purpose. Organise the world into campaigns and scenes, prepare scenes privately, and move the party between them.

Campaign. Name, cover image, rules pack (`srd-5.2.1` default; `srd-5.1` optional pack, see §33.5), display units (ft/m), house rules (§19.6), members and roles, active scene, created/updated timestamps. The Admin chooses which campaign the table runs.

Scene. Name; map (`image` | `model` | `procedural` | `blank`); calibration (feet per pixel for images; transform for models); floor style for procedural/blank (stone flagstones, wooden planks, grass, sand, parchment, cavern rock; all procedural shaders); ambient light (`bright` daylight, `dim` dusk/moonlight, `dark` underground) with a tint colour; fog mode (`off` , `painted` , `dynamic` ; see F08); bounds (auto from map, editable); party spawn point; optional scene music preset; thumbnail (captured automatically from the DM's view); sort order; archived flag.

DM scene tools (DM panel → Scenes). Scene list with thumbnails and drag-to-reorder; **New scene** wizard (source → upload/choose → calibrate → name); duplicate; archive; delete (soft, undoable); **Activate for players**; **Preload** (tells clients to fetch the scene's assets in the background); **Prep view** — open any scene for editing while players stay where they are, with a banner "Only DMs see this scene" (protocol in §13.7: the DM's client loads a full snapshot of that scene and receives its patches; players receive nothing).

Map calibration (image maps). A modal over the uploaded map with three methods: (1) presets "pixels per 5 ft": 50, 70 (default), 100, 140, 200; (2) **Known distance**: drag a line over a feature and type its real length (for example, one printed grid square = 5 ft, or a door = 5 ft); (3) **Map width in feet**. A live ruler preview confirms the scale. Calibration can be reopened later; changing it rescales walls, zones and lights proportionally (tokens keep their world positions relative to the map).

3D maps. A GLB map is placed at the origin. The DM aligns it with a transform gizmo (move / rotate about Y / uniform scale; numeric fields too). **Generate walls** slices the mesh with a horizontal plane at 5 ft (configurable 1–20 ft) using `three-mesh-bvh` shapecast (plane × triangle intersections), merges collinear segments, simplifies with Ramer–Douglas–Peucker (tolerance 0.25 ft), and creates editable walls.

Scene activation. Players see a transition (fade to black, scene name in the display face, fade in), receive only the new scene's entities, and load its assets (with a parchment progress bar if loading takes more than 400 ms). Characters without a token on the new scene are placed around the party spawn point without overlap (spiral placement with 5-ft spacing, avoiding walls).

Acceptance criteria.

- **AC-SCN-01** Creating a scene from an uploaded image opens calibration with the three methods; after calibration a 5-ft reference feature measures $5\text{ ft} \pm 0.1\text{ ft}$ with the ruler.
- **AC-SCN-02** Activating a scene moves every player within 2 s with a transition, and players receive only the new scene's entities.
- **AC-SCN-03** A DM can edit a non-active scene while players remain on the active one; players receive no data from the prep scene (asserted by network inspection in a test).
- **AC-SCN-04** A GLB map can be positioned, rotated and scaled; **Generate walls** produces editable, merged, simplified wall segments from the mesh slice.
- **AC-SCN-05** Procedural floors (stone, wood, grass, sand, parchment, cavern) render with no image asset and tile without visible seams.
- **AC-SCN-06** Newly admitted players' characters without a token on the active scene are auto-placed around the party spawn point without overlapping walls or other tokens.
- **AC-SCN-07** Changing a scene's ambient light or fog mode applies live for all clients within 500 ms.
- **AC-SCN-08** Recalibrating an image map rescales walls, zones, lights and tokens so that they stay aligned with the map art.

8.4 F04 — The board and camera (BRD)

Purpose. A tactile, cinematic 3D tabletop that stays readable.

Behaviour.

- World units: **1 unit = 1 foot**. The map lies on the XZ plane at $y = 0$ (three.js Y-up).
- **Table environment.** The map sits on a large dark wooden table (procedural wood shader: fbm rings, subtle grain, low roughness variation) that fades into darkness beyond 60 ft from the map edge, with a vignette and faint dust motes drifting through a warm key light. The map itself stays crisp and true to its colours.
- **Camera** (drei `CameraControls`, perspective): wheel = dolly toward the cursor; right-drag = orbit (yaw) and tilt (pitch); left-drag on empty board = pan (Select tool); Space + drag = pan with any tool; middle-drag = pan. Pitch clamped to 25° – 90° ; distance clamped 8–400 ft; target clamped to scene bounds + 20%. Presets with smooth 400 ms transitions: **Shift+1** Top-down (90° , optional orthographic toggle **0** for precise measuring), **Shift+2** Tabletop (55° , default), **Shift+3** Low (30°); **T** toggles between top-down and tabletop. (Plain number keys belong to the action hotbar.) **F** focuses the selected token; **Shift+F** follows it.
- **DM Spotlight:** the DM can pull every player's camera to a point (Alt+Shift+click or a button). Players can opt out in settings ("Let the DM move my camera").
- **Performance tiers** (Ultra / High / Medium / Low) chosen from device heuristics (WebGL `MAX_TEXTURE_SIZE`, `navigator.hardwareConcurrency`, `deviceMemory`, touch/mobile, first-second frame times) and adapted at runtime by drei `PerformanceMonitor` (step down after 3 s below target, step up after 10 s comfortably above). Users can pin a tier in settings. Tier details in §24.6.
- **Map variants:** the client picks the largest server-generated variant not exceeding `MAX_TEXTURE_SIZE` and the tier's cap.
- **Loading & transitions:** first load shows the "candle ignition" intro (the flame lights, the board fades in from darkness, HUD elements stagger in 60 ms apart); scene switches use the fade transition from F03.

Acceptance criteria.

- **AC-BRD-01** The board renders the table environment and map at 1 unit = 1 ft, and loads nothing from outside the server (zero CSP violations and zero non-origin requests in the E2E network log).
- **AC-BRD-02** Camera controls work as specified (zoom toward cursor, orbit, pan, presets Shift+1/2/3 and T with 400 ms transitions, pitch and distance clamps, bounds clamp).
- **AC-BRD-03** Performance tiers are auto-selected, adapt at runtime, and can be pinned by the user; Low disables shadow maps, bloom and ambient occlusion and caps DPR at 1.
- **AC-BRD-04** DM Spotlight moves all opted-in players' cameras to the target over 600 ms.
- **AC-BRD-05** The first-load intro and scene transitions show no white flash and no layout jump.
- **AC-BRD-06** Image maps up to $16\,384 \times 16\,384$ source pixels are accepted and displayed using a variant that fits the device's texture limit.

8.5 F05 — Tokens and minis (TOK)

Purpose. Every creature and object on the board, in 3D or 2D, readable at a glance.

Appearance modes.

- **3D model** — a validated GLB mini, standing on a base.
- **Standee** — an image on an upright card with a thin cardboard edge and a small base, billboard around the vertical axis only (it turns to face the camera but never tilts).
- **Coin** — the image in a circular frame lying flat on the map (top-down friendly), with a bevelled rim in the owner/disposition colour.
- **Auto** (default for images) — coin when camera pitch > 70°, standee otherwise, crossfading over 200 ms.

Bases and sizes. A circular base whose **diameter equals the creature's space**: Tiny 2.5 ft, Small 5 ft, Medium 5 ft, Large 10 ft, Huge 15 ft, Gargantuan 20 ft, or a custom value (DM). The base ring shows the owner's player colour (characters) or disposition colour (NPCs: hostile ember-red, neutral brass, friendly verdigris; see §27.2). 3D models are normalised at upload: bounding box computed, grounded at y = 0, centred on the base, facing +Z, scaled so height matches the size category (Tiny 1.5 ft, Small 3.5 ft, Medium 5.5 ft, Large 11 ft, Huge 17 ft, Gargantuan 25 ft), with per-asset overrides for scale, rotation and vertical offset. Idle animations inside a GLB (a clip named **Idle / idle**, or the first clip) loop subtly if present (setting, default on).

Overlay (above each visible token). Name plate (Cinzel small caps), HP bar, and **up to 6 condition/status icons directly under the HP bar** (then "+N"). HP display mode per token: **Exact** (numbers and bar), **Bar** (bar only), **Descriptor** (Healthy = 100%, Hurt = 51–99%, Bloodied = 26–50%, Critical = 1–25%, Down = 0), **Hidden**. Players always see Exact for their own tokens. Defaults: characters Exact; NPCs Bar (campaign setting). The HP bar: current HP fill (verdigris above 50%, brass 26–50%, ember at 25% and below), temp HP as a separate cyan segment after it, a thin tick at 50% (the Bloodied threshold), and a "ghost" segment that lingers 400 ms and drains over 600 ms when damage lands. Overlays scale with zoom within limits and fade out when very far away. Hovering a token for 400 ms shows a DOM tooltip card: portrait, name, HP (per mode), AC (DM/owner only), speeds, conditions with their one-line summaries.

States. Selected (brass ring glow + subtle pulse); hovered (thin outline); active turn (golden ring and a soft light shaft); DM-hidden (DMs see it at 40% opacity with an eye-slash badge; players never receive it); invisible condition (seen only by those who can see it, with a refractive shimmer); prone (3D minis tip over 80° onto their side; standees lie flat); dead (desaturated, lying down, skull icon); flying (lifted to its elevation with a thin stem to a ground shadow ring and a "+15 ft" label).

Interaction. Click to select; Shift+click toggles multi-selection and Shift+drag on empty board draws a selection box (DM); drag to move (F06); double-click to open the sheet (or the unit card for sheetless units); right-click or long-press opens a **radial menu** with actions permitted for the viewer: Sheet, Conditions, Damage/Heal, Light on/off, Elevation, Facing, Target, Hide/Reveal (DM), Lock (DM), Duplicate (DM), Delete (DM). DM copy/paste (Ctrl/Cmd+C, Ctrl/Cmd+V) at the cursor. On your own token, the radial menu also has an **Emote** slice.

Creation. DM: drag from the Library onto the board; **Quick Unit** dialog (name, size, HP, AC, speeds, senses, disposition, image or model) for "any unit with any values"; from the Bestiary (optional SRD monsters pack, §33.5). Players: their character token is created by the DM or auto-placed at the party spawn point.

Linked and unlinked tokens. A **linked** token (the default for player characters) is a view of its actor: HP, conditions, exhaustion, death saves, concentration, speeds, senses and light come from the actor's sheet, so every token of that character in every scene shows the same state. An **unlinked** token (the default for NPCs, Bestiary spawns and Quick Units) carries its own copy of stats and status, so ten goblins spawned from one entry each have their own HP. The DM can switch a token between linked and unlinked in its DM menu (unlinking copies the current values).

Acceptance criteria.

- **AC-TOK-01** Tokens support 3D model, standee, coin and auto modes; switching is immediate and persisted.
- **AC-TOK-02** Base diameters follow size (2.5 / 5 / 5 / 10 / 15 / 20 ft) or a custom value; the base ring shows the owner colour or the disposition colour.
- **AC-TOK-03** Uploaded GLB minis are automatically grounded, centred, oriented and scaled to their size category, with per-asset overrides that persist.
- **AC-TOK-04** The overlay shows the name, the HP bar, and up to 6 condition icons directly under the bar with "+N" overflow; HP display modes Exact/Bar/Descriptor/Hidden work per token.

- **AC-TOK-05** Damage animates a trailing ghost segment (400 ms hold, 600 ms drain); temp HP is a separate segment; a tick marks 50% HP.
- **AC-TOK-06** Right-click or long-press opens a radial menu containing only actions the viewer is permitted to perform.
- **AC-TOK-07** Flying tokens can be raised or lowered in 5-ft steps (Alt+wheel or a HUD stepper) and show a stem, ground ring and elevation label; measured distances include the vertical component.
- **AC-TOK-08** DM-hidden tokens render at 40% opacity with an eye-slash badge for DMs and are never present in any player's state.
- **AC-TOK-09** The Quick Unit dialog creates a sheetless unit with arbitrary values in one submit.
- **AC-TOK-10** Twenty tokens using the same GLB share geometry and materials (renderer memory info shows one geometry set, not twenty).
- **AC-TOK-11** Auto mode crossfades between coin and standee at a camera pitch of 70°.
- **AC-TOK-12** The hover card appears after 400 ms and shows AC only to the DM and the token's owners.
- **AC-TOK-13** Linked and unlinked tokens behave as specified: goblins spawned from one Bestiary entry keep separate HP and conditions; a player character shows the same HP and conditions on every scene; unlinking a token copies the current values; relinking asks before overwriting.

8.6 F06 — Movement and measurement (MOV)

Purpose. Free, gridless movement with tangible distances: a Divinity-style path line that shows cost and remaining movement, enforced for players, optional for the DM. There is **no grid** anywhere in Gloam.

Two modes of play.

- **Exploration** (no combat running): tokens move freely with no budget. The path line and distance label still show during a drag.
- **Combat** (F12): each creature has a **movement budget** for its turn = effective speed (§19.4) + bonuses – movement used. Only the active combatant's owner can move it (unless the DM grants free movement).

Moving a token.

- **Drag** the token. A translucent ghost follows the pointer. The preview path is **routed** from the token's current position to the pointer around movement-blocking walls, impassable zones and (when the creature-space rule applies) the spaces of non-allies that can't be passed, using the shared pathfinder (§16). Holding **Alt** switches to **freehand**: the path follows the pointer's actual trail (simplified with RDP at 0.5 ft) and collisions truncate it.
- **Click-to-move** (setting, default on): with a token you control selected, hovering empty floor shows the routed preview and clicking commits the move. **Ctrl/Cmd+click** adds a waypoint instead; the path is routed through the waypoints in order, and the next plain click (or **Enter**) commits. Clicking another token selects it rather than moving; **Esc** deselects. With the setting off, only dragging moves tokens.
- **The path line** is a ribbon on the floor (0.6 ft wide) with dashes flowing toward the destination. Within budget it is verdigris; beyond budget it becomes ember-red with a hatched pattern. A hollow **max-reach marker** sits at the exact point where the budget runs out. A pill label at the pointer reads, for example, `20 ft · 10 left`, `35 ft (10 difficult) · 5 over`, or `No path`. Difficult-terrain portions are drawn with a double-dash pattern.
- **Opportunity-attack warnings.** In combat, if the preview path leaves the reach (default 5 ft; per-token reach setting) of a hostile creature the mover can see, a small crossed-swords marker appears at the exit point with the tooltip "Opportunity attack from Goblin 2". Hint only; suppressed while the mover has a *Disengaged* marker.
- **Movement range overlay** (selected token you control; toggle with **G**): a soft floor overlay of the geodesic region reachable around walls and difficult terrain — within the remaining budget in combat, or within one move at the creature's speed in exploration — with a bright isoline at the limit (fast-marching field, §16.6).
- **Commit.** On release/click, the client sends `move.commit`. The server validates (§16.5). Over budget: by default the move is **clamped** to the max-reach point (house rule *Overlong moves*: `clamp` | `reject`). A path through a wall the player couldn't see is **truncated** at the contact point, and the player sees a toast "You bump into something unseen".
- **Reset move** (combat; button on the action bar, or Backspace): returns the token to where it started this turn and restores the full budget. Allowed at any time during the owner's own turn under the default house rule *Move reset: always*, as Xini asked (alternatives: `until an action is used` — which stops a move-attack-reset pattern — and `never`). **Ctrl/Cmd+Z** after a move undoes only the last segment and refunds its cost. Players can undo moves only during their own turn; after the turn passes, only

the DM can revert them (History). Outside combat there is no budget, so there is nothing to reset; Ctrl/Cmd+Z undoes the last move.

- **Speeds and modes.** Walk by default. If the creature has fly/swim/climb/burrow speeds, a mode switcher appears on the action bar; flying allows elevation changes; water zones cost double without a swim speed (§19.4). Switching mode mid-turn keeps the movement already used: remaining = the new mode's budget – used (never below 0). **Dash** adds the creature's current speed to the budget. **Stand up** costs half speed; while prone, moving without standing up (crawling) costs double. Speed-zero conditions block movement with an explanation ("Can't move — Grappled"), which the DM can override.
- **Creature spaces** (house rule *Enforce creature spaces*, default on in combat), following SRD 5.2.1 p. 14: a player's move may not **end** with its base overlapping any other creature's base by more than 50% of the smaller diameter. **Passing through** is free for allies' spaces; a non-ally's space can be passed through only if that creature is Tiny, Incapacitated, or two or more size categories larger or smaller than the mover, and such a space (unless the creature is Tiny) counts as difficult terrain; any other non-ally blocks passage. DM moves ignore all of this.
- **DM moves** ignore budgets and blocking. A checkbox "Count as movement" in the DM's token menu charges the budget anyway. The DM can also **Teleport** (place without a path).
- **Others see planning.** While someone drags, viewers who can see that token see its ghost and path (throttled to 15 Hz). Committed moves animate along the path at 30 ft/s visual speed (at least 250 ms, at most 2 s) with easing and a footstep-thump sound.

Measurement tools (everyone): **Ruler** (click-to-add waypoints, shows each segment and the total), **Radius** (circle), **Cone** (53.13° cone, the 5e shape), **Line** (5 ft wide by default, adjustable), **Cube/Square**. Live labels in the campaign's units. By default a finished measurement is shown to everyone for 3 s (toggle "Share my rulers"). Esc clears.

Units. All internal maths is in feet. Display: feet rounded to the nearest 0.5 ft; metres use the 5 ft = 1.5 m convention (1 ft = 0.3 m) rounded to 0.1 m. Comparisons against budgets use an epsilon of 0.05 ft so that a 30.00001-ft path is still 30 ft.

Acceptance criteria.

- **AC-MOV-01** In combat on their own turn, dragging a token shows a routed path around walls with a distance/remaining label; the in-budget part is verdigris, the rest ember-red, with a max-reach marker at the exact budget point.
- **AC-MOV-02** The client preview cost equals the server's authoritative cost within 0.05 ft for 200 randomised shared-test scenarios.
- **AC-MOV-03** Difficult terrain doubles the cost of the part of the path inside it, and the label shows the difficult portion.
- **AC-MOV-04** Releasing beyond budget clamps the move to the max-reach point by default; with *Overlong moves: reject* the move is refused with an explanation.
- **AC-MOV-05** **Reset move** returns the token to its turn-start position and restores the full budget; allowed any time during the owner's turn under the default rule.
- **AC-MOV-06** Ctrl/Cmd+Z after a move undoes only the last segment and refunds its cost.
- **AC-MOV-07** Players cannot move tokens they don't control; in combat they can move only on their own turn unless granted free movement; DM moves ignore budgets unless "Count as movement" is ticked.
- **AC-MOV-08** The server rejects path segments that cross movement-blocking walls; walls hidden from the player truncate the move at the contact point with the "unseen" toast.
- **AC-MOV-09** Dash adds the current speed to the budget; Stand up costs half speed; crawling while prone doubles cost; speed-zero conditions block movement with a message.
- **AC-MOV-10** The movement range overlay shows the reachable region around walls and difficult terrain (remaining budget in combat, speed in exploration), with the limit isoline within 1 ft of the true geodesic limit in test scenes.
- **AC-MOV-11** Ruler, radius, cone (53.13°), line and cube tools show live distances in the campaign's units and are shared to others for 3 s by default.
- **AC-MOV-12** Feet display rounds to 0.5 ft; metres use 5 ft = 1.5 m and round to 0.1 m; all internal maths is in feet.
- **AC-MOV-13** Viewers who can see a token see its drag preview (≤ 15 Hz) and the committed move animating along the path at 30 ft/s (250 ms–2 s).
- **AC-MOV-14** Holding Alt while dragging switches to freehand; collisions truncate the freehand path.
- **AC-MOV-15** A path that leaves a visible hostile creature's reach shows an opportunity-attack marker at the exit point, suppressed while the mover is Disengaged.
- **AC-MOV-16** Creature-space rules are enforced for players when *Enforce creature spaces* is on (ending overlap blocked; allies passable; passable non-ally spaces cost double; other non-allies block), and ignored for DM moves.

- **AC-MOV-17** Ctrl/Cmd+click waypoints route the path through each waypoint in order, the total cost includes all legs, and a plain click or Enter commits.
- **AC-MOV-18** DM bonus movement (+X ft for this turn / N rounds / until removed) adds to the budget per \$19.4 (not doubled by Dash), shows in the budget label ("30 + 10 ft"), and expires as set.

8.7 F07 — Walls, doors and zones (WAL)

Purpose. Tell the engine where creatures can walk and what blocks sight and light; give the DM fast tools to draw them.

Wall kinds and blocking matrix.

Kind	Blocks movement	Blocks sight	Blocks light	Notes
Wall	✓	✓	✓	Default
Door (closed / locked)	✓	✓	✓	Players within reach can open/close unless locked
Door (open)	—	—	—	Drawn as an open door marker
Window	✓	—	—	Glass: see and light pass, bodies don't
Curtain	—	✓	✓	Walk through, can't see through
Invisible wall	✓	—	—	Force field; can be hidden from players
Secret door	✓	✓	✓	Looks like a wall to players until the DM reveals it
Occluder (players' copy of a hidden wall)	—	✓	✓	Not a DM-drawable kind: when a wall is hidden from players but blocks sight, players receive it as an anonymous occluder used only to render vision; movement truth stays on the server

Every wall also has **Hidden from players** (not sent to players at all; movement through it is truncated with the "unseen" toast).

Editor tools (DM, left toolbar → Walls). Draw wall chains (click, click, ... double-click or Esc to finish); endpoint snapping within 1 ft (a snap ring shows); **Shift** snaps the angle to 15°; **Ctrl/Cmd** disables snapping; **Room** tool (drag a rectangle or click a polygon to create an enclosed room); select, move, split and join endpoints; change kind; box-select and bulk edit; delete. A **walls overlay** (DM only by default) draws walls in kind-specific colours; doors show a handle icon at their midpoint.

Doors. Players open/close unlocked doors by clicking the handle icon when their token is within 5 ft of the door segment (edge distance). Locked doors rattle with a lock icon when a player tries. DMs toggle, lock and unlock any door anywhere. Door changes update paths and vision immediately and play open/close sounds.

3D walls. Scene toggle **Walls in 3D** extrudes walls to 8 ft with a procedural stone material (useful for procedural or blank maps and the demo); doors become hinged leaves that swing open over 300 ms; windows render as thin glass with a faint reflection.

Zones (DM, left toolbar → Zones). Polygon, rectangle or circle. Kinds: **Difficult terrain** (movement ×2, drawn with a subtle hatch), **Water** (×2 without a swim speed), **Hazard** (label plus triggers: on enter / start of turn inside / end of turn inside → a DM prompt with an optional save and damage, for example "Burning floor: 1d4 fire"), **Impassable** (blocks movement like walls), **Label** (a named area such as "Altar"). Each zone has a colour, a visibility setting (players see it or not) and an optional note.

Acceptance criteria.

- **AC-WAL-01** The six wall kinds (door with closed/open/locked states) follow the blocking matrix exactly, verified by unit tests on the shared blocking predicates.
- **AC-WAL-02** Wall drawing supports chaining, 1-ft endpoint snapping, Shift angle snapping to 15°, Ctrl/Cmd to disable snapping, the Room tool, and Backspace to remove the last placed segment while drawing.
- **AC-WAL-03** Players can open/close unlocked doors within 5 ft; locked doors refuse with a lock indicator; DMs can toggle any door; paths and vision update within 200 ms.
- **AC-WAL-04** Secret doors are indistinguishable from walls in players' state until revealed (no door flag, no door handle, same kind value as a wall).
- **AC-WAL-05** Zones of each kind behave as specified; hazard triggers create DM prompts at the right moments.

- **AC-WAL-06** "Walls in 3D" extrudes walls with a stone material and animates doors opening and closing.
- **AC-WAL-07** Editing stays responsive (under 16 ms per frame) with 1 000 wall segments in a scene.

8.8 F08 — Vision, light and fog of war (vis)

Purpose. Make light and senses matter, so that players see different things and have to talk to each other; and give the DM simple fog painting when dynamic vision isn't wanted.

Fog modes (per scene).

- **Off** — everyone sees the whole map and all non-hidden tokens.
- **Painted** — the DM paints what players know. Revealed areas (the all-players layer plus the player's own layer) are fully visible, and a non-hidden token is visible to a player if any of its sample points lies in that player's revealed area; everything else is fog (protocol in §15.8).
- **Dynamic** — each player sees what their own character(s) can currently perceive (walls, light, senses). Areas seen earlier stay as **explored memory** (desaturated, dimmed, no creatures shown). The DM's painted reveals also count as explored memory (for example, when the party finds a map of the dungeon).

DM fog tools (left toolbar → Fog). Brush reveal/hide (size slider, soft edge), rectangle, polygon, **Reveal room** (click inside a wall-enclosed region: flood fill bounded by sight-blocking walls on the fog raster), Reveal all, Hide all, and a *Target* selector: all players (default) or specific players. Every fog operation is undoable.

Senses (on every creature's sheet or unit). Darkvision, blindsight, tremorsense, truesight (ranges in ft). Normal sight has no range limit inside lit areas (capped at the scene diagonal). A Blinded creature has no sight (blindsight and tremorsense still work); an Unconscious creature perceives nothing at all.

Light. Light sources can be free-standing (sconce, brazier, campfire) or carried by a token (torch, lantern, Light cantrip on a shield). Each has bright radius, dim radius, colour, intensity, animation (none, torch, candle, pulse, magical shimmer), optional cone (direction and angle, for the bullseye lantern), a *magical* flag, a *pierce darkness* flag (only for lights that overcome magical darkness, such as Daylight over a lower-level Darkness), a *DM-only* flag (a DM vision aid that is invisible to players and ignored by perception), and an enabled toggle. Presets come from the verified table in §34.3. Owners can toggle their token's light and lower a hooded lantern's shutter (bright 0, dim 5 ft). Each scene has an ambient light level (bright/dim/dark).

Obscurement and darkness from effects (F13): heavily obscured areas (Fog Cloud, Stinking Cloud, Sleet Storm...) block sight into and through them; lightly obscured areas (Web, Insect Plague) render a haze but don't block; **magical darkness** (the Darkness spell) blocks darkvision and nonmagical light, and only truesight and blindsight see through it.

What a player sees (dynamic mode), per point on the map. Visible if some creature the player controls (or has shared vision of) has line of sight to it (walls that block sight, heavily obscuring volumes they can't see through) **and** at least one of: the point is in bright or dim light (and not inside magical darkness, unless the light is magical and the rules allow); it is within that creature's darkvision range (and not in magical darkness); it is within blindsight range (ignores light, obscurement and invisibility, but not total cover); it is within truesight range (sees through normal and magical darkness and invisibility). Full rules in §15.3.

Rendering (player view). Bright = full colour. Dim = 55% brightness with a slight cool shift. **Darkvision-only = greyscale** (darkvision sees only in shades of grey in darkness). Explored memory = desaturated, 35% brightness, faint blue tint, no creatures. Unknown = drifting dark "war fog" (animated fbm noise, not flat black) with soft 1–2 ft feathered edges. Reveals animate over 300 ms.

Creatures. A creature is visible to a player if any of its base sample points is visible (§15.4). Invisible creatures are visible only to viewers who perceive them with blindsight, or perceive them by sight while having truesight in range or an active See Invisibility effect, or while they are outlined (Faerie Fire). **Tremorsense** reveals grounded creatures within range as a **sensed** pulse marker without name, appearance or HP. Hidden (DM-hidden) tokens are never visible to players. When a creature leaves a viewer's sight mid-move, that viewer sees it move to the last visible point on the path, then fade (§15.6).

Vision sharing. Off by default (each player sees only what their own character sees). The DM can share any token's vision with chosen players (a familiar, an allied NPC), or turn on campaign-wide party vision.

DM view. DMs see everything. Fog is drawn as a translucent hatched overlay so they can see the map underneath. **View as...** (dropdown of players) previews exactly what that player sees; it changes nothing for anyone else.

Acceptance criteria.

- **AC-VIS-01** Painted mode: brush/rectangle/polygon reveal and hide work (for all players or chosen players); Reveal room fills the wall-enclosed region under the cursor; players see only their revealed areas and the tokens inside them. (Undo of fog operations is covered by AC-UNDO-01.)
- **AC-VIS-02** Dynamic mode: each player sees only what their own character(s) can currently perceive; previously seen areas remain as desaturated memory without creatures.
- **AC-VIS-03** A token carrying a torch lights 20 ft bright plus 20 ft dim, blocked by walls, with flicker; owners can toggle their token's light.
- **AC-VIS-04** In darkness, a character with 60 ft darkvision sees within 60 ft rendered in greyscale; dim light within range renders as bright; unlit areas beyond range are not visible.
- **AC-VIS-05** Two players in the same dark room: A (darkvision) sees goblin G, B (no darkvision, no light) does not; B's client state never contains G (asserted via the test hook and by inspecting WebSocket frames).
- **AC-VIS-06** Magical darkness blocks darkvision and nonmagical light; truesight and blindsight within range see through it.
- **AC-VIS-07** Heavily obscuring effects block sight into and through their area; a viewer inside one sees only their own space; lightly obscuring effects render a haze without blocking.
- **AC-VIS-08** Invisible creatures are visible only to viewers with truesight or blindsight in range, See Invisibility, or while outlined by Faerie Fire.
- **AC-VIS-09** Tremorsense shows a sensed marker (no name, appearance or HP) for grounded creatures in range that are not otherwise seen.
- **AC-VIS-10** DMs see all with a hatched fog overlay; **View as** reproduces a player's rendering exactly and changes nothing for other clients.
- **AC-VIS-11** A viewer who loses sight of a creature mid-move sees it travel only to the last visible point on the path and never receives its destination.
- **AC-VIS-12** Server vision recomputation for 8 viewers, 500 walls and 50 lights takes under 10 ms at p95 on the host; client fog reveals animate over 300 ms.
- **AC-VIS-13** Vision sharing is off by default; the DM can share a token's vision with chosen players or enable party vision.
- **AC-VIS-14** Explored memory persists across server restarts per player and scene; the DM can reset it per player or for everyone.
- **AC-VIS-15** Blinded creatures provide no sight-based vision (blindsight and tremorsense still apply); Unconscious creatures provide no perception at all.

8.9 F09 — Dice (DICE)

Purpose. Fair, visible, delightful dice. Everyone sees results (that's the fun); the DM can still roll behind the screen; physical dice are welcome.

Dice tray (bottom action bar button, hotkey **D**; a bottom sheet on phones): quick buttons d4, d6, d8, d10, d12, d20, d100 (click adds one; right-click removes); count and modifier steppers; **Advantage / Disadvantage** toggles for d20; a formula field with syntax highlighting, autocomplete for **@** references and inline errors; a label field ("Perception", "Longsword"); a visibility selector — players: **Public** (default), **Private to DM**, **Self**; DMs: **Public** or **Private** (only DMs see it; players see "The DM rolls..."). **Blind** rolls (the player rolls but only DMs see the number) come only from DM roll requests; **Roll**; and **I rolled physically...** which opens manual entry (one field per die, or a total). Players' last 10 formulas and pinned macros appear as chips.

Roll feed (bottom-left, collapsible; a tab on phones): cards newest-first with roller portrait and colour, label, formula, per-die chips (kept, dropped struck through, exploded with a spark, natural 20 golden, natural 1 ember), total in large display numerals, and a hand icon for manual entries. Click to expand the full breakdown. Rolls other players aren't allowed to see appear as a masked card ("Mira rolled privately", or "The DM rolls..." for the DM's own secret rolls) with "?" chips (§18.3).

3D dice. After the server decides a roll, every client that may see it plays a physics tumble in a HUD overlay layer (drei **Hud**) so it's readable at any camera angle. Dice are thrown from the roller's side of the screen (bottom for yourself, top edge for others), bounce off invisible tray walls, settle in 1.2–2.5 s, and **land showing exactly the server's numbers** (technique in §18.4). They fade after 2.5 s. More than 20 dice → only the first 20 are physical, the rest appear as chips. The total appears on the card when the dice settle. Settings: skip animation (results appear instantly), reduce motion.

Dice skins (per player): body colour, material (resin, gemstone translucent, metal, bone, obsidian), number colour. Everyone sees a player's dice in that player's skin. Chosen in the waiting room or settings.

Sound. Clacks and rolls are driven by the physics contacts in the simulation (volume from impulse, pitch varied by material), plus a soft "settle" tick and a crit sparkle for natural 20s (§31).

Roll requests (DM). DM panel → Requests, or from the token radial menu: pick target **creatures** (tokens or characters; "all party members" is a shortcut), type (**ability check** with skill, **saving throw** with ability, **attack**, **custom** formula), optional DC (hidden from players by default), advantage state, and visibility of results (Public, Private to DM, or **Blind** — the player rolls but only the DM sees the number, for secret checks like Perception). The controller of each target creature gets a card with the computed formula from their sheet (for example `1d20 + 5 (DEX save)`) and three buttons: **Roll**, **Enter physical roll**, **Skip**. The DM sees a live board: pending / rolled / entered / skipped, each result coloured success/failure against the DC. The DM can **Roll for them** (uses their modifiers), **Set result**, or **Close request**. NPC and unit targets are rolled by the DM with one click.

Rolling from anywhere. Every roll-able number on a sheet, every attack and every spell rolls through the same engine. Alt-click = advantage, Ctrl/Cmd-click = disadvantage; on touch, long-press opens a small menu (Normal / Advantage / Disadvantage / Physical). Conditions add hints ("Poisoned: disadvantage on attack rolls — applied") that the roller can remove.

Acceptance criteria.

- **AC-DICE-01** The formula grammar in §18.1 is fully supported and invalid formulas show inline errors without submitting.
- **AC-DICE-02** All outcomes are generated on the server with `crypto.randomInt`; the client never decides a result.
- **AC-DICE-03** 3D dice for d4, d6, d8, d10, d12, d20 and d100 (two d10s) always come to rest showing exactly the server's values.
- **AC-DICE-04** Public results reach everyone within 300 ms of the server result; Private-to-DM shows to the roller and DMs; Blind shows only to DMs (the roller sees "?"); Self shows only to the roller.
- **AC-DICE-05** Manual entries are recorded with a hand icon and are visible to everyone like normal rolls.
- **AC-DICE-06** Roll requests deliver a card with Roll / Enter / Skip to each target, show the DM live status and success/failure against a hidden DC, and allow the DM to roll for a player.
- **AC-DICE-07** Each player's dice skin is visible to everyone for that player's rolls.
- **AC-DICE-08** Dice sounds are triggered by simulated contacts, with volume scaled by impulse.
- **AC-DICE-09** Limits are enforced: at most 100 dice per term and 500 dice rolled in total (including rerolls and explosions), at most 1000 sides, explosion depth at most 20, formula length at most 200 characters.
- **AC-DICE-10** In test mode only, `GLOAM_TEST_SEED` makes results reproducible; production builds ignore it.
- **AC-DICE-11** Condition-derived advantage/disadvantage hints are applied to relevant rolls and can be removed by the roller before rolling.

8.10 F10 — Character sheets (**SHEET**)

Purpose. A sheet that automates the common D&D numbers and still bends to any homebrew format.

Structure. Two layers (schema in §12.3 and Appendix F):

1. **Core** (used by automation): name, owner, portrait, token art, species/ancestry (text), classes with levels (list), background, alignment, XP, level (derived), proficiency bonus (derived from total level, overridable), six ability scores with modifiers, saving throw proficiencies and bonuses, 18 skills with proficiency level (none / half / proficient / expertise) and bonuses, passive Perception/ Investigation/Insight, AC (value plus note), initiative bonus, speeds (walk, fly, swim, climb, burrow; hover flag), size, HP (max, current, temp), hit dice by die type, death saves, senses (darkvision, blindsight, tremorsense, truesight), damage resistances / immunities / vulnerabilities, condition immunities, current conditions, exhaustion level, Heroic Inspiration, concentration, spellcasting (ability, save DC, attack bonus — derived; slots per level with used counts; pact slots), spells known/prepared, attacks (name, attack bonus formula, damage formula with typed tags, range, properties), carried light source, currency, inventory (name, quantity, weight, equipped, attuned, notes), features and traits (rich text with optional uses: max, used, recharge), languages, proficiencies, notes.
2. **Custom blocks** (for homebrew): an ordered list of blocks the player or DM can add, rename and reorder: **text** (Markdown), **number**, **counter** (value/max, can be *pinned to the token* as an extra thin bar under the HP bar, for example "Sanity 8/10"), **checklist**, **table**

(named columns), **key–value list**, **image**. Custom blocks never feed automation (unless the DM maps a counter to a rule in a later version); they're for everything that doesn't fit D&D's standard shape.

Templates. A DM or player can save a sheet's custom-block layout as a **template** (for example "Homebrew Arcana sheet") and create new characters from it.

Derived values. Modifiers, proficiency bonus, saves, skills, passives, spell DC/attack, initiative and carrying stats are computed; any derived field can be **overridden** (an override badge shows; click to revert to auto).

UI. A side panel on desktop (a full-screen page on phones) styled as parchment (a document surface, §27.2) with tabs: **Overview** (portrait, HP stepper with damage/heal buttons, AC, speeds, conditions, death saves, inspiration), **Abilities & Skills**, **Actions** (attacks and abilities with roll buttons), **Spells** (by level, slot pips, prepare toggles, Cast buttons), **Inventory**, **Features**, **Custom** (homebrew blocks), **Notes**, **Token** (appearance, light source, vision sharing). Clicking any score, save, skill, attack or spell rolls it.

Quick create. One dialog: name, class and level text, max HP, AC, speed, darkvision, portrait/token art. The rest can be filled later.

Ownership and locks. Players own and edit their sheets freely by default. The DM can set per sheet (or campaign default): **Unlocked**, **Core locked** (players may still change play-state fields: current/temp HP, slots used, uses, hit dice used, death saves, conditions, inventory quantities, currency, notes, custom counters; they may not change max HP, scores, proficiencies, AC, speeds, features, spells known) or **Fully locked** (read-only for players). On a locked field, players can **Propose change** → the DM gets an approval with a before/after diff (approve / deny with note).

Import / export. Export any sheet as JSON. Import JSON validated against the published schema (`/api/v1/schemas/character.json`), with a preview and a diff against the existing sheet. **Import with AI...** opens a dialog explaining the no-account workflow: copy the ready-made prompt (it embeds the JSON Schema and conversion rules, Appendix F.3), paste it into Claude together with a photo or PDF of any sheet, paste the JSON reply back into Gloam. The MCP server (F23) can also import sheets directly.

Character art.

- **Drawing pad:** a 1024 × 1024 transparent canvas with pencil, ink (pressure-sensitive width via Pointer Events `pressure`), marker and eraser; 16 swatches plus a picker; size slider; undo/redo (50 steps); optional faint reference layer from a photo; export PNG → asset (queued for approval if a player made it, unless auto-approve images is on) → "Use as portrait / standee / coin".
- **Paper cutout:** for a phone photo of a drawing on paper, run in a Web Worker: honour EXIF orientation, downscale to ≤ 2048 px, estimate the paper colour from border pixels (median), flood-fill from the borders within a tolerance (ΔE in Lab, default 18), remove islands under 0.5% of the area, feather 1 px, trim, and add a white **sticker outline** (default 10 px) and a soft drop shadow. Live preview with sliders (tolerance, outline). The result goes through the normal upload pipeline.

Acceptance criteria.

- **AC-SHEET-01** Quick create makes a playable character (name, class/level, HP, AC, speed, darkvision, art) in one dialog.
- **AC-SHEET-02** Derived values compute correctly (unit tests against worked examples) and every derived field can be overridden and reverted.
- **AC-SHEET-03** Custom blocks of all seven types can be added, renamed, reordered and deleted; counters can be pinned to the token as an extra bar.
- **AC-SHEET-04** Sheet templates can be saved and used to create new characters.
- **AC-SHEET-05** The three lock levels restrict exactly the specified fields; proposals on locked fields reach the DM with a diff and apply only on approval.
- **AC-SHEET-06** JSON import validates against the published schema, shows a preview/diff, and rejects invalid input with readable errors; export → import round-trips losslessly.
- **AC-SHEET-07** The Import-with-AI dialog copies a prompt that contains the current JSON Schema and the conversion rules from Appendix F.3.
- **AC-SHEET-08** Every roll-able element rolls the right formula; Alt/Ctrl modifiers and the touch long-press menu set advantage/disadvantage.
- **AC-SHEET-09** Sheet changes to HP, temp HP, conditions, speeds, senses, size and light update the linked token within 200 ms.
- **AC-SHEET-10** The drawing pad produces a transparent PNG with pressure-sensitive strokes and undo/redo, usable as portrait and token art.

- **AC-SHEET-11** Paper cutout turns a photo of a drawing on white paper into a clean transparent sticker, with adjustable tolerance and outline.

8.11 F11 — HP, damage, conditions and death (HP)

Purpose. Track health and status visibly and correctly, with the DM in control of every consequence.

Damage and healing. Apply from the token radial menu, the sheet, the DM panel, or resolution cards (F13). Damage has a type and can be split into typed parts (for example **12 slashing + 7 fire**). The pipeline follows SRD 5.2.1 (§19.2): per damage instance, bonuses/penalties/multipliers (including "half on a successful save") first, then resistance (halve, round down), then vulnerability (double); immunity means zero. Temporary HP absorb damage first. The DM (and the player, for their own character) sees a preview ("Goblin takes 7 → 0 HP; overflow 3") and can edit it before applying.

Temporary HP. They don't stack: when a creature with temp HP gains more, a prompt offers keep or replace (default: the higher value).

Conditions and status markers. The 15 SRD conditions: Blinded, Charmed, Deafened, Exhaustion (levels 1–6), Frightened, Grappled, Incapacitated, Invisible, Paralyzed, Petrified, Poisoned, Prone, Restrained, Stunned, Unconscious. Status markers: Bloodied, Concentrating, Death saves (at 0 HP), Stable, Dead, Hidden, Surprised, Dodging, Disengaged, Dashing, Heroic Inspiration, Blessed, Banned, Hasted, Slowed, Burning, Flying, Readied, plus **DM custom markers** (name, colour, glyph from the icon set, description, optional duration in rounds). Each has a unique icon (§30, Appendix G), a name and a one-line summary tooltip (conditions summarised from SRD 5.2.1 with attribution). The condition picker is a searchable grid of icons with optional duration and source (for example "Frightened — from Dragon — 1 min").

Automation (all hints or prompts, never silent). Conditions carry metadata (§19.3) that drives hints and some enforcement: speed-zero conditions block movement; Prone changes movement costs; Incapacitated breaks concentration; Blinded removes sight; roll hints apply advantage/disadvantage. **Exhaustion** shows its level on the icon and applies $-2 \times \text{level}$ to d20 tests and $-5 \text{ ft} \times \text{level}$ to speed; reaching level 6 prompts the DM "Dead?". **Bloodied** is applied automatically at or below half HP (campaign toggle).

Concentration. When a concentrating creature takes damage, its owner gets a CON save prompt, $\text{DC} = \max(10, \lfloor \text{damage} \div 2 \rfloor)$, capped at 30 (one prompt per damage instance). Failure ends the concentration and removes linked effects (undoable). Becoming Incapacitated also ends concentration.

Zero HP and death (player characters). At 0 HP a PC gains Unconscious (and Prone), the Death saves marker appears, and the token lies down. At the start of each of their turns (in combat) — or when the DM clicks **Request death save** outside combat — the owner gets a **Death Saving Throw** card (Roll / Enter physical / Skip). DC 10: success/failure pips (3 hearts, 3 skulls). Natural 20 → regain 1 HP (auto-applied with a "Back on your feet!" toast). Natural 1 → two failures. Three successes → **Stable**. Three failures → the DM gets **Mark dead?** (confirm / keep dying). Damage while at 0 HP adds one failure (two from a critical hit). **Massive damage:** if damage reduces a creature to 0 HP and the remaining damage is at least its HP maximum, the DM gets **Instant death?**. Healing at 0 HP restores consciousness and resets death saves. **Stabilise** button (Help action / Spare the Dying) sets Stable.

Zero HP (NPCs/units). The DM gets a small prompt: **Dead** (default; house rule), **Unconscious**, **Keep at 0**.

Rests (DM). DM panel → Party → **Short rest** or **Long rest** for selected characters (or the whole party), as one undoable command. Short rest: each player gets a card to spend Hit Dice (Roll / Enter / Skip per die; healing = die + Con modifier), and features with short-rest recharge reset. Long rest (per the campaign's rules pack; SRD 5.2.1: regain all HP and all spent Hit Dice, Exhaustion –1): spell slots and long-rest features restored, temporary HP removed, death saves cleared. The DM previews the changes per character and can untick anything before applying (P2).

Feedback. Floating numbers coloured by damage type (healing verdigris with a "+"), a short shake and red flash on hit, a warm glow on heal, the death lie-down animation, and matching sounds.

Acceptance criteria.

- **AC-HP-01** The damage pipeline implements the SRD 5.2.1 order exactly (modifiers and save halving → resistance → vulnerability; immunity zero), then temp HP absorbs first; at least 30 table-driven unit tests pass.
- **AC-HP-02** Healing never exceeds max HP; healing a creature at 0 HP restores consciousness and resets death saves.
- **AC-HP-03** Gaining temp HP while having some offers keep or replace, defaulting to the higher value.

- **AC-HP-04** All 15 SRD conditions and all listed status markers can be added and removed from the token menu, the sheet and the DM panel, each with a unique icon, name and summary tooltip.
- **AC-HP-05** Exhaustion shows its level; $-2 \times \text{level}$ applies to d20 tests and $-5 \text{ ft} \times \text{level}$ to speed; level 6 prompts the DM.
- **AC-HP-06** Bloodied is applied and removed automatically at the half-HP threshold when enabled.
- **AC-HP-07** Damage to a concentrating creature prompts a CON save with DC $\max(10, \lfloor \text{dmg}/2 \rfloor)$ capped at 30; failure ends concentration and linked effects, undoably.
- **AC-HP-08** A PC at 0 HP goes Unconscious and gets death save prompts at the start of each of its turns; natural 20, natural 1, 3 successes and 3 failures behave as specified.
- **AC-HP-09** Damage at 0 HP adds a failure (two on a critical hit); massive damage prompts the DM for instant death.
- **AC-HP-10** An NPC reaching 0 HP prompts the DM with Dead / Unconscious / Keep at 0, defaulting to Dead.
- **AC-HP-11** Floating typed damage numbers, hit shake/flash, heal glow and death animation play for everyone who can see the token.
- **AC-HP-12** Every automatic consequence (damage, condition, concentration loss, death) can be edited or skipped by the DM before it applies.
- **AC-HP-13** Short and long rests apply the rules pack's recovery (Hit Dice spending with roll/enter/skip cards; HP, Hit Dice, slots, feature uses, exhaustion and temp HP per §34.2) to the chosen characters as one undoable command with a per-character preview.

8.12 F12 — Combat and initiative (CMB)

Purpose. Structured turns when the DM wants them, free play when they don't, and the DM's hand on every lever.

Starting combat. Quick start (one click, or **Ctrl/Cmd+Shift+C**, which toggles: quick start when no combat is running, stop when one is): combat begins immediately with every non-hidden creature on the scene, using the campaign's default initiative method (house rule, §19.6), no dialog. **Start combat...** opens the full dialog: participants (preselected: selected tokens, or all tokens on the scene the DM chooses; toggles for PCs and NPCs), initiative method — **Roll for everyone, Players roll, I roll NPCs** (default), **Fixed initiative** (10 + the creature's initiative modifier, +5 with advantage, -5 with disadvantage; 2024 optional rule), **Skip rolls — I'll set the order** — plus **Group identical NPCs** (one roll per group of identical creatures) and **Surprised** checkboxes per participant (disadvantage on initiative).

Initiative collection. Players get an initiative card (Roll / Enter physical / Skip) with automatic hints: Invisible → advantage, Incapacitated → disadvantage, Surprised → disadvantage. NPC rolls happen with one DM click. The tracker fills live; the DM can **Roll remaining**, type any value, or **Begin** at any point (unrolled combatants go last, ordered by Dex modifier). Ordering: initiative descending; ties by Dex modifier, then PCs before NPCs, then name (house rule can change tie-break); the DM can drag to reorder at any time.

Turn tracker. A portrait strip at the top centre: circular portraits in order with initiative numbers, the active one larger with a golden frame, round counter ("Round 3"), small HP bars per the HP display rules, and a divider showing where the round wraps. Players only see combatants they can currently perceive (others appear as a dark "Unknown" silhouette, or not at all, per the DM's setting *Reveal hidden combatant count*). On phones the strip condenses to current + next two.

Turn flow. On turn start: the creature's movement budget resets per §19.4 (effective speed plus any DM bonus movement); action pips reset (Action, Bonus Action, Reaction, Object interaction; Reaction resets at the start of the creature's own turn); start-of-turn processing runs (death save prompt at 0 HP; hazard zone and effect triggers; ongoing damage such as Burning; expiring effects "until the start of your next turn"); the owner hears a "your turn" chime and sees a banner; the camera focuses on the creature if the player opted in. **End turn** (owner or DM) runs end-of-turn processing (end-of-turn saves, for example "Hold Person: repeat the save"), then advances. The round increments after the last combatant. The DM has **Next**, **Previous**, **Delay** (move a combatant later in the order), **Add combatant** (with an initiative roll prompt), **Remove**, **Set initiative**, and **Stop combat**.

Enforcement. During combat, players can move only the active combatant they control, within budget (F06), unless the DM grants free movement to a token or to everyone. Players can still measure, ping, emote, open sheets, roll dice (for example, reactions), and plan.

Action economy pips. On the action bar for the active creature: Action ●, Bonus Action ▲, Reaction ◆, Object interaction ■, plus the movement bar. App-driven attacks and spells mark them automatically according to casting time; players and the DM can toggle them manually. Pips are a tracker, not a jail (P2): using an already-used pip shows a gentle "already used — continue?" confirmation.

Stopping. Stop combat returns to exploration (no budgets, no turn restrictions) and writes a log entry: rounds, knocked-out creatures, damage dealt and taken per combatant.

Acceptance criteria.

- **AC-CMB-01** The start dialog supports all four initiative methods, participant selection, grouping identical NPCs, and per-participant surprise.
- **AC-CMB-02** Player initiative cards offer Roll / Enter / Skip with automatic advantage/disadvantage from Invisible, Incapacitated and Surprised.
- **AC-CMB-03** Identical NPCs can share one initiative roll.
- **AC-CMB-04** The tracker shows portraits in order, the round number and the active highlight; players don't see combatants they can't perceive.
- **AC-CMB-05** Turn start resets the budget and pips, runs start-of-turn prompts, plays the chime for the owner, and focuses the camera if opted in.
- **AC-CMB-06** During combat only the active combatant's controller can move it (unless free movement is granted); End turn advances; Next/Previous/Delay/Add/Remove/Set initiative/drag-reorder work for the DM.
- **AC-CMB-07** Stop combat returns to exploration and logs a combat summary.
- **AC-CMB-08** Action pips track usage, are marked automatically by app-driven attacks and spells, can be toggled manually, and the Reaction resets at the start of the creature's own turn.
- **AC-CMB-09** Effect durations tick in rounds relative to their creator's turn and expire with a notification.
- **AC-CMB-10** Ties are broken by Dex modifier, then PCs before NPCs, then name, unless a house rule says otherwise.
- **AC-CMB-11** Quick start begins combat in one click with all non-hidden creatures on the scene and the default initiative method, and Stop combat returns everyone to free movement in one click.

8.13 F13 — Spells, abilities and effects (SPL)

Purpose. Every SRD spell available and castable with the right area on the board; homebrew as a first-class citizen; the DM in control of resolution.

Content. The SRD 5.2.1 pack (339 spells, \$33) loads at startup. Homebrew spells live in the campaign database. A **Spell browser** (DM panel, sheet Spells tab and a global search) with filters: level, school, class, casting time, concentration, ritual, damage type, save ability, area shape, source (SRD / homebrew).

Spell card. A parchment card: name, level and school, casting time, range, components, duration (concentration badge), description (Markdown), "Using a Higher-Level Spell Slot" / cantrip upgrade text, and a structured strip (area, save, damage by slot level) when known. SRD cards carry a small attribution line.

Casting flow.

1. Cast from the sheet (Spells tab or the hotbar). Choose a slot level if levelled (default: lowest available; upcasting shows scaled dice). Rituals and "free cast" can skip slot use.
2. **Targeting**, based on the spell's data: - **Area** spells: a translucent template follows the pointer with the caster's range ring drawn on the floor. Spheres and cylinders are centred on the pointer; cones and lines originate at the caster's base edge and aim at the pointer; cubes attach to the caster's edge for self-origin spells or follow the pointer for ranged ones (rotate with  or the mouse wheel in 15° steps; on touch, a rotate handle); emanations centre on the caster and follow them. The template turns ember-red when out of range or when the point isn't reachable by line of effect. Affected creatures highlight live. - **Targeted** spells: click up to N creatures (N from the spell and slot); range and line of sight are checked; a counter shows "2/3 targets". - **Self** spells apply immediately. - **Narrative cast**: skip targeting and just post the spell card to the log (P2).
3. Confirm. The VFX plays immediately for everyone who can see the area, and a **resolution card** opens.

Resolution card (DM sees the full card; the caster sees their parts; other players see a public summary: "Mira casts Fireball (3rd level) — 4 creatures"):

- Targets list (auto-detected, §17.3). The DM can add or remove targets.
- **Attack rolls** (spell attacks): the caster rolls per target; the DM sees hit/miss against AC (AC hidden from players); critical hits double damage dice.

- **Saving throws:** target players get save cards (Roll / Enter / Skip) showing the ability, not the DC (unless the DM reveals it); NPC saves roll with one DM click using their save bonuses; results show success/failure.
- **Damage/healing roll:** the caster rolls (or the DM enters a number). Scaled by slot level or character level from the spell data.
- **Per-target outcome:** full / half / none (from the spell's save effect), resistances/vulnerabilities/immunities (auto, each toggleable), conditions to apply (checkboxes, for example Hold Person → Paralyzed), final numbers editable.
- **Apply / Apply all / Skip target / Cancel cast (refund slot).** Applying runs the HP pipeline (F11), concentration prompts and 0-HP flows, and marks action pips.

Concentration. Casting a concentration spell while concentrating asks to end the previous one. Effects created by a concentration spell are linked and end when concentration ends.

Persistent effects. Spells with duration and area create an **effect** on the board: shape, position/orientation, attached token (emanations and "moves with the object" areas), remaining duration, concentration link, properties (difficult terrain, light, heavy/light obscurement, magical darkness, blocks sight like an opaque wall), triggers (on enter, start of turn inside, end of turn inside, per 5 ft moved inside) that create DM prompts with the configured save and damage, visibility (everyone / DM only), and movement rules (for example Moonbeam: the caster can move it up to 60 ft; Cloudkill drifts 10 ft away each turn). The effects that must work out of the box (they're in the SRD and test the engine): **Fog Cloud, Darkness, Daylight, Light, Spirit Guardians, Moonbeam, Web, Spike Growth, Sleet Storm, Stinking Cloud, Cloudkill, Wall of Fire, Silence, Faerie Fire, Flaming Sphere, Call Lightning.**

Weapons and abilities. Attacks and features on sheets use the same resolution card (attack → damage → apply).

Cover hint. For attacks and saves, the card shows a cover hint computed from walls between attacker (or area origin) and target (§17.5): none / half (+2) / three-quarters (+5) / total. It is a hint; the DM decides.

Homebrew builder. A form with every field of the spell schema, live card preview, and validation. Players can **propose** homebrew (DM approval). The DM can duplicate an SRD spell as a starting point ("Poison Ball" from Fireball: change the damage type, VFX preset and name).

Import. Import spells dialog (paste or upload JSON; dry run → report of valid/invalid/conflicts → choose skip/overwrite/rename → import), the REST endpoint `POST /api/v1/content/spells:import`, and the MCP tool `import_spells`. "Copy AI prompt" gives a ready prompt (Appendix F.2) for converting any text list into valid JSON.

VFX presets (§24.5): fire, cold, lightning, thunder, acid, poison, necrotic, radiant, force, psychic, healing, arcane. Each has a cast burst, a projectile or instant form, and a subtle persistent loop for lasting areas. The preset is chosen from the damage type and can be overridden per spell.

Acceptance criteria.

- **AC-SPL-01** The SRD 5.2.1 pack loads exactly 339 spells with per-level counts 27, 57, 57, 42, 34, 38, 31, 20, 17, 16 (cantrip → 9th), asserted at startup and in tests.
- **AC-SPL-02** The spell browser searches and filters by level, school, class, casting time, concentration, ritual, damage type, save and area shape.
- **AC-SPL-03** Casting chooses a slot (with upcast scaling), shows area templates with range rings and line-of-effect validation, supports multi-target selection, and applies self spells immediately.
- **AC-SPL-04** Area targeting detects affected creatures per §17.3, and the DM can add or remove targets on the card.
- **AC-SPL-05** The resolution card handles attack rolls, save requests, NPC one-click saves, damage/healing rolls, half/none on save, resistances/vulnerabilities/immunities, conditions, editable finals, and Apply/Apply all/Skip/Cancel & refund.
- **AC-SPL-06** Slots are consumed on cast and restored on cancel or undo; rituals and free casts can skip slot use.
- **AC-SPL-07** Casting a second concentration spell prompts to end the first; ending concentration removes linked effects.
- **AC-SPL-08** The 16 named persistent effects behave as specified (duration, attachment, movement, light/obscurement/darkness integration with vision, difficult terrain, trigger prompts).
- **AC-SPL-09** Each of the 12 VFX presets plays on cast, and persistent areas animate subtly within the performance budget.
- **AC-SPL-10** The homebrew builder validates every field with a live preview; players' proposals require DM approval; SRD spells can be duplicated as templates.
- **AC-SPL-11** JSON import via the UI and REST validates against the published schema, supports dry runs, and resolves conflicts by skip/overwrite/rename. (The MCP path is covered by AC-API-04.)

- **AC-SPL-12** Weapon attacks and features use the same resolution card flow.
- **AC-SPL-13** The cover hint appears on attack and save cards and matches the §17.5 algorithm in unit tests.

8.14 F14 — Undo, redo and history (UNDO)

Purpose. Mistakes are cheap.

Behaviour. Every mutating command records a history entry with forward and inverse operations (§14.4). **Ctrl/Cmd+Z** undoes the user's own most recent undoable action; **Ctrl/Cmd+Shift+Z** or **Ctrl+Y** redoes it. If someone else changed the same field afterwards, a player's undo is refused with an explanation ("Can't undo: the DM changed Goblin 2's HP since"); the DM gets **Undo anyway** (applies the inverse to those fields as they are now). The **History panel** (DM) lists entries with actor, summary, scene, time, and filters by person, type and scene; each entry has **Revert**; **Restore to here** reverts every later entry in reverse order after a confirmation that lists what will change. Not undoable: dice rolls (facts), joins, kicks, bans, approvals, emotes, pings, music playback.

Acceptance criteria.

- **AC-UNDO-01** Ctrl/Cmd+Z and Ctrl/Cmd+Shift+Z (or Ctrl+Y) undo and redo the user's own token moves, token edits, fog operations, wall edits, damage/healing applications, condition changes and sheet edits.
- **AC-UNDO-02** A conflicting undo is refused for players with an explanation; DMs can force it.
- **AC-UNDO-03** The History panel lists and filters entries; Revert and Restore-to-here work with a confirmation summary.
- **AC-UNDO-04** An accidental fog wipe, a deleted token and a doubly applied damage can each be restored in at most two clicks.
- **AC-UNDO-05** Dice rolls, joins, kicks, bans, approvals, emotes, pings and music playback are not undoable.

8.15 F15 — Persistence, saves and recovery (PER)

Purpose. It's like a multiplayer game save that you never have to think about.

Behaviour. Every accepted command is written to SQLite in the same transaction as its history entry **before** it is broadcast (write-through; this is the autosave). Explored-fog memory is batched and flushed every 5 s and at shutdown. **Snapshots** (full campaign state, gzip JSON in `data/snapshots/`): automatic every 10 minutes while the table is open (keep the last 24), on scene activation, on Close table and on shutdown; manual **Save now** with a name; **Restore** any snapshot (a `pre-restore` snapshot is taken first). **Backups**: daily SQLite online backup to `data/backups/` (keep 14) plus **Backup now**. **Export campaign** to a single `.gloam` file (zip: `campaign.json` + `assets/`), and **Import campaign** on another machine (every asset re-runs the full validation pipeline). Clients that lose their connection show a non-blocking banner "Connection lost — reconnecting..." with a small candle animation, and resynchronise without reloading when the server comes back.

Acceptance criteria.

- **AC-PER-01** Every accepted command is persisted in the same SQLite transaction as its history entry before any broadcast (verified by an integration test that inspects the database inside the broadcast hook).
- **AC-PER-02** After SICKILL mid-session and a restart, the board, tokens, combat state, sheets, effects and fog reveals are intact; at most the last 5 s of explored memory may be missing.
- **AC-PER-03** Automatic snapshots run every 10 minutes (last 24 kept), on scene activation, on close and on shutdown; manual named snapshots work; restoring creates a pre-restore snapshot first.
- **AC-PER-04** A daily online backup is written (14 kept) and Backup now works while the table is open.
- **AC-PER-05** Export produces a `.gloam` file that imports on a fresh install with identical content; imported assets are re-validated.
- **AC-PER-06** Clients show a reconnecting banner on disconnect and resynchronise without a reload after the server returns.
- **AC-PER-07** Crash recovery end to end: kill the server process mid-session, restart, open the table; old invite codes no longer work, the previous session is closed in the log as ended unexpectedly, and a returning player who knocks with their PIN and is admitted gets the same character, sheet, position and dice skin.

8.16 F16 — Assets and uploads (AST)

Purpose. Let people bring maps, minis, drawings and music without letting anything harmful onto the host's PC.

Accepted types (detected by magic bytes, never by extension or client MIME).

Class	Formats	Size cap	Processing
Image	PNG, JPEG, WebP, GIF (first frame), AVIF	Maps: 80 MB and 268 megapixels (16 384 ²). Tokens, portraits, handouts: 25 MB and 50 megapixels	Re-encoded, metadata stripped, variants generated (§21.3)
3D model	GLB (binary glTF, self-contained)	60 MB	Validated, stripped, optimised, re-validated (§21.4)
Audio	MP3, OGG (Vorbis/Opus), WAV, M4A/AAC, FLAC	50 MB	Magic-byte and container sanity checks, stored as-is

Not accepted: SVG (script risk), multi-file glTF, FBX/OBJ/STL (the upload UI explains "export as .glb from Blender"), archives (except **.gloam** campaign imports by the Admin).

Approval. Player uploads are **pending** until a DM or the Admin approves them in the Approvals inbox (image preview, or a 3D preview rendered from the optimised GLB, with file stats). DM/Admin uploads are approved automatically. Setting **Auto-approve player images** (default off). Rejected files are deleted after 24 h.

Quotas. Players: 200 MB of pending plus approved assets each. People still in the lobby: 20 MB, character-art images only. DM/Admin: unlimited.

Library panel. Tabs Minis (3D), Tokens (2D), Maps, Audio, Handouts; search; filter by tag, uploader and status; editable tags and names; drag onto the board to create a token or a scene; asset detail with usage count; soft delete (undoable).

Acceptance criteria.

- **AC-AST-01** Only the listed formats are accepted, detected by content; everything else is rejected with a readable reason.
- **AC-AST-02** Size caps and player quotas are enforced while streaming (the upload is aborted as soon as a cap is exceeded).
- **AC-AST-03** Images are re-encoded with metadata stripped into variants; GLBs are validated (0 errors, no external URIs), stripped, optimised and re-validated; processing runs in a separate child process (not a worker thread) with memory and time limits, so a crash or runaway native allocation can't take the server down.
- **AC-AST-04** Player uploads enter the approval queue; DM/Admin uploads are auto-approved; auto-approve player images works when enabled.
- **AC-AST-05** Assets are served only to admitted table members, with immutable caching, fixed Content-Type, **nosniff** and a sandbox CSP.
- **AC-AST-06** The Library supports search, filters, tags, drag-to-board and soft delete.
- **AC-AST-07** A fuzz suite of malformed files (truncated PNG, zip renamed to .glb, polyglot image, decompression bomb, GLB with an external URI, GLB with an embedded script-like extension) is rejected without crashing or hanging the server.

8.17 F17 — Audio and music (AUD)

Purpose. Sound makes the table feel alive; the group is on Discord voice, so Gloam's audio is effects, music and ambience, never voice.

Channels. Master, Dice, Effects (combat, spells, doors, footsteps), UI (clicks, notifications, knocks), Music, Ambience. Each has a volume slider and mute in the settings popover (per device, stored in **localStorage**). A global **mute** toggle sits in the top bar.

Sound effects are synthesized at runtime with ZzFX and small Web Audio graphs (recipes in §31): no audio files are needed for any built-in sound. Board sounds are panned by screen position and attenuated by distance from the camera target.

Music (DM). Upload tracks (F16) → build playlists → **Play / Pause / Next / Previous / Loop / Shuffle / Stop**, crossfade 2 s, per-track volume. The server holds the playback state (track, server start time, paused position, volume); every client plays in sync using the clock offset (§25.3); late joiners start at the current position. Scenes can have a music preset that starts on activation (optional).

Ambience (DM). Procedural layers — rain, wind, fire crackle, flowing water, cave drips, night insects — each with a level slider. The DM's mix is broadcast; each client synthesizes locally. Presets: *Crypt*, *Forest night*, *Storm*, *Tavern hearth* (fire + low murmur of filtered noise), *Cave*.

Generative music (DM). So the table has music on night one without uploading anything, include four procedural music presets built with Web Audio (§25.5): **Dungeon drone** (slow evolving minor pad and low pulses), **Tavern** (plucked pentatonic arpeggios with soft hand-drum), **Battle** (tense ostinato with low drums, tempo 110), **Wonder** (bright pad with bell motifs). Each plays indefinitely without

obvious loops, crossfades with the others and with uploaded tracks, and syncs like tracks do. Phase 0 task R4 also lists CC0 music packs the DM could import (none are bundled).

Audio unlock. Browsers block audio until a user gesture. The first click in the app (join, admit screen, or anything) resumes the AudioContext. If audio is still blocked, a small speaker chip "Tap to enable sound" appears in the top bar.

Acceptance criteria.

- **AC-AUD-01** All built-in sound events in §31 play from runtime synthesis with no audio files, routed through the six channels with volume and mute.
- **AC-AUD-02** The DM music player supports upload, playlists, play/pause/next/previous/loop/shuffle/stop and a 2-s crossfade; every client stays within 75 ms of the server timeline (so within 150 ms of each other); late joiners start at the current position.
- **AC-AUD-03** The six ambience layers are synthesized procedurally, mixed by the DM, and heard by everyone.
- **AC-AUD-04** Audio unlocks on the first user gesture, with the enable-sound chip as a fallback.
- **AC-AUD-05** Board sounds are stereo-panned by screen position and attenuated by distance.
- **AC-AUD-06** The four generative music presets play continuously without audible loop points for at least 10 minutes, crossfade with each other and with uploaded tracks, and stay in sync across clients like tracks do.

8.18 F18 — Table flavour: emotes, pings, handouts and the log (FUN)

Purpose. The little social things that make remote play feel like sitting at a table.

- **Emote wheel** (hotkey **E**; on touch, long-press your own portrait in the party list or action bar, or pick **Emote** in your token's radial menu): 12 emotes (laugh, gasp, thumbs up, clap, heart, fire, skull, thinking, sleepy, popcorn, facepalm, party) and 8 quick phrases ("Nat 20!", "Wait for me!", "Let's go!", "I have a plan...", "Bad idea.", "Nice roll!", "Is it my turn?", "BRB"). Each player can add up to 6 custom phrases (≤ 40 characters). An emote pops above the player's token (or their portrait if they have no token visible) for 2.5 s with a bouncy animation and a tiny sound, and appears in a transient feed. Rate limit: 1 per 1.5 s, bursts of 3.
- **Ping**: Alt+click (desktop) or long-press on empty board (touch) → an expanding ring in the player's colour at that spot, visible to everyone with a soft ping sound. **DM Spotlight ping** (Alt+Shift+click) also pulls cameras (F04).
- **Hand raise** (**H**): toggles a raised-hand badge on the player's portrait in the party list and in the turn tracker; the DM hears a soft chime once.
- **Handouts**: the DM creates handouts (title, Markdown text, optional image) and **Shows** them to everyone or selected players: a parchment card unfurls on their screens with a paper sound. Players keep received handouts in a Handouts list. The DM can also send a **secret note** to one player ("Only you notice the glyph glowing").
- **Campaign log (journal)**: automatic entries (session opened/closed, scene changes, combat summaries, deaths and stabilisations, level changes on sheets, handouts shown) plus manual entries by the DM and players ("Session recap"). Grouped by session; searchable; **Export as Markdown**. Readable through the API and MCP (F23), so Xini's own Claude can write recaps.

Acceptance criteria.

- **AC-FUN-01** The emote wheel offers 12 emotes, 8 quick phrases and up to 6 custom phrases; emotes display for 2.5 s above the token or portrait with animation and sound; the rate limit applies.
- **AC-FUN-02** Pings appear for everyone in the sender's colour; the DM Spotlight ping also moves opted-in cameras.
- **AC-FUN-03** Hand raise shows on the portrait and the tracker and notifies the DM once.
- **AC-FUN-04** Handouts and secret notes reach exactly the chosen recipients with the reveal animation and stay in their Handouts list.
- **AC-FUN-05** The campaign log records the automatic events, accepts manual entries, groups by session, and exports Markdown.

8.19 F19 — DM control panel (DMP)

Purpose. Everything the DM needs, two clicks away, without leaving the board.

Layout. On the right dock, a **DM** tab with a vertical icon rail of sections (tooltips, and a search box "Jump to..." with **Ctrl/Cmd+K**):

Section	Contents
Scenes	List, new, prep view, activate, preload, calibrate, ambient light, fog mode, spawn point
Tokens & Units	Tokens on scene (list with filters), Quick Unit, Bestiary, bulk hide/reveal, disposition, HP display modes
Vision & Fog	Fog tools, explored-memory resets, vision sharing, View as...
Walls & Zones	Tool shortcuts, walls overlay toggle, 3D walls toggle, zone list
Lights	Light list, presets, add free light, global ambient
Combat	Start/stop, tracker controls, initiative edits, round effects list
Requests	New roll request, open requests board
Effects	Active effects list (duration, concentration link, move, end)
Library	Assets (upload, tags, approvals shortcut)
Sound	Music player, playlists, ambience mixer
Party	Characters with HP, conditions and passive scores at a glance; short and long rests (F11); request death save; grant Heroic Inspiration
Handouts & Notes	Handouts, secret notes, DM notes per scene and per token (never sent to players)
Approvals	Inbox: lobby knocks, uploads, sheet proposals, homebrew proposals (badge count)
History	Timeline with revert/restore
House rules	Campaign rule toggles (§19.6)

Per-token overrides (token radial menu → DM, or the Tokens list): speed override; bonus movement (+X ft for this turn / N rounds / until removed); free movement; lock movement; ignore condition speed effects; share vision with...; reveal (by vision — the default; always to everyone; always to chosen players) and hide (DM only); linked/unlinked; HP display mode; sheet lock level.

Act as. The DM (or Admin) can **take control** of any character to move, roll, cast and use its sheet on the player's behalf, for players who are stuck or away. Actions are logged as "DM as Dave's Thorin".

Acceptance criteria.

- **AC-DMP-01** Every DM section listed is reachable within two clicks or taps, and **Ctrl/Cmd+K** jumps to any section or command by name.
- **AC-DMP-02** All per-token overrides work and are shown as badges in the DM's token hover card.
- **AC-DMP-03** Act as lets the DM control any character; resulting log entries and history entries record "DM as".
- **AC-DMP-04** The Approvals inbox aggregates knocks, uploads, sheet proposals and homebrew proposals with a live badge count.
- **AC-DMP-05** DM notes (per scene and per token) are never sent to players (asserted in a test).

8.20 F20 — Admin console (ADM)

Purpose. Run the server, the door, the people and the data.

Route `/admin` (full-page console; also reachable as a tab inside the table for the Admin). Sections: **Table** (F01), **People** (profiles, status, last seen, role per campaign, rename, set/clear PIN, assign DM, kick, ban/unban, delete profile with character reassignment), **Campaigns** (create, rename, archive, delete with confirmation, export/import, choose the table's campaign), **Saves & Backups** (snapshots list with restore, backups, backup now, data directory size), **Assets** (all campaigns, storage use, orphan cleanup), **Content** (packs enabled per campaign, homebrew overview), **API & MCP** (tokens, scopes, "Connect Claude" instructions), **Settings** (port, LAN, tunnel mode and token, cloudflared path, defaults for new campaigns, auto-admit returning players, DMs can admit, auto-approve images, allow admin login through the doorway, allow remote API — the port, LAN, tunnel and cloudflared-path settings can only be changed from the host PC itself, §22.3), **Security log**, **About & Credits** (version, SRD attribution text, third-party licences).

A **first-run checklist** card sits on top until complete: set password ✓, install cloudflared, create or import a campaign, add a map, open the table.

Acceptance criteria.

- **AC-ADM-01** All sections listed exist and work end to end.
- **AC-ADM-02** People management supports rename, PIN set/clear, DM assignment, kick, ban/unban and delete with character reassignment.
- **AC-ADM-03** The Admin has every DM capability in every campaign plus the admin-only actions; admin sessions expire after 12 hours of inactivity; by default the Admin can log in only from the host PC (setting: allow admin login through the doorway).
- **AC-ADM-04** The Security log records logins, failed logins, knocks, approvals, denials, kicks, bans, invite rotations, API token use and rejected uploads, with time and client IP.
- **AC-ADM-05** About & Credits shows the exact SRD 5.2.1 attribution statement (Appendix I) and the licences of bundled fonts and libraries.
- **AC-ADM-06** The first-run checklist tracks the five steps and disappears when all are done.

8.21 F21 — Responsive and touch (RSP)

Purpose. Play from a phone on the sofa or a tablet at the table.

Breakpoints. Phone < 640 px wide; tablet 640–1023 px; desktop ≥ 1024 px. Phone landscape is supported (compact HUD).

Phone layout. The board fills the screen. Top: a compact turn strip (current + next two) and a status chip. Bottom: a tab bar — **Board, Sheet, Dice, Log, More** — and a floating **End turn** button on your turn. Panels open as bottom sheets with snap points at 30%, 60% and 95% and a drag handle. The DM on a phone gets the essentials (move/hide tokens, combat controls, fog brush, requests); full editing is best on tablet or desktop, and the UI says so politely.

Gestures (touch). One finger on a token: drag to move. One finger on empty board: pan. Pinch: zoom. Two-finger twist: orbit. Two-finger vertical drag: tilt. Tap: select. Long-press: radial menu (on a token) or ping (on empty board). Double-tap: focus. Pen input uses pressure in the drawing pad. Pointer capture keeps drags alive when the finger leaves the token.

Acceptance criteria.

- **AC-RSP-01** Layouts work at 360×740, 390×844, 844×390, 768×1024, 1024×768, 1440×900 and 1920×1080 with no horizontal scrolling and no overlapping controls (screenshot review).
- **AC-RSP-02** All listed gestures work (Playwright touch emulation tests for drag-to-move, pinch zoom and long-press menu).
- **AC-RSP-03** Touch targets are at least 44 × 44 px; HUD text is at least 12 px.
- **AC-RSP-04** In Playwright WebKit (iPhone) and Chromium (Pixel) emulation: audio unlocks on the first gesture, the layout uses 100dvh, the board doesn't rubber-band scroll, and drags keep pointer capture; `docs/HOSTING.md` includes a short manual checklist for real phones.
- **AC-RSP-05** On the Low tier at DPR 1 the benchmark scene (§37) holds at least 30 fps in **headed** Chromium on the host GPU with 4× CPU throttling (a proxy for a mid-range phone; headless WebGL is software-rendered and not representative).

8.22 F22 — Settings and accessibility (A11Y)

Purpose. Comfortable for everyone at the table.

- **Per device** (settings popover): volumes, graphics tier, UI scale (90–130%), reduced motion (defaults to the OS setting), colour-blind palette, "Let the DM move my camera", "Focus camera on my turn", "Share my rulers", dice animation on/off, units display override.
- **Keyboard:** every DOM control is reachable with Tab, has a visible brass focus ring, and icon buttons have ARIA labels. `?` opens the shortcut sheet (Appendix H).
- **Reduced motion** disables camera flights (jump cuts instead), screen shake, dice tumbling (results appear as chips), and parallax.
- **Colour-blind palette** swaps path, HP and disposition colours to an Okabe–Ito-derived set and adds patterns: dashed segments for out-of-range paths, striped fill for low HP.
- **Visual equivalents for sound:** every sound event also has a visual cue (toast, badge or animation).

Acceptance criteria.

- **AC-A11Y-01** All DOM UI is keyboard-operable with a visible focus ring, and icon-only buttons have accessible names (axe-core check on the main screens has no serious violations).
- **AC-A11Y-02** Reduced motion disables camera flights, shake, dice tumbling and parallax.
- **AC-A11Y-03** The colour-blind palette swaps the specified colours and adds the patterns.
- **AC-A11Y-04** UI scale works from 90% to 130% without layout breakage; body text contrast is at least 4.5:1.
- **AC-A11Y-05** Every sound event has a visual counterpart.

8.23 F23 — Local API and MCP server (API)

Purpose. "Connect this with AI" without adding an AI service to the app: Xini's own Claude can import spell lists and characters, read the campaign log, and more, through a local API.

REST v1 (§26.1) under `/api/v1`, authenticated by the session cookie (browser) or `Authorization: Bearer <token>` (API tokens created in Admin → API & MCP, shown once, stored as SHA-256 hashes, with scopes: `content:read`, `content:write`, `sheets:read`, `sheets:write`, `log:read`, `log:write`, `campaign:read`). By default API tokens only work from the host machine: requests that came through Cloudflare (a `cf-ray` or `cf-connecting-ip` header) are refused unless **Allow remote API** is enabled.

JSON Schemas for every import format at `/api/v1/schemas/{spell,character,monster,item,handout,campaign-log-entry}.json`, generated from the zod schemas with `z.toJSONSchema(..., { target: "draft-2020-12" })`.

MCP server (`packages/mcp`): a stdio MCP server built on `@modelcontextprotocol/server` v2 that calls the REST API with an API token. Tools listed in §26.2. The Admin console's **Connect Claude** page shows copy-paste setup for Claude Code (`claude mcp add gloam --env GLOAM_URL=http://127.0.0.1:4747 --env GLOAM_TOKEN=... -- node <repo>/packages/mcp/src/index.ts`) and for Claude Desktop (JSON config snippet).

Acceptance criteria.

- **AC-API-01** REST v1 endpoints accept session cookies or bearer tokens and enforce scopes; tokens are shown once and stored hashed.
- **AC-API-02** API tokens are refused on requests carrying Cloudflare headers unless Allow remote API is enabled.
- **AC-API-03** JSON Schemas are served for every import format and match the zod schemas (a test regenerates and diffs them).
- **AC-API-04** The MCP server passes a round-trip test with an MCP client: list tools, import 3 spells (dry run, then real), import 1 character, read the log.
- **AC-API-05** The Connect Claude page shows working setup instructions with the real repository path and a freshly created token.

8.24 F24 — First run and the demo campaign (DEMO)

Purpose. Something beautiful to click on five minutes after the build finishes, with zero external art.

The Lantern Crypt (created on request from the setup page or Admin → Campaigns): a procedurally generated small dungeon — an entrance hall, a pillared crypt, a narrow corridor, a flooded chamber (water zone), and a treasure room behind a **secret door** — on the procedural stone floor with **Walls in 3D** on. Lights: wall sconces (torch preset), a brazier, a moonlit shaft of dim light, one **magical Darkness** pocket. Four goblin units (coin tokens with generated glyph art: a stylised goblin head drawn with canvas paths), one "Crypt Warden" large unit, a party spawn point, two handouts (a torn map, a riddle), ambience preset *Crypt*, and a pre-made pregenerated character sheet for quick tests. Scene fog mode: Dynamic. Everything is generated by code in `packages/server/src/demo/`.

Acceptance criteria.

- **AC-DEMO-01** Creating the demo campaign produces the Lantern Crypt with all listed elements, using no external or uploaded assets.
- **AC-DEMO-02** The demo showcases lighting (sconces flicker, darkness pocket blocks darkvision), fog of war (dynamic), a secret door, water and difficult terrain, and a ready-to-run combat.

- **AC-DEMO-03** A scripted E2E test of the first-run flow (setup → demo → open table locally → join as a test player in a second browser context → admit → move a token) completes in under 3 minutes of wall-clock time, and every step shows guidance text a first-time user can follow.

8.25 Cross-cutting acceptance criteria

Security (SEC) — details in §22.

- **AC-SEC-01** Every REST body and room message is validated with a strict zod schema (unknown keys rejected, numbers range-checked, strings length-capped); invalid input never reaches domain code (fuzz test).
- **AC-SEC-02** Every state-changing REST request authenticated by cookie requires a valid CSRF token and an allowed `Origin` (requests authenticated by a bearer API token and carrying no cookies are exempt); room messages require an authenticated, admitted session.
- **AC-SEC-03** The CSP and security headers in §22.4 and the cookie attributes in §22.2 are present on every response (test asserts headers).
- **AC-SEC-04** Local-only endpoints (setup, magic admin link, API tokens by default) refuse requests that carry Cloudflare headers or come from a non-loopback address.
- **AC-SEC-05** Per-user rate limits on room messages (§13.5) disconnect abusive clients after a warning; per-IP limits protect auth endpoints.
- **AC-SEC-06** Secrets (admin password hash, PIN hashes, session and invite hashes, tunnel token, API token hashes, `secret.key`) never appear in logs, API responses or client state (grep test over logs and snapshots).
- **AC-SEC-07** Hidden information (hidden tokens, secret doors, hidden walls other than as anonymous occluders, carried lights' and effects' token links while the carrier isn't perceived, DM notes, NPC exact HP and AC when not permitted, blind roll results, other players' private rolls, other scenes' prep data) never appears in a player's received WebSocket frames (automated frame inspection in the E2E suite).
- **AC-SEC-08** The data directory and `secret.key` are created with owner-only permissions where the OS supports it.
- **AC-SEC-09** `pnpm audit --prod` shows no known high or critical vulnerabilities at the end of the build, or each exception is justified in DECISIONS.md.

Performance (PERF) — budgets in §37.

- **AC-PERF-01** The benchmark scene holds 60 fps median (p95 frame $\leq$ 20 ms) on the High tier at 1080p in headed Chromium on the host machine; the bench report records that machine's CPU/GPU. (Target hardware class: Apple M1 / Intel Iris Xe.)
- **AC-PERF-02** The initial table-route JavaScript is at most 1.2 MB gzipped (the dice physics worker loads lazily and is prefetched when idle).
- **AC-PERF-03** Server command handling is under 5 ms at p95 and vision recomputation under 10 ms at p95 in the benchmark.
- **AC-PERF-04** A player joining an already-loaded scene sees the board within 4 s on a 20 Mbps connection (assets cached afterwards).
- **AC-PERF-05** No frame drops longer than 100 ms from shader compilation during play (shaders precompiled with `gl.compileAsync` /warm-up during the intro).

Design system (DS) — Part D.

- **AC-DS-01** All UI colours, type sizes, spacing, radii and motion come from the tokens in §27 (no stray hex values outside the token file, enforced by a lint script).
- **AC-DS-02** None of the banned patterns in §27.6 appear in the final screenshots (visual-critic check).
- **AC-DS-03** Every custom icon in Appendix G is implemented as a React component and in the WebGL icon atlas, and renders crisply at 16, 20 and 24 px.
- **AC-DS-04** The first-load sequence (candle ignition → board fade-in → staggered HUD) runs once per load and respects reduced motion.
- **AC-DS-05** Empty states, loading states and error states exist for every panel (no blank panels).
- **AC-DS-06** Fonts are self-hosted from `@fontsource` packages; no font request leaves the origin.

Part C — Engineering

9. Stack and versions

Versions below were verified against the npm registry and official documentation on **26 September 2026**. Use these versions (or newer patch releases). For any package not listed, use the latest stable release at build time and record it in `docs/DECISIONS.md`. Phase O's `stack-verifier` re-checks every row.

9.1 Runtime and tooling

Package	Version	Notes
Node.js	24 LTS (24.21.x)	"engines": { "node": " ≥ 24.11 " }; must also run on Node 26 (LTS from 28 Oct 2026). Node 20 is end-of-life.
pnpm	12.6.0	"packageManager": "pnpm@12.6.0"; enable with <code>corepack enable</code> . See §9.4 for build-script rules.
TypeScript	7.0.2	Native compiler; used only for type-checking (<code>tsc -b</code> / <code>tsc --noEmit</code>). It has no programmatic API; nothing in this stack needs one. Fallback: <code>typescript@6.0.3</code> .
Vite	8.3.1	Rolldown-based. Node ^20.19 or ≥ 22.12.
@vitejs/plugin-react	latest for Vite 8	
Biome	2.5.14	Lint + format (no ESLint/Prettier).
Vitest	5.0.2	Unit/integration tests.
Playwright	1.63.0	E2E + screenshots; add <code>@axe-core/playwright</code> for accessibility checks.

9.2 Client

Package	Version	Notes
react, react-dom	~19.3.0	Pin the minor: R3F9.8's peer range is <code> ≥ 19 <19.4</code> .
three	~0.186.1	Pin: <code>postprocessing 6.39.5</code> requires three <code><0.187</code> . Matching <code>@types/three</code> .
@react-three/fiber	9.8.1	
@react-three/drei	10.7.9	See §24.9 for helpers that must not fetch from CDNs.
postprocessing / @react-three/postprocessing	6.39.5 / 3.1.2	
@dimforge/rapier3d-deterministic-compat	0.21.0	Dice physics in a Web Worker; API-identical to <code>-compat</code> , cross-platform deterministic so every player sees the same tumble. Worker chunk $\approx$ 4.3 MB (1.6 MB gzipped): lazy-load.
three-mesh-bvh	0.9.15	Raycasting, 3D-map slicing (shapecast technique from its "clipped edges" example).
troika-three-text	0.52.5	Via drei <code><Text></code> , always with a local font file.
zustand	5.0.15	Client state.
@tanstack/react-query	5.104.0	REST data (admin console, library, content search).
zod	4.6.5	Shared schemas; <code>z.toJSONSchema()</code> for published schemas.
motion	13.4.4	UI animation; import from <code>motion/react</code> .
tailwindcss + @tailwindcss/vite	4.3.3	Tokens defined as CSS variables (§27).

Package	Version	Notes
@use-gesture/react	10.3.1	Touch/pointer gestures for DOM overlays and the drawing pad.
@colyseus/sdk	0.18.4	Realtime client (renamed from <code>colyseus.js</code>).
@colyseus/react	0.18.2	Optional helpers; the core sync layer is hand-written (§23.3).
zzfx	1.3.2 (MIT)	Sound synthesis.
react-router	latest 7.x	Declarative mode.
react-markdown + remark-gfm	latest	Markdown without raw HTML.
lucide-react	latest	Only for generic UI glyphs (close, chevrons, settings); every game concept uses the custom icon set (§30).
@fontsource-variable/fraunces, @fontsource/alegreya-sans, @fontsource/cinzel, @fontsource-variable/jetbrains-mono	5.3.0 (OFL-1.1)	Self-hosted fonts.

9.3 Server

Package	Version	Notes
@colyseus/core	0.18.17	Exports <code>defineServer</code> , <code>defineRoom</code> , <code>matchMaker</code> , <code>Room</code> , <code>StateView</code> . Do not depend on the <code>colyseus</code> meta-package (see §9.4).
@colyseus/ws-transport	0.18.x	WebSocket transport (set <code>maxPayload</code> to 256 KB).
@colyseus/tools	0.18.4	<code>listen()</code> .
@colyseus/schema	5.0.34	Use the <code>schema()</code> builder with <code>t.*</code> (no decorators).
express	5.2.1	HTTP.
helmet	8.3.0	Security headers.
pino	10.3.1	Logging.
drizzle-orm / drizzle-kit	0.45.3 / 0.31.11	<code>drizzle-orm/better-sqlite3</code> .
better-sqlite3	13.0.3	Node ≥ 22 ; prebuilt binaries ship inside the package.
@node-rs/argon2	2.2.1	argon2id; prebuilt, no install script.
sharp	0.35.4	Images.
file-type	22.1.1	ESM-only magic-byte detection (GLB detected as <code>model/gltf-binary</code>).
gltf-validator	2.0.0-dev.3.10	Khronos validator (<code>validateBytes</code>). Only dev-tagged versions exist; pin exactly.
draco3dgltf	latest	Server-side Draco decoding of uploaded GLBs (local WASM; registered as gltf-transform's <code>draco3d.decoder</code>)
@gltf-transform/core, /functions, /extensions	4.5.0	GLB optimisation.
meshoptimizer	1.3.0	Meshopt encoder/decoder/simplifier.
busboy	latest	Streaming multipart parsing with limits.
fflate	latest	Zip for <code>.gloam</code> export/import.
nanoid	6.0.1	IDs.
p-limit	7.3.3	Concurrency limits.
@modelcontextprotocol/server (+ / client for tests)	2.1.0	MCP v2 stable line; needs zod ≥ 4.2 .

9.4 Known gotchas (verified; don't rediscover them)

G1 — pnpm 12 blocks dependency build scripts by default and fails the install (`ERR_PNPM_IGNORED_BUILDS`) until each flagged package is allowed or denied in `pnpm-workspace.yaml`. `onlyBuiltDependencies` no longer exists. Use:

```
# pnpm-workspace.yaml
packages:
  - "packages/*"
allowBuilds:
  better-sqlite3: false # prebuilt binaries are bundled; building needs a C++ toolchain
  msgpackr-extract: false # prebuilt platform package is used anyway
  esbuild: true # only if something still pulls esbuild in
```

Also note `minimumReleaseAge` (packages published < 24 h ago won't resolve) and `blockExoticSubdeps: true` (git/URL sub-dependencies are blocked).

G2 — The `colyseus` meta-package fails to install on pnpm 11/12 because it pulls `@colyseus/uwebsockets-transport` → `uWebSockets.js` from GitHub (`ERR_PNPM_EXOTIC_SUBDEP`). Depend on `@colyseus/core`, `@colyseus/ws-transport`, `@colyseus/tools` and `@colyseus/schema` directly.

G3 — Colyseus 0.18 changes: client package is `@colyseus/sdk`; server bootstrap uses `defineServer / defineRoom`; `onLeave(client, code)`; client state callbacks via `Callbacks.get(room)`; `setMetadata` replaces instead of merging; `Client#id` removed; **a schema may have at most 63 fields** (inherited fields count) — split with nested schemas.

G4 — StateView tags are bitmasks: use powers of two (1, 2, 4...). A `.view()` collection is **undefined, not empty**, on a client with nothing in its view: guard for it. Every re-add delivers a **new client object**, so register per-item listeners inside `onAdd`.

G5 — Plain `class X extends Token {}` without new fields breaks schema registration; use `Token.extend({...}, "Name")` or inline methods.

G6 — TypeScript 7 defaults: `strict: true`, `module: esnext`, and `types: []` — list `node`, `vite/client` explicitly. `baseUrl` and `moduleResolution: node10` are removed.

G7 — Server TypeScript runs natively on Node 24 (type stripping): no build step for the server. Therefore use `erasableSyntaxOnly: true` (no enums, namespaces, parameter properties or decorators), `verbatimModuleSyntax: true`, and `.ts` extensions in relative imports. Run with `node --disable-warning=ExperimentalWarning`. If the Phase 0 spike hits a blocker, fall back to `tsx` and record it.

G8 — drei helpers that fetch from CDNs are forbidden (P1): `useGLTF` must not use Draco (its default decoder path is a Google CDN) — use meshopt only; drei `<Environment preset=...>` downloads HDRIs — use procedural environments; drei `<Text> / troika` fall back to a CDN font — always pass a local `font` URL (use `.woff`, not `.woff2`).

G9 — WebGL screenshots come out blank unless the renderer has `preserveDrawingBuffer: true` — enable it only in test mode.

G10 — Quick tunnels don't support Server-Sent Events and cap in-flight requests at 200: use WebSockets only, and limit client asset fetch concurrency to 6.

G11 — `gltf-validator` is only published with dev tags; `validateBytes(new Uint8Array(buf), { format: "glb", maxIssues: 100, writeTimestamp: false })` → check `report.issues.numErrors`.

10. Repository layout

A pnpm monorepo. Package names are scoped `@gloam/*`.

```
gloam/
├─ package.json           # root scripts ($10.1), packageManager, engines
├─ pnpm-workspace.yaml    # packages + allowBuilds
├─ tsconfig.base.json     # shared compiler options
├─ biome.json
├─ .nvmrc                 # 24
├─ .gitignore             # node_modules, data/, artifacts/, dist/, *.log
└─ CLAUDE.md              # Appendix B
```

```

├─ .claude/
├─ settings.json          # Appendix D
├─ agents/                # Appendix C
├─ hooks/post-edit.mjs, stop-check.mjs
├─ docs/
├─ SPEC.md FEATURES.json PROGRESS.md DECISIONS.md HOSTING.md
├─ research/              # Phase 0 outputs
├─ packages/
├─ shared/                # isomorphic: no Node or DOM APIs
├─   └─ src/
├─     ├─ constants.ts    # APP_NAME, limits, palettes
├─     ├─ units.ts        # ft ↔ m formatting
├─     ├─ schemas/        # zod: entities, sheets, spells, messages, imports
├─     ├─ state/          # Colyseus schema() classes shared by server & client
├─     ├─ protocol/       # message names, payload types, request map, error codes
├─     ├─ geometry/       # vec2, segments, polygons, visibility sweep, spatial hash, raster
├─     ├─ vision/         # senses, light levels, point & token perception
├─     ├─ movement/      # obstacles, visibility graph, A*, cost integration, FMM field
├─     ├─ aoe/            # shapes, inclusion, line of effect, cover
├─     ├─ dice/           # lexer, parser, evaluator (injectable RNG), formatter
├─     └─ rules/          # conditions metadata, damage, death saves, speed, initiative
├─ content/               # SRD pack build scripts + generated packs (committed)
├─   └─ scripts/          # fetch-srd, parse-srd-pdf, import-foundry, merge, verify
├─   └─ overlays/        # hand-authored light/obscurement/area data
├─   └─ packs/srd-5.2.1/  # spells.json, conditions.json, light-sources.json, MANIFEST.json, CONTRIBUTION.md
├─ server/
├─   └─ src/
├─     ├─ index.ts config.ts logger.ts
├─     ├─ http/           # app, security headers, csrf, rate limits, routes, dev Vite middleware
├─     ├─ auth/           # admin, sessions, invites, profiles, local-only guard
├─     ├─ tunnel/         # cloudflared manager
├─     ├─ db/             # drizzle schema, client, migrations, repositories
├─     ├─ rooms/          # LobbyRoom, TableRoom, view sync, message handlers
├─     ├─ engine/         # command bus, commands, permissions, history, ops
├─     ├─ domain/         # campaign, scene, tokens, walls, lights, zones, effects, combat,
├─     │                   # sheets, dice, spells, hp, audio, handouts, log
├─     ├─ vision/         # vision service, light raster, explored store, view sync
├─     ├─ persistence/    # snapshots, backups, export/import, fog flush
├─     ├─ assets/         # upload, worker pipeline, serving, quotas
├─     ├─ content/        # pack loader, homebrew, importers
├─     ├─ api/            # REST v1, schemas endpoint, API tokens
├─     ├─ demo/           # Lantern Crypt generator, glyph art
├─     └─ test/           # in-process harness, fixtures
├─ web/
├─   └─ index.html vite.config.ts
├─   └─ src/
├─     ├─ main.tsx App.tsx
├─     ├─ routes/         # Join, WaitingRoom, Table, Admin, Setup, Closed
├─     ├─ net/            # colyseus client, lobby, table sync, requests, clock
├─     ├─ state/          # zustand stores + render registry
├─     ├─ board/          # R3F: Board, CameraRig, TableSurface, MapLayer, fog/, tokens/,
├─     │                   # walls/, zones/, effects/, movement/, measure/, pings/, vfx/,
├─     │                   # lighting/, postfx/, editor/
├─     ├─ dice/           # DiceHud, physics.worker, geometry, symmetry, skins
├─     ├─ hud/            # TopBar, TurnTracker, ActionBar, Toolbar, RollFeed, Toasts,
├─     │                   # EmoteWheel, RadialMenu, CommandPalette
├─     ├─ panels/         # sheet/, spells/, dm/, log/, library/, handouts/, settings/
├─     ├─ admin/          # console sections
├─     ├─ ui/             # design-system components
├─     ├─ icons/          # condition & game icons (React) + WebGL atlas builder
├─     ├─ audio/          # engine, sfx recipes, music, ambience
├─     ├─ art/            # drawing pad, paper-cutout worker
├─     ├─ input/          # shortcuts, gestures
├─     ├─ styles/         # tokens.css, globals.css
├─     └─ test/           # test hooks (test mode only)
├─ mcp/                   # stdio MCP server (runs with node directly)
├─ tools/                 # extract-features, features-status, shots, bench,
├─                         # check-tokens, fuzz-uploads
├─ e2e/                   # playwright.config.ts, fixtures/, journeys/*.spec.ts
├─ artifacts/             # screens/, logs/, bench/ (gitignored)

```

10.1 Root scripts

Script	Does
<code>pnpm dev</code>	Server with <code>node --watch</code> + Vite in middleware mode on the same origin (<code>http://localhost:4747</code>)
<code>pnpm build</code>	Build the web app (<code>packages/web/dist</code>); content packs are already committed

Script	Does
<code>pnpm start</code>	Production server (serves <code>packages/web/dist</code>)
<code>pnpm start --open</code>	Start and open the table immediately using the saved mode
<code>pnpm typecheck</code>	<code>tsc -b</code> across packages
<code>pnpm lint</code>	<code>biome ci . + node tools/check-tokens.mjs</code>
<code>pnpm test</code>	Vitest (shared + server)
<code>pnpm test:e2e</code>	Playwright journeys
<code>pnpm shots</code>	Screenshots of key screens at three viewports
<code>pnpm bench</code>	Benchmark scene: fps, frame times, server timings → <code>artifacts/bench/</code>
<code>pnpm check</code>	typecheck + lint + test (must pass before every commit)
<code>pnpm check:fast</code>	typecheck + lint (used by the Stop hook)
<code>pnpm content:build</code>	Rebuild SRD packs from pinned sources and verify counts
<code>pnpm features:extract /</code> <code>features:status</code>	Build <code>docs/FEATURES.json</code> from this spec / print pass counts
<code>pnpm db:generate</code>	drizzle-kit migration generation

11. Runtime, configuration and the data directory

Ports. HTTP + WebSocket: `4747` (env `PORT`). cloudflared metrics: `4748` on loopback (`METRICS_PORT`). Vite HMR in dev: `24678`.

Configuration comes from environment variables (Appendix J) with defaults, then from the `settings` table for things the Admin changes in the UI (tunnel mode and token, LAN mode, defaults). Validate config with zod at startup and print a readable error on failure.

Data directory (`DATA_DIR`, default `./data`, created with mode `0700`):

```
data/
├─ gloam.db gloam.db-wal gloam.db-shm
├─ secret.key           # 32 random bytes, mode 0600, created on first run
├─ assets/ab/<fileId>-<variant>.<ext> # processed outputs, content-addressed by the original's SHA-256
├─ tmp/                 # upload staging (cleared at start)
├─ snapshots/<campaignId>/<iso>-<kind>.json.gz
├─ backups/gloam-<date>.db
├─ exports/
└─ logs/gloam-<date>.log      # pino, rotated daily, 14 kept
```

Startup sequence. Load and validate config → ensure data directory → load or create `secret.key` → open SQLite (pragmas in §20.1) → if migrations are pending, take a database backup first (`db.backup()`), then run migrations → **crash hygiene**: revoke every invite code still marked active, and close any table session left open by a crash (log entry "Session ended unexpectedly") → load content packs and **assert pack counts** → start the asset processor (a child process, §21.1) → build the Express app → `defineServer` with rooms and transport → `listen` → create the lobby room (`autoDispose = false`) → create the table room for the selected campaign (`roomId = campaignId`, `autoDispose = false`) → start schedulers (snapshots, backups, fog flush, rate-limit sweeps) → print the banner.

Two different sequences (implement them as separate functions):

- `closeTable()` (Admin clicks Close table; the server keeps running): stop accepting knocks → revoke all invite codes → broadcast `table.closing` → disconnect non-admin clients (players see the closed screen) → flush explored fog → write a `close` snapshot → stop cloudflared → end the session in the log.
- `shutdown()` (SIGINT, SIGTERM, or `process.on('uncaughtException')` after logging): if the table is open, run `closeTable()` → stop accepting commands → write a `shutdown` snapshot → stop the asset processor → close SQLite → exit (hard timeout 10 s).

Server bootstrap sketch (verify exact exports in the Phase 0 spike):

```
// packages/server/src/index.ts (sketch)
import { defineServer, defineRoom, matchMaker } from "@colyseus/core";
import { WebSocketTransport } from "@colyseus/ws-transport";
import { listen } from "@colyseus/tools";
import { buildHttpApp } from "../http/app.ts";
import { LobbyRoom } from "../rooms/LobbyRoom.ts";
import { TableRoom } from "../rooms/TableRoom.ts";

const server = defineServer({
  transport: new WebSocketTransport({ maxPayload: 256 * 1024 }),
  rooms: { lobby: defineRoom(LobbyRoom), table: defineRoom(TableRoom) },
  express: (app) => buildHttpApp(app), // static SPA, REST, assets, dev Vite middleware
});
await listen(server); // reads PORT
matchMaker.controller.exposedMethods = ["joinById", "reconnect"]; // clients cannot create rooms
await matchMaker.createRoom("lobby", {});
await matchMaker.createRoom("table", { campaignId }); // TableRoom sets this.roomId = campaignId
```

Dev mode. In `buildHttpApp`, when `NODE_ENV ≡ "production"`, create Vite with `server: { middlewareMode: true, hmr: { port: 24678 } }`, `appType: "spa"`, `root: "packages/web"` and `app.use(vite.middlewares)`. Everything shares one origin, exactly like production. In production, serve `packages/web/dist` with long-cache headers for hashed files and an SPA fallback for routes.

12. Data model and database

12.1 Conventions

- IDs: `nanoid(16)` with a type prefix — `cmp_`, `scn_`, `tok_`, `act_`, `wal_`, `lgt_`, `zon_`, `eff_`, `usr_`, `ses_`, `inv_`, `rol_`, `req_`, `his_`, `snp_`, `hnd_`, `log_`, `tpl_`, `api_`, `ast_`. Asset file IDs (`asset_files.id`) are the SHA-256 hex of the original upload; per-campaign asset references use `ast_` IDs.
- Times: integer milliseconds since the epoch.
- JSON columns hold zod-validated documents with a `schemaVersion`. Read paths validate; a row that fails validation is logged and quarantined (the app keeps running).
- Coordinates: feet, `float`, world XZ plane (`x`, `y` in 2D terms = world X, Z).

12.2 Tables (Drizzle, SQLite)

Table	Columns (type)	Notes
<code>settings</code>	<code>key</code> text PK, <code>value_json</code> text	Global settings (tunnel mode, token (encrypted with <code>secret.key</code> , AES-256-GCM), LAN, defaults)
<code>admin</code>	<code>id</code> int PK=1, <code>password_hash</code> text, <code>created_at</code> , <code>updated_at</code>	argon2id
<code>users</code>	<code>id</code> , <code>display_name</code> , <code>color</code> , <code>pin_hash</code> null, <code>is_admin</code> bool, <code>created_at</code> , <code>last_seen_at</code> , <code>banned_at</code> null, <code>ban_reason</code> null, <code>dice_skin_json</code> , <code>prefs_json</code>	Profiles
<code>devices</code>	<code>id</code> , <code>user_id</code> , <code>device_hash</code> , <code>label</code> , <code>created_at</code> , <code>last_seen_at</code>	Recognised browsers
<code>sessions</code>	<code>id</code> , <code>user_id</code> , <code>token_hash</code> , <code>kind</code> (admin player), <code>created_at</code> , <code>last_seen_at</code> , <code>expires_at</code> , <code>revoked_at</code> , <code>table_session_no</code> , <code>status</code> (pending admitted denied kicked)	One per browser login
<code>invite_codes</code>	<code>id</code> , <code>code_hash</code> , <code>created_at</code> , <code>expires_at</code> null, <code>max_uses</code> null, <code>uses</code> , <code>revoked_at</code> null	
<code>campaigns</code>	<code>id</code> , <code>name</code> , <code>cover_asset_id</code> , <code>rules_pack</code> , <code>units</code> , <code>house_rules_json</code> , <code>settings_json</code> , <code>active_scene_id</code> , <code>session_no</code> , <code>created_at</code> , <code>updated_at</code> , <code>archived_at</code>	

Table	Columns (type)	Notes
memberships	campaign_id, user_id, role (dm player spectator), created_at	PK (campaign_id, user_id)
scenes	id, campaign_id, name, sort, map_kind, map_asset_id, calibration_json, floor_json, ambient_json, fog_mode, fog_cell_ft, bounds_json, spawn_json, music_json, walls3d bool, thumbnail_asset_id, created_at, updated_at, archived_at	
walls	id, scene_id, ax, ay, bx, by, kind, door_state, hidden bool, created_at	Blocking flags derive from kind + door_state (§8.7 matrix)
lights	id, scene_id, token_id null, x, y, elevation, bright, dim, color, intensity, animation, cone_deg null, direction_deg, magical bool, pierce_darkness bool, enabled bool, dm_only bool (DM vision aid: invisible to players and ignored by perception), preset	
zones	id, scene_id, kind, shape_json, label, color, visible bool, triggers_json, note	
actors	id, campaign_id, kind (character npc), owner_user_id null, template_id null, lock_level, sheet_json, created_at, updated_at	Sheet document (§12.3)
sheet_templates	id, campaign_id, name, blocks_json, created_by	
tokens	id, scene_id, actor_id null, link (linked unlinked), name, x, y, elevation, rotation, size_ft, appearance_json, owner_ids_json, disposition, hidden bool, reveal_json, hp_display, stats_json null, status_json null, overrides_json, light_id null, locked bool, created_at, updated_at	Linked tokens read stats and status (conditions, markers, exhaustion, concentration, death saves) from the actor; unlinked tokens store their own copies here (F05)
effects	id, scene_id, source_json (content ref, caster token, slot), shape_json, props_json, triggers_json, attached_token_id null, concentration_token_id null, expires_json, visibility, vfx, created_at	
fog_masks	scene_id, layer (reveal:all reveal:<userId> explored:<userId>), cell_ft, origin_x, origin_y, w, h, data blob (deflate bitset), updated_at	PK (scene_id, layer)
combats	id, scene_id, active bool, round, turn_index, data_json (combatants, groups), started_at, ended_at	
rolls	id, campaign_id, scene_id, user_id, token_id null, formula, result_json, total, visibility, purpose, request_id null, manual bool, seed, created_at	
roll_requests	id, campaign_id, created_by, data_json, status, created_at, closed_at	
content	id, campaign_id null (null=pack), pack, type (spell monster item), slug, name, data_json, status (active proposed rejected), created_by, created_at, updated_at	Packs are loaded from files; this table holds homebrew and proposals
history	id (autoincrement), campaign_id, scene_id null, user_id, acting_as null, type, ops_json, inverse_json, summary, undoable bool, created_at, undone_at null, undone_by null	Also the audit trail of game changes
snapshots	id, campaign_id, name, kind (auto manual scene close shutdown pre-restore), path, bytes, created_at	Files in data/snapshots/
asset_files	id (SHA-256 of the original upload), kind, mime, bytes, width, height, duration_ms, variants_json (paths of processed outputs), meta_json (stats, validator summary), created_at	Global, content-addressed, de-duplicated file store
assets	id (ast_...), campaign_id, file_id → asset_files, name, tags_json, uploader_id, status (pending approved rejected), created_at, reviewed_by, reviewed_at, deleted_at	Per-campaign reference; the same file can be used by several campaigns
handouts	id, campaign_id, title, body_md, image_asset_id, recipients_json, created_by, created_at	

Table	Columns (type)	Notes
log_entries	id, campaign_id, session_no, kind, text, data_json, visibility, user_id, created_at	
api_tokens	id, name, token_hash, scopes_json, created_at, last_used_at, revoked_at	
security_log	id, event, user_id null, ip null, detail_json, created_at	

Indexes: every foreign key; history(campaign_id, id), rolls(campaign_id, created_at), tokens(scene_id), walls(scene_id), content(campaign_id, type, slug) unique.

12.3 Core documents (TypeScript shapes; zod schemas live in `packages/shared/src/schemas`)

```

type Ft = number; // feet
type Vec2 = { x: Ft; y: Ft };
type Size = "tiny" | "small" | "medium" | "large" | "huge" | "gargantuan";
type DamageType = "acid"|"bludgeoning"|"cold"|"fire"|"force"|"lightning"|"necrotic"
  | "piercing"|"poison"|"psychic"|"radiant"|"slashing"|"thunder";
type Ability = "str"|"dex"|"con"|"int"|"wis"|"cha";

interface Senses { darkvision: Ft; blindsight: Ft; tremorsense: Ft; truesight: Ft }
interface Speeds { walk: Ft; fly: Ft; swim: Ft; climb: Ft; burrow: Ft; hover: boolean }

interface TokenStatus {
  conditions: { id: ConditionId; source?: string; untilRound?: number }[];
  markers: { id: MarkerId | `custom:${string}`; label?: string; untilRound?: number }[];
  exhaustion: 0|1|2|3|4|5|6;
  concentration?: { effectId?: string; spellId?: string };
  deathSaves?: { successes: 0|1|2|3; failures: 0|1|2|3; stable: boolean; dead: boolean };
  outlined: boolean; // e.g. Faerie Fire: can't benefit from Invisible
  seeInvisible: boolean; // e.g. See Invisibility active
}

interface Token {
  id: string; sceneId: string; actorId?: string; name: string;
  pos: Vec2; elevation: Ft; rotationDeg: number; sizeFt: Ft;
  appearance: { mode: "model"|"standee"|"coin"|"auto"; assetId?: string; scale: number;
    offsetY: Ft; rotationOffsetDeg: number; tint?: string };
  ownerIds: string[]; // users who control it
  disposition: "party"|"friendly"|"neutral"|"hostile";
  link: "linked"|"unlinked"; // linked: stats & status come from the actor (F05)
  hidden: boolean; // DM-hidden: never sent to any player
  revealTo: "vision" | "all" | string[]; // "vision" (default): perception decides;
  // "all" / user IDs: always visible to them regardless of vision
  hpDisplay: "exact"|"bar"|"descriptor"|"hidden";
  stats?: { hp: number; hpMax: number; hpTemp: number; ac: number; speeds: Speeds; // unlinked only
    senses: Senses; saves: Partial<Record<Ability, number>>; dexMod: number; initBonus: number;
    resist: DamageType[]; immune: DamageType[]; vuln: DamageType[]; reachFt: Ft };
  status?: TokenStatus; // unlinked only; linked tokens read actor.status
  overrides: { speedOverride?: Ft; bonusMove?: { ft: Ft; until: "turn"|"rounds"|"removed";
    rounds?: number }; freeMovement?: boolean; lockMovement?: boolean;
    ignoreConditionSpeed?: boolean; shareVisionWith?: string[]; countAsMovement?: boolean };
  lightId?: string; locked: boolean;
}

interface Wall { id: string; sceneId: string; a: Vec2; b: Vec2;
  kind: "wall"|"door"|"window"|"curtain"|"invisible"|"secret";
  doorState?: "closed"|"open"|"locked"; hidden: boolean }

interface Light { id: string; sceneId: string; tokenId?: string; pos: Vec2; elevation: Ft;
  bright: Ft; dim: Ft; color: string; intensity: number;
  animation: "none"|"torch"|"candle"|"pulse"|"shimmer"; coneDeg?: number; directionDeg: number;
  magical: boolean; pierceDarkness: boolean; // true only for lights that overcome magical darkness
  enabled: boolean; dmOnly: boolean; // dmOnly: DM vision aid, never sent, ignored by perception
  preset?: string }

interface Effect { id: string; sceneId: string;
  source: { kind: "spell"|"feature"|"custom"; contentId?: string; casterTokenId?: string; slot?: number };
  shape: AreaShape; // §17.1
  attachedTokenId?: string; // emanations, "moves with object"
  props: { difficult?: boolean; obscurement?: "light"|"heavy"; magicalDarkness?: boolean;

```

```

    opaque?: boolean; light?: { bright: Ft; dim: Ft; color: string; magical: boolean; pierceDarkness: boolean };
  an };
  silence?: boolean };
  triggers: { when: "enter"|"startTurn"|"endTurn"|"per5ft"; save?: { ability: Ability; dc: number;
    onSuccess: "half"|"none"|"special" }; damage?: { formula: string; type: DamageType };
    condition?: ConditionId; note?: string }[];
  concentrationTokenId?: string;
  expires?: { round: number; turnOff: string; when: "start"|"end" } | { never: true };
  visibility: "everyone"|"dm"; vfx: VfxPreset; movement?: { by: "caster"|"dm"; maxFt?: Ft;
    drift?: { ft: Ft; direction: "awayFromCaster"|"chosen" } } }

```

The **sheet document** (`actors.sheet_json`) follows the structure in F10 and is fully specified as a JSON Schema in Appendix F.3; the **spell document** in Appendix F.2.

13. Realtime protocol

13.1 Rooms

- **lobby** — one instance (`roomId = "lobby"`). Joined by pending users and by admitted users briefly during admission. Its state contains only each client's own knock status (per-client view).
- **table** — one instance per running campaign (`roomId = campaignId`). All play happens here.
- Restrict matchmaking: `matchMaker.controller.exposedMethods = ["joinById", "reconnect"]`. Clients join with `client.joinById("lobby")` and `client.joinById(campaignId)`.
- Both rooms set `autoDispose = false`.

13.2 Authentication

```

// In each Room class
static async onAuth(_token: string, _options: unknown, context: { headers: Headers; ip: string }) {
  const sid = parseCookie(context.headers.get("cookie") ?? "")["gloam_sid"];
  const session = await sessions.verify(sid); // hash lookup, not revoked/expired
  if (!session) throw new ServerError(401, "UNAUTHENTICATED");
  // lobby: status pending|admitted ; table: status admitted for the current table session
  return { userId: session.userId, sessionId: session.id, role: resolveRole(session) };
}

```

The value returned becomes `client.auth`. The Colyseus SDK sends same-origin cookies with its matchmaking requests (`credentials: "include"`); verify this in the Phase 0 spike, and if it doesn't hold, pass a short-lived signed join ticket (from `POST /api/table/ticket`) in the join options instead.

13.3 Synchronised state (Colyseus schema)

Shared definitions live in `packages/shared/src/state` so the client can pass the root class as the third argument to `joinById` (typed objects with methods). Keep each schema under 63 fields.

```

import { schema, t } from "@colyseus/schema";

// View tags are bitmasks (powers of two). A client sees a tagged field only after view.add(item, TAG).
export const TAG_HP = 1; // exact HP numbers: DMs, the token's controllers, and players when HP display is Exact
export const TAG_OWNER = 2; // controller-only numbers: AC, movement budget, speeds, turn start
export const TAG_VISION = 4; // senses: controllers, players the token's vision is shared with, DMs
export const TAG_DM = 8; // DM-only fields
export const TAG_LINK = 16; // token links of carried lights and attached effects: only while that token is perceivable

export const V2 = schema({ x: t.float32(), y: t.float32() }, "V2");

export const Presence = schema({
  userId: t.string(), name: t.string(), color: t.string(), role: t.string(),
  online: t.boolean(), handRaised: t.boolean(), spectator: t.boolean(),

```

```

}, "Presence");

export const TokenHp = schema({ hp: t.int32(), hpMax: t.int32(), hpTemp: t.int32() }, "TokenHp");

export const TokenOwner = schema({
  ac: t.int16(), budgetFt: t.float32(), usedFt: t.float32(), turnStart: V2, mode: t.string(),
  pips: t.uint8(), // bitmask: 1 action, 2 bonus action, 4 reaction, 8 object interaction (used)
  dashes: t.uint8(), bonusMoveFt: t.float32(),
  speedWalk: t.float32(), speedFly: t.float32(), speedSwim: t.float32(),
  speedClimb: t.float32(), speedBurrow: t.float32(),
}, "TokenOwner");

export const TokenVision = schema({
  darkvision: t.float32(), blindsight: t.float32(), tremorsense: t.float32(), truesight: t.float32(),
  blinded: t.boolean(), unconscious: t.boolean(), seeInvisible: t.boolean(),
}, "TokenVision");

export const TokenDm = schema({ secretNote: t.string(), dmHidden: t.boolean(), link: t.string() }, "TokenDm");

export const Token = schema({
  id: t.string(), actorId: t.string(), name: t.string(),
  pos: V2, elevation: t.float32(), rotation: t.float32(), sizeFt: t.float32(),
  mode: t.string(), assetId: t.string(), scale: t.float32(), offsetY: t.float32(),
  ringColor: t.string(), disposition: t.string(), ownerIds: t.array("string"),
  hpDisplay: t.string(), hpBand: t.uint8(), hpFrac: t.float32(), // band/frac per display rules
  conditions: t.array("string"), markers: t.array("string"), exhaustion: t.uint8(),
  concentrating: t.boolean(), prone: t.boolean(), dead: t.boolean(), invisibleFx: t.boolean(),
  outlined: t.boolean(), lightOn: t.boolean(), moveSeq: t.uint32(), pinnedBars: t.array("string"),
  hp: t.ref(TokenHp).view(TAG_HP),
  own: t.ref(TokenOwner).view(TAG_OWNER),
  vis: t.ref(TokenVision).view(TAG_VISION),
  dm: t.ref(TokenDm).view(TAG_DM),
}, "Token");

export const Sensed = schema({ id: t.string(), pos: V2 }, "Sensed"); // tremorsense; id is per-viewer opaque

export const Wall = schema({ id: t.string(), ax: t.float32(), ay: t.float32(), bx: t.float32(),
  by: t.float32(), kind: t.string(), door: t.string(),
  // players: secret doors arrive as kind "wall", door ""; hidden sight-blocking walls as kind "occluder"
  dmKind: t.string().view(TAG_DM), dmHidden: t.boolean().view(TAG_DM) }, "Wall");

export const LinkS = schema({ tokenId: t.string(), casterId: t.string() }, "Link");

export const LightS = schema({ id: t.string(), x: t.float32(), y: t.float32(), elevation: t.float32(),
  bright: t.float32(), dim: t.float32(), color: t.string(), intensity: t.float32(), anim: t.string(),
  coneDeg: t.float32(), dirDeg: t.float32(), magical: t.boolean(), pierceDarkness: t.boolean(),
  on: t.boolean(),
  link: t.ref(LinkS).view(TAG_LINK) }, "Light"); // carried lights: the server moves x/y with the carrier

export const ZoneS = schema({ id: t.string(), kind: t.string(), points: t.array(V2), label: t.string(),
  color: t.string() }, "Zone");

export const EffectS = schema({ id: t.string(), shapeJson: t.string(), propsJson: t.string(),
  vfx: t.string(), roundsLeft: t.int16(),
  link: t.ref(LinkS).view(TAG_LINK) }, "Effect"); // attached token and caster, only while perceivable

export const CombatS = schema({ active: t.boolean(), round: t.uint16(), turnSeq: t.uint32() }, "Combat");
// The ordered tracker is per client (it depends on perception): see the `combat.view` message.

export const SceneS = schema({ id: t.string(), name: t.string(), mapKind: t.string(),
  mapAssetId: t.string(), calibJson: t.string(), floorJson: t.string(), ambient: t.string(),
  ambientTint: t.string(), fogMode: t.string(), boundsJson: t.string(), walls3d: t.boolean() }, "Scene");

export const Table = schema({
  campaignId: t.string(), campaignName: t.string(), units: t.string(), sessionNo: t.uint16(),
  presence: t.map(Presence),
  scene: SceneS, // the ACTIVE scene only; DM prep scenes use §13.7
  tokens: t.map(Token).view(), sensed: t.map(Sensed).view(),
  walls: t.map(Wall).view(), lights: t.map(LightS).view(), zones: t.map(ZoneS).view(),
  effects: t.map(EffectS).view(),
  combat: CombatS,
  musicJson: t.string(), ambienceJson: t.string(), houseRulesJson: t.string(),
}, "Table");

```

Rules:

- Collections marked `.view()` are per-client: an item exists for a client only after `client.view.add(item)` (§13.4).
- Things that are large, rarely needed, or per-client documents are **not** in state: sheets, history, roll history, content, fog rasters, DM notes. They travel as messages or `room.request()` responses.
- Field-level secrets use tagged refs (`hp`, `own`, `vis`, `dm`). Grant a tag to a client with `client.view.add(token, TAG_HP)` (and remove it with `view.remove(token, TAG_HP)`) whenever its eligibility changes.
- Position updates for animation use `moveSeq` + a `token.moved` message carrying the (per-viewer clipped) path; clients animate from the old position along the path to `pos`.

13.4 Per-client views

On join and after every invalidation (§15.4), the server computes for each client:

Collection	DM / Admin	Player	Spectator
tokens	all, with every tag	perceivable tokens (§15.4, or revealed area in painted mode, §15.8) + tokens they control + tokens revealed to them; <code>TAG_HP</code> on tokens they control and on tokens whose HP display is Exact; <code>TAG_OWNER</code> on tokens they control; <code>TAG_VISION</code> on tokens they control or whose vision is shared with them	union of players' perceivable tokens; <code>TAG_VISION</code> on every player's viewer tokens (so the spectator can render party vision)
sensed	—	tremorsense markers for this player	—
walls	all (+ <code>TAG_DM</code>)	all except hidden walls; secret doors sanitised (<code>kind: "wall"</code> , <code>door: ""</code>); hidden walls that block sight arrive as anonymous <code>kind: "occluder"</code> segments (used only to render vision, so fog matches the server)	same as player
lights	all (with <code>TAG_LINK</code>)	all except <code>dmOnly</code> ; a light carried by a token is in view only while that token is perceivable by the player or the light's lit area intersects the player's current sight; <code>TAG_LINK</code> (its token ID) only while the carrier is perceivable	union of players'
zones	all	<code>visible</code> zones	same as player
effects	all (with <code>TAG_LINK</code>)	effects with visibility <code>everyone</code> whose area intersects the player's current sight or explored memory, or which involve a token they control; <code>TAG_LINK</code> (attached token and caster) only while those tokens are perceivable	union of players'

Combat tracker. `combat.active`, `round` and `turnSeq` are public. The ordered list is sent per client as a `combat.view { entries, activeIndex }` message whenever it changes. Each entry is `{ key, tokenId?, name, portraitAssetId?, initiative?, unknown, hpBand? }` where `key` is an opaque per-client string, `tokenId` / `initiative` / `hpBand` appear only for combatants the player perceives or controls, and `activeIndex` is the index of the active entry, or `-1` when the active combatant is hidden from this player and no placeholder is shown. The list contains real entries for combatants the player perceives (or controls), and `—` only if the DM enabled *Reveal hidden combatant count* — "Unknown" placeholders in the right slots for the others (placeholders carry no IDs). DMs receive the full list. Spectators receive the union view.

Implementation (`packages/server/src/rooms/views.ts`): keep, per client, the set of item IDs currently in view; after recomputation, diff and call `view.add` / `view.remove`. Because a `.view()` collection is `undefined` until the first add, the client sync layer must tolerate missing collections.

13.5 Message catalogue

Client → server messages are validated with zod (`packages/shared/src/protocol/messages.ts`) and rate-limited per user with token buckets. Every command carries a client command id `cid` for de-duplication after reconnects. Requests (`req`) use `room.request()` and return a typed response or a rejection `{ code, message }`.

Name	Kind	Who	Payload (summary)	Rate limit
<code>token.create</code> / <code>token.update</code> / <code>token.delete</code>	cmd	DM	token fields	20/s
<code>move.preview</code>	msg	controller	<code>tokenId</code> , <code>points≤64</code> , <code>cost</code>	15/s

Name	Kind	Who	Payload (summary)	Rate limit
<code>move.commit</code>	req	controller	<code>tokenId, points≤256, mode, elevations?, cid</code>	5/s
<code>move.reset / move.undoSegment</code>	req	controller	<code>tokenId</code>	5/s
<code>move.dash / move.standUp / move.mode</code>	req	controller	<code>tokenId / mode</code>	5/s
<code>token.elevation / token.facing / token.light</code>	cmd	controller	value	10/s
<code>door.toggle</code>	req	reach or DM	<code>wallId, action</code>	5/s
<code>wall.* / zone.* / light.*</code>	cmd	DM	geometry/fields (batch allowed ≤ 500 items)	20/s
<code>fog.paint</code>	cmd	DM	<code>sceneId, tool, shape, mode(reveal/hide), target(all/ userIds)</code>	30/s
<code>fog.resetExplored</code>	cmd	DM	<code>userIds \ "all"</code>	1/s
<code>scene.*</code>	cmd	DM	create/update/activate/preload/delete/reorder/calibrate	5/s
<code>combat.start / combat.stop / combat.next / combat.prev / combat.set / combat.reorder / combat.add / combat.remove</code>	cmd	DM	as named	10/s
<code>turn.end / turn.pip</code>	cmd	active controller or DM	<code>pip, used</code>	5/s
<code>dice.roll</code>	req	anyone admitted	<code>formula, label?, visibility, purpose?, context?</code>	5/s
<code>dice.manual</code>	req	anyone admitted	<code>formula, values[], label?, requestId?</code>	5/s
<code>request.create / request.close</code>	cmd	DM	targets, type, dc?, adv	2/s
<code>request.respond</code>	req	target	<code>requestId, roll \ manual \ skip</code>	5/s
<code>sheet.get</code>	req	owner, DM (others if public)	<code>actorId</code>	10/s
<code>sheet.patch</code>	req	owner (lock rules) or DM	JSON Patch (≤ 200 ops)	10/s
<code>sheet.propose / sheet.decide</code>	req	owner / DM	patch, note / decision	2/s
<code>cast.begin</code>	req	caster controller	<code>actorId, spellId, slot, targeting</code>	2/s
<code>resolution.update / resolution.apply / resolution.cancel</code>	req	DM (caster for own parts)	card edits	10/s
<code>effect.move / effect.end</code>	cmd	caster (per spell) or DM	geometry	5/s
<code>hp.apply</code>	req	DM (players via DM-grantable)	<code>targets[], parts[{amount,type}], mode</code>	10/s
<code>condition.set</code>	cmd	DM, owner for own	<code>tokenId, id, on, meta</code>	10/s
<code>deathsave.request</code>	cmd	DM	<code>tokenId</code>	2/s
<code>emote.send / ping.send / hand.toggle</code>	msg	anyone admitted	id/phrase, point	1/1.5 s burst 3; 3/s; 1/s
<code>handout.show / note.secret</code>	cmd	DM	recipients, content	2/s
<code>audio.music / audio.ambience</code>	cmd	DM	state changes	5/s

Name	Kind	Who	Payload (summary)	Rate limit
<code>history.undo</code> / <code>history.redo</code>	req	anyone (own entries)	—	5/s
<code>history.list</code> / <code>history.revert</code> / <code>history.restoreTo</code>	req	DM	filters / entryId	5/s
<code>view.as</code>	req	DM	<code>userId</code> \ <code>null</code>	2/s
<code>camera.spotlight</code>	msg	DM	point	1/s
<code>clock.sync</code>	req	anyone	<code>t0</code>	1/s
<code>lobby.decide</code>	req	Admin/DM (Ban: Admin only)	<code>sessionId</code> , <code>decision</code>	5/s
<code>table.kick</code>	req	Admin/DM	<code>userId</code> (back to the lobby)	2/s
<code>admin.ban</code> / <code>admin.unban</code>	req	Admin	<code>userId</code> , <code>reason?</code>	2/s
<code>fog.snapshot</code>	req	anyone admitted	<code>sceneId</code> → the reveal layers this client may see + its explored raster	2/s
<code>prep.open</code> / <code>prep.close</code>	req	DM	<code>sceneId</code> → full snapshot of a non-active scene (§13.7)	2/s
<code>measure.share</code>	msg	anyone admitted	shape + points (shown to others for 3 s)	5/s
<code>rest.short</code> / <code>rest.long</code>	req	DM	<code>actorIds[]</code> , per-actor exclusions	1/s
<code>combat.quickStart</code>	req	DM	—	1/s

Server → client messages: `token.moved` {`id`, `path`, `appear?`, `disappear?`, `durationMs`}, `roll.result`, `roll.masked`, `request.card`, `request.status` (DM), `resolution.card`, `resolution.update`, `hp.popup` {`tokenId`, `parts`}, `sheet.patch`, `effect.vfx`, `emote`, `ping`, `handout`, `note`, `toast`, `knock` (DM/Admin), `admitted` / `denied` / `banned` (lobby room), `explored.patch` {`sceneId`, `rect`, `rle`}, `fog.patch` {`sceneId`, `layer`, `rect`, `rle`} (painted-mode reveal changes this client may see), `prep.patch` {`sceneId`, `ops`} (DMs viewing a prep scene), `combat.view` {`entries[]`, `activeIndex`}, `measure.shared`, `audio.sync`, `camera.spotlight`, `table.closing`, `kicked`.

Rejection codes: `UNAUTHENTICATED`, `FORBIDDEN`, `NOT_YOUR_TURN`, `MOVEMENT_LOCKED`, `SPEED_ZERO`, `OVER_BUDGET`, `BLOCKED`, `INVALID`, `CONFLICT`, `NOT_FOUND`, `LOCKED_SHEET`, `RATE_LIMITED`, `TABLE_CLOSED`. Clients map each to a friendly toast.

Abuse handling: a client exceeding a bucket gets `RATE_LIMITED`; three bursts within 60 s disconnect it with a warning toast (it may reconnect).

13.6 Client sync (Colyseus 0.18)

```
import { Client, Callbacks } from "@colyseus/sdk";
import { Table } from "@gloam/shared/state";

const client = new Client(window.location.origin);
const room = await client.joinById(campaignId, {}, Table);
const cb = Callbacks.get(room);
cb.onAdd("tokens", (tok, id) => { // fires for existing items too
  entities.upsertToken(tok);
  cb.listen(tok, "hpFrac", () => entities.patchToken(id, { hpFrac: tok.hpFrac }));
  cb.onChange(tok.pos, () => registry.setTarget(id, tok.pos)); // nested schema
});
cb.onRemove("tokens", (_tok, id) => entities.removeToken(id));
room.onMessage("token.moved", (m) => registry.animatePath(m));
const res = await room.request("dice.roll", { formula: "1d20+5", visibility: "public" });
```

Batch store writes per patch (collect changes in callbacks and commit once in `room.onStateChange`) so React re-renders at most once per server patch (20 Hz).

13.7 DM prep view

The synchronised state holds only the **active** scene. When a DM opens another scene, the client calls `prep.open { sceneId }`; the server loads that scene into its in-memory model registry (so commands validate and plan against it exactly as for the active scene; it stays loaded while at least one DM has it open and is unloaded, after flushing, when the last one closes it) and returns a full snapshot of that scene (scene, walls, lights, zones, tokens, effects, fog layers) and subscribes that DM connection to `prep.patch { sceneId, ops }` messages for it. Editing commands already carry a `sceneId`; when it isn't the active scene, the command bus applies and persists the change as usual but, instead of touching the Colyseus state, sends `prep.patch` to the DMs subscribed to that scene. `prep.close` (or opening another scene) unsubscribes. Players are never subscribed. Activating a prep scene loads it into the state for everyone (F03).

13.8 Clock synchronisation

`clock.sync` five times at join (and every 60 s): the client sends `t0`, the server replies with `serverNow`; keep the sample with the lowest round-trip time; `offset = serverNow - (t0 + rtt/2)`. Used for music, timed effects and dice timing.

14. Commands, permissions and history

14.1 The command bus

Every mutation — from the room or from REST — is a **command** handled by `packages/server/src/engine/commandBus.ts`:

```
interface CommandDef<P> {
  type: string; // e.g. "token.move"
  schema: z.ZodType<P>; // strict
  undoable: boolean;
  authorize(ctx: Ctx, p: P): Allow | Deny; // uses permissions.ts + house rules + overrides
  plan(ctx: Ctx, p: P): Plan; // pure: reads model, returns ops + effects; no mutation
}
interface Plan { ops: Op[]; summary: string; after?: PostCommit[] } // after: vision, sounds, log, toasts
```

Execution order (synchronous up to broadcast):

1. Parse with zod (reject `INVALID`).
2. `authorize` (reject `FORBIDDEN`, `NOT_YOUR_TURN`, ...).
3. `plan` against the in-memory model (reject domain errors like `OVER_BUDGET`).
4. **One SQLite transaction**: apply `ops` to the affected rows + insert the history row.
5. Apply `ops` to the in-memory model and the Colyseus state.
6. Run post-commit effects: vision invalidation and view sync, messages, sounds, log entries.
7. Respond (for requests) with the result.

If step 4 throws, nothing else happens and the client gets an error toast. Because better-sqlite3 is synchronous, steps 4–5 cannot interleave with other commands.

14.2 Permissions

`packages/shared/src/rules/permissions.ts` exposes `can(actor, action, target, ctx)`, built from the matrix in §6, the campaign's house rules and per-token overrides. The server calls it in every command's `authorize`; the client imports the same module to decide which controls to show, but the server's answer is final.

14.3 Operations

```
type Op =
  | { k: "set"; e: EntityKind; id: string; path: string[]; value: unknown; prev: unknown }
  | { k: "create"; e: EntityKind; id: string; value: unknown }
  | { k: "delete"; e: EntityKind; id: string; prev: unknown }
```

```
| { k: "fog";   sceneId: string; layer: string; rect: Rect; before: Uint8Array; after: Uint8Array } // RLE
| { k: "sheet"; actorId: string; patch: JsonPatch; inverse: JsonPatch };
```

`inverse(ops)` = reversed list with `set.value↔prev`, `create↔delete`, `fog after↔before`, `sheet patch↔inverse`.

14.4 Undo, redo and conflicts

- Each user has an in-memory undo stack (max 200) of their undoable history IDs for the current table session, plus a redo stack cleared by any new command from that user.
- Undo** entry E: compute E's *touched keys* (`e:id:path` for sets, `e:id` for create/delete, fog layer rects, sheet paths). If any later, not-undone entry by anyone touches an overlapping key → conflict: players get **CONFLICT** with a description; DMs may force (`history.undo { force: true }`), which applies the inverse of E only to the keys where the current value still equals E's `value`, and for the others sets `prev` anyway (last writer wins), recorded as a new entry "Forced undo of ...".
- Undo is itself executed as a command whose ops are `inverse(E.ops)`; the original entry gets `undone_at`. Redo re-applies `E.ops` after checking that the keys still hold the undo's values.
- Revert** (DM, any entry) = undo with force semantics. **Restore to here** = revert every later entry in reverse order inside one transaction.
- Not undoable (`undoable: false`): dice rolls, joins/kicks/bans/approvals, emotes/pings/hand raises, music playback, snapshots.
- Movement special case: `move.reset` returns to `turnStart` and restores the budget; it's one command (undoable) with ops that restore position, `usedFt` and the segment list.

15. Vision and lighting engine

All geometry and perception logic lives in `packages/shared/src/{geometry,vision}` and is used by both the server (authoritative decisions) and the client (rendering). Coordinates are feet on the 2D map plane.

15.1 Inputs

- Walls** with derived flags (§8.7 matrix): `blocksMove`, `blocksSight`, `blocksLight`. Open doors block nothing. Opaque effect walls (Wall of Fire, Prismatic Wall) are added as temporary sight- and light-blocking segments while active.
- Lights** (§8.8), including token-carried lights at the token's position and the `light` of active effects (Daylight, Moonbeam, Flaming Sphere...); DM-only lights are excluded (they are a DM vision aid).
- Obscuring volumes** from effects: `heavy` (Fog Cloud etc.), `light` (Web etc.), `magicalDarkness` (Darkness), each a 2D footprint polygon (circles as 48-gons) plus a vertical extent.
- Viewers**: creatures a player controls or has shared vision of, with position, elevation, base radius, senses, and conditions (Blinded, Unconscious).
- Scene ambient** level: bright, dim or dark.

15.2 Visibility polygons

`visibilityPolygon(eye, segments, radius)` returns the region visible from `eye` within `radius`, bounded by the given segments.

- Cull segments with a uniform spatial hash (cell 10 ft) to those intersecting the circle of `radius` around `eye`; add a 64-gon at `radius` as bounding segments.
- Algorithm: **rotational sweep** (Asano; see Red Blob Games' "2D visibility" article). Sort endpoint events by angle; maintain the set of segments crossing the current ray ordered by distance; emit a vertex whenever the nearest segment changes. $O(n \log n)$.
- Robustness: ignore zero-length segments; treat endpoints within $1e-6$ ft as equal; if the eye lies on a wall line, nudge it 0.01 ft toward the token centre; handle collinear overlapping segments; T-junctions must not leak.
- Property test: for 2 000 random scenes, compare point-in-polygon results against a brute-force ray caster at 500 random points each; zero disagreements outside a 0.01-ft tolerance band.

Polygons per purpose:

Polygon	Blocking set	Used for
LOS_sight(v)	blocksSight walls + opaque effect walls	Normal vision, darkvision, truesight
LOS_blind(v)	blocksMove walls (physical barriers; windows block, curtains don't)	Blindsight ("not behind total cover")
LIT(l)	blocksLight walls + opaque effect walls; for cone lights also clipped to the cone	Area a light reaches
REACH(v)	blocksMove walls	Door reach, line of effect

15.3 Perception of a point

For viewer **v** and point **p** at distance **d** (3D distance including elevation difference):

```

perceive(v, p):
  if v.unconscious:                return NONE          # unaware of surroundings: no senses at all
  if v.blindsight ≥ d and p ∈ LOS_blind(v): return BLIND      # ignores light, obscurement, invisibility
  if v.blinded:                    return NONE
  if p ∉ LOS_sight(v):              return NONE
  for each obscuring volume 0 crossed by segment v→p or containing p or v:
    if 0.magicalDarkness and not (v.truesight ≥ d): return NONE
    if 0.heavy (not darkness):      return NONE          # truesight doesn't pierce fog
  L = lightLevel(p)                # BRIGHT | DIM | DARK (below)
  if L == BRIGHT:                  return BRIGHT
  if L == DIM: return (v.darkvision ≥ d) ? BRIGHT : DIM      # darkvision: dim counts as bright
  # L == DARK
  if v.truesight ≥ d:               return DARKVISION        # sees in normal and magical darkness
  if v.darkvision ≥ d and not inMagicalDarkness(p): return DARKVISION # greyscale
  return NONE

lightLevel(p):
  if inMagicalDarkness(p): only lights with magical=true and pierceDarkness=true count, ambient = DARK
  level = scene.ambient
  for each enabled light L with p ∈ LIT(L):
    r = dist2D(L, p)
    if r ≤ L.bright: level = max(level, BRIGHT)
    elif r ≤ L.bright + L.dim: level = max(level, DIM)
  return level

```

- **pierceDarkness** is true only for lights created by spells the rules say dispel or overcome the darkness (Daylight over Darkness of level ≤ 3, Sunburst). The spell system also shows a DM hint when a Daylight area overlaps a Darkness effect ("Daylight dispels this Darkness — end it?").
- Lightly obscured volumes don't change **perceive**; they add a haze in rendering and a "disadvantage on sight-based Perception" hint.
- A viewer inside a heavily obscuring volume perceives only its own base (plus blindsight).

15.4 Perception of a creature

For a player **P** and a creature token **t** (not DM-hidden):

1. **t** is always perceivable if **P** controls it, or its **revealTo** is **all** or includes **P**.
2. Otherwise, take 9 sample points of **t**: centre and 8 points at $0.8 \times$ base radius. For each viewer **v** of **P**, **t** is **seen** if any sample is perceived as **BLIND**, **BRIGHT**, **DIM** or **DARKVISION**.
3. **Invisibility**: if **t** has the Invisible condition and is not **outlined** (Faerie Fire), a sample counts only if it is perceived as **BLIND**, or it is perceived by sight (**BRIGHT** / **DIM** / **DARKVISION**) **and** the viewer has **truesight** ≥ **d** or **seeInvisible**.
4. **Tremorsense**: if not seen, but some viewer that isn't Unconscious has **tremorsense** ≥ **d**, **t.elevation** = 0, and **t** isn't flagged as flying/incorporeal → **sensed** (a **Sensed** entry with an opaque per-viewer ID and the position rounded to 1 ft).
5. Spectators perceive the union of all players' perceptions.

15.5 The server vision service

`packages/server/src/vision/visionService.ts` keeps caches and recomputes only what changed.

- **Caches:** wall spatial hash per scene; `LIT(ℓ)` per light (invalidate when the light moves or changes, or when walls/doors within its radius change); `LOS_*` per viewer token (invalidate on its move, on wall/door changes within its range, on condition/sense changes).
- **Invalidation events** from commands: token moved/created/deleted, door toggled, wall/zone/light/effect changed, condition changed (Blinded, Invisible, Unconscious), senses or vision sharing changed, scene fog mode changed, player joined/left.
- **Recompute** for affected players only: perceivable token set, sensed set → diff → `view.add/remove` (§13.4); tag eligibility (`TAG_HP`, `TAG_OWNER`, `TAG_VISION`, `TAG_LINK`); spectators.
- **Explored memory** (dynamic fog mode): per player and scene, a bitset raster at `fog_cell_ft` (default 1 ft) over the scene bounds. After a recompute, rasterise the player's visible region (union over viewers of `LOS_sight ∩ lit-or-darkvision`) by scanline-filling the LOS polygon and testing each cell centre against a cached **light raster** (per scene, 1-ft cells storing max light level, rebuilt when lights or walls change). OR into the explored raster; mark the dirty rectangle; send `explored.patch` to that player; persist dirty rasters every 5 s (deflate). **The server's explored raster is the only explored memory:** clients render it (upsampled and blurred) and never accumulate their own, so memory can't drift between devices or reloads. Spectators receive the union of all players' explored rasters.
- **Budget:** recompute for 8 players with 500 walls and 50 lights under 10 ms at p95 (measured in `pnpm bench`). If exceeded, move the service to a worker thread.

15.6 Moves seen partially

When a token moves along path Π , for each player P who could perceive it at the start or end, or at any point along the path: sample Π every 1 ft; mark samples where t would be perceivable if standing there; send P a `token.moved` with the sub-path from the first to the last perceivable sample, plus `appear` (fade in at the first sample) and/or `disappear` (fade out at the last sample) flags. Players who perceive none of it receive nothing, and the token is added to or removed from their view accordingly. The destination is never sent to a player who can't perceive it there.

15.7 Client rendering pipeline

The client computes its own rendering from the data it legitimately has (walls except hidden ones, non-DM-only lights, visible effects, its own viewers and their senses). DMs compute it for the selected **View as** player.

1. **Render targets** covering the scene bounds at `px_per_ft` (tier: Ultra 6, High 4, Medium 3, Low 2; capped at 4096²): - `visionRT` (RGBA8): R = normal sight LOS, G = within darkvision range $\cap$ LOS, B = within blindsight range $\cap$ LOS_{blind}, A = within truesight range $\cap$ LOS. Each viewer's polygons are drawn already clipped to that viewer's sense ranges (LOS $\cap$ darkvision circle into G, and so on) with max blending, so any number of viewers (a party-vision spectator, shared familiars) needs no per-viewer shader uniforms. Walls of kind `occluder` block sight and light here exactly as on the server. - `lightRT` (RGBA16F): R = bright coverage, G = dim coverage (radial falloff with a 2-ft soft edge), B = magical flag; `lightColorRT` for tint. - `obscureRT` (RGBA8): R = heavy, G = magical darkness, B = light haze. - `exploredRT` (R8): the server's explored raster for this player (from `fog.snapshot` and `explored.patch`), upsampled and blurred 1 cell; in painted mode, the player's revealed layers (§15.8).
2. Draw polygons from the shared visibility code as triangle fans (via `ShapeGeometry` or a custom fan buffer) into the targets. Redraw only when inputs change (not every frame), except animated light flicker, which modulates intensity uniforms.
3. **Composite** in the map material and in a board-wide overlay pass: per fragment compute the perception class exactly as §15.3 (from the RT channels alone — no per-viewer uniforms, so the number of viewers is unlimited), then grade: BRIGHT full colour; DIM 55% brightness, slight cool shift; DARKVISION luminance-only greyscale (with a faint grain); BLIND greyscale with a subtle ripple; remembered (exploredRT but not visible) desaturated 35% brightness with a blue tint; unknown → the war-fog shader (animated fbm, deep blue-black, soft 1–2 ft feather).
4. **Tokens** are shaded with the same light data (so a mini in dim light looks dim) — tokens outside perception are simply absent from state.
5. **3D minis** also receive real point lights from the N nearest light sources (tier: Ultra 8, High 4, Medium 2, Low 0), with intensities matching the light map.
6. Reveals animate: blend the new `visionRT` over 300 ms.

15.8 Painted fog mode and what map pixels reveal

- **Layers:** each scene has an all-players reveal layer plus optional per-player layers (`fog_masks: reveal:all, reveal:<userId>`). A player's revealed area = `reveal:all u reveal:<their id>`.
- **Delivery:** on joining a scene a client calls `fog.snapshot`, receiving its revealed area (RLE bitsets) and, in dynamic mode, its explored raster. DM fog operations produce `fog.patch { sceneId, layer, rect, rle }` sent only to the players whose revealed area that layer affects (DMs get every layer). Undo of a fog operation (AC-UNDO-01) produces the inverse patch the same way.
- **Token visibility in painted mode:** a non-hidden token is visible to a player if any of its 9 sample points lies in that player's revealed area, or it is revealed to them explicitly (`revealTo`), or they control it. Recompute on paint, on token move, and on scene change; diff views as in §13.4.
- **In dynamic mode,** painted reveals count as explored memory (desaturated map, no creatures); only live perception shows creatures.
- **Map pixels are not secret.** Every admitted player downloads the whole map image (§21.6), so fog of war hides the map only visually. Principle P4 protects entities — tokens, hidden and secret walls, notes, rolls, sheets — not map art. A DM who needs truly secret areas puts them in a separate scene.

16. Movement and pathfinding engine

Lives in `packages/shared/src/movement`; identical code runs in the browser (preview) and on the server (authority).

16.1 Obstacles

- **Walls** with `bBlocksMove`: walls, closed/locked doors, windows, invisible walls, secret doors. On the client, only walls the player knows about; on the server, all walls (hidden walls cause truncation, §16.5).
- **Impassable zones** (polygon edges).
- **Scene bounds.**
- **Creature bases** (players' moves when *Enforce creature spaces* applies): a non-ally creature that isn't Tiny or Incapacitated and is within one size category of the mover is a solid circle; other non-ally creatures (Incapacitated, or two or more sizes different) are passable difficult terrain unless Tiny; allies are ignored. The end point must not overlap any creature by more than 50% of the smaller base diameter.

16.2 Clearance

A moving creature is a circle of radius `r_c = squeeze × sizeFt`, default `squeeze = 0.4` (Tiny 1.0 ft, Small/Medium 2.0 ft, Large 4.0 ft, Huge 6.0 ft, Gargantuan 8.0 ft). This lets a Medium creature pass a corridor drawn as "5 ft" even if the DM's wall lines are slightly inside, and matches D&D's leniency about squeezing. The value is a campaign setting.

16.3 Graph and search

- **Nodes:** for every wall endpoint, zone vertex and creature-obstacle circle, 12 points on a circle of radius `r_c + 0.05` around it; keep nodes that are inside the scene bounds and not within `r_c` of any obstacle. Plus start, goal and waypoints.
- **Edges:** between two nodes if the segment between them keeps clearance `r_c` from every obstacle (segment–segment distance $\geq r_c$; circle obstacles: point–segment distance $\geq r_c + r_{\text{obstacle}}$). Build lazily during A* (test neighbours within `remainingBudget + 10 ft` of the start), cache edge results per scene version.
- **Cost of an edge** = its length integrated over regions (§16.4). **Heuristic:** straight-line distance (admissible, because every multiplier is ≥ 1).
- **Result:** a polyline of nodes (already taut). Expose `route(start, goal, waypoints, opts) → { points, cost, segments: [{from, to, cost, difficultFt}] }`.
- **Performance:** under 4 ms for 500 walls on a mid-range laptop (preview runs on pointer move, throttled to one query per animation frame).

16.4 Cost integration

For each segment, split it at every crossing with zone boundaries and creature circles, and sum `length × multiplier` for each piece:

```
multiplier = 1
+ 1 if inside difficult terrain, or a passable non-ally non-Tiny creature's space,
    or water without a swim speed          # difficult terrain is never cumulative
+ 1 if crawling (prone and not standing up), or climbing without a climb speed,
    or swimming without a swim speed (when not already counted above)
```

This yields the SRD costs: normal 1, difficult 2, crawling 2, crawling through difficult 3. Flying movement uses 3D segment length (elevation changes count). All values in feet; compare against budgets with $\epsilon = 0.05$ ft.

16.5 Server validation (`move.commit`)

1. Permission: controller of the token (or DM); not `locked`; not `lockMovement`; in combat and not DM: token must be the active combatant and the player its controller (unless free movement).
2. Speed-zero conditions (unless `ignoreConditionSpeed` or DM) → `SPEED_ZERO`.
3. The path must start within 0.5 ft of the current position; at most 256 points; every point inside the scene bounds.
4. Walk the path against the `true` obstacle set. At the first collision, truncate at the contact point minus `r_c`, set `bumped = true` (players get the "unseen" toast if the blocking wall is hidden from them).
5. Compute cost (§16.4). If a budget applies and `cost > remaining + ε`: house rule `clamp` → cut the path at arc length where the cumulative cost equals `remaining`; reject → `OVER_BUDGET`.
6. Creature-space rule at the end point (if enforced): if the end overlaps illegally, walk back along the path to the last legal point; if none, reject `BLOCKED`.
7. Commit: new position, `usedFt += cost`, push the segment onto the turn's segment list (for undo), history entry, then post-commit vision and `token.moved` messages (§15.6).

Budget bookkeeping per combatant turn: `budget = effectiveSpeed(mode) × (1 + dashes) + bonusMove` (§19.4), `used`, `turnStart` (position, elevation and prone state), `segments[]`. `Reset` puts the token back at `turnStart` exactly as it was (including prone), sets `used = 0` (which refunds any stand-up cost), and keeps any Dash already granted this turn. `Undo segment` pops the last entry of `segments[]` and refunds its cost.

16.6 Movement range field

For the selected token (remaining budget in combat; one move at its speed in exploration): compute a geodesic distance field with the **fast marching method** on a grid centred on the token covering `remaining + 5 ft` (cell 0.5 ft when `remaining ≤ 30` ft, else 1 ft), with speed `1 / multiplier` per cell and walls as blocked cell transitions (a transition is blocked if the segment between the two cell centres intersects a movement-blocking wall inflated by `r_c`). Run it in a Web Worker; target under 30 ms for a 60-ft budget. Render as a soft floor overlay (alpha falling off toward the limit) with a bright isoline at `remaining`. The overlay is a visual aid; the authoritative cost is always the path cost.

16.7 Animation and facing

Tokens glide along the received path at 30 ft/s (minimum 250 ms, maximum 2 s total, ease-in/out over the first and last 120 ms). 3D minis turn to face their direction of travel (setting *Auto-facing*, default on) with a 150 ms slerp. A soft footstep thump plays every 5 ft for tokens within 60 ft of the camera target.

17. Areas of effect and targeting

Lives in `packages/shared/src/aoe`.

17.1 Shapes (2024 definitions, SRD 5.2.1)

```
type AreaShape =
  | { kind: "sphere";    origin: Vec3; radius: Ft }           // origin included
  | { kind: "cylinder";  origin: Vec3; radius: Ft; height: Ft } // origin at centre of top or
bottom
  | { kind: "cone";      origin: Vec3; dirDeg: number; length: Ft } // width at distance x equals x
  | { kind: "cube";      origin: Vec3; dirDeg: number; size: Ft; originOnFace: boolean }
  | { kind: "line";      origin: Vec3; dirDeg: number; length: Ft; width: Ft } // default width 5
  | { kind: "emanation"; sourceTokenId: string; distance: Ft } // extends from the creature's
space
  | { kind: "wall"; points: Vec2[]; closed: boolean; height: Ft; thickness: Ft; // Wall of Fire / Stone / Force...
    opaque: boolean; blocksMove: boolean; damagingSide?: "left"|"right"|"both" };
```

2D footprints on the map plane:

- **Sphere / cylinder:** circle of `radius` around the origin.
- **Cone:** isosceles triangle with apex at the origin, axis along `dirDeg`, length `L`, half-angle `atan(0.5) ≈ 26.565°` (full angle `≈ 53.13°`), far edge width `L`.
- **Cube:** square of side `size`; when originating from a creature, one face is centred on the creature's base edge facing `dirDeg`; when placed at range, the user positions the square directly (rotation in 15° steps).
- **Line:** rectangle `length × width` starting at the origin along `dirDeg`.
- **Emanation:** circle of radius `distance + sourceBaseRadius` centred on the source; it moves with the source unless the effect is instantaneous or stationary.
- **Wall:** a polyline (or a closed ring) of the given thickness; while active, opaque walls add temporary sight- and light-blocking segments and solid walls add movement-blocking segments; triggers can apply to one side (the damaging side of Wall of Fire) or both.

Vertical extent (for flying targets): sphere `origin.z ± radius`; cylinder `[base, base + height]`; cone and line `origin.z ± (width at that distance)/2`; cube `[base, base + size]`; emanation `source.z ± distance`.

17.2 Origins

Point-origin spells (Fireball) place the origin at a chosen point within range. Self-origin cones, lines and cubes (Burning Hands, Lightning Bolt, Thunderwave) put the origin on the caster's base edge in the aimed direction, so the caster is outside the area. Per the SRD, the origin point of a cone, cube, line or emanation is **not** included unless the creator chooses (a checkbox "Include myself" on the template).

17.3 Who is affected

A creature is affected if:

1. its base circle intersects the footprint (house rule *Area coverage*: `touches` (default) — any overlap; `centre` — its centre is inside), and
2. its elevation range overlaps the vertical extent, and
3. **line of effect:** at least one of its 9 sample points has an unblocked straight line from the origin point, where blocking walls are those that provide total cover (`blocksMove`); if all lines are blocked, it's excluded (shown on the card as "blocked", and the DM can add it back).

Range: measured from the caster's base edge to the origin point; *Touch* = within the caster's reach (5 ft default); *Self* has no range. The template turns ember-red when out of range or when the origin point isn't reachable by line of effect from the caster.

17.4 Mapping spells to shapes

The content importer (§33) produces `area` data for spells; the targeting UI reads it. For spells where the data has no area, the UI falls back to target selection. The DM can always switch the template shape or size on the fly (P2).

17.5 Cover hint

From the attacker's centre (or the area's origin) cast 5 rays to the target: centre and 4 points at $0.9 \times$ base radius, perpendicular pairs relative to the line of attack. Count rays blocked by walls with `blocksMove` or `blocksSight`: 0 → no cover; 1–2 → half cover (+2 AC and Dex saves); 3–4 → three-quarters cover (+5); 5 → total cover. If the centre ray passes through another creature's base (neither attacker nor target), report at least half cover. It's a hint on the card; the DM decides.

18. Dice engine

Lives in `packages/shared/src/dice` (parser, evaluator with an injectable RNG, formatter). The server evaluates; the client parses only for validation and highlighting.

18.1 Grammar

```
formula = expr [ ws tag ] [ ws ( "adv" | "dis" ) ] ; (* a trailing tag types every untagged term *)
expr    = term { ( "+" | "-" ) term } ;
term    = factor { ( "*" | "/" ) factor } ; (* "/" rounds down *)
factor  = "-" factor | primary ;
primary = number | dice | "(" expr ")" | ref ;
dice    = [ count ] "d" sides { modifier } [ tag ] ;
count   = number | "(" expr ")" ;
sides   = number | "%" ;
modifier = keep | drop | reroll | explode | minmax ;
keep    = ( "kh" | "kl" | "k" ) [ number ] ; (* k = kh; default 1 *)
drop    = ( "dh" | "dl" ) [ number ] ;
reroll  = ( "r" | "ro" ) [ compare ] ; (* r: repeat while matching (max 20); ro: once; default =1 *)
explode = "!" [ compare ] ; (* default: on max face *)
minmax  = ( "min" | "max" ) number ; (* e.g. min3 for Great Weapon Fighting *)
compare = [ "<" | ">" | "<=" | ">=" | "=" ] number ;
tag      = "[" label "]" ; (* damage type or free label, e.g. [fire] *)
ref      = "@" ident { "." ident } ; (* @str @dex @prof @level @spellmod @str.save @skill.stealth @init *)
```

- `adv` / `dis` turn the first `1d20` term into `2d20kh1` / `2d20kl1`.
- `@` references resolve against the roller's sheet (or the unit's stats) on the server; unknown refs are errors.
- Tags aggregate totals per damage type (`byTag`), which feeds the damage pipeline. A tag after a dice term applies to that term; a **trailing** tag after the whole formula applies to every term that has no tag of its own (so `1d8 + @str [bludgeoning]` is 100% bludgeoning); untyped totals are marked `untyped` and the DM picks the type when applying.
- **Limits**: count ≤ 100 per term; ≤ 500 dice rolled in total including rerolls and explosions; sides 1–1000; explosion depth ≤ 20 ; formula length ≤ 200 characters. Violations are `INVALID` with a readable message.

18.2 Randomness and results

- Production: `crypto.randomInt(1, sides + 1)` per die.
- Test only (`NODE_ENV=test` and `GLOAM_TEST_SEED` set): a seeded xoshiro128** generator.

```
interface RollResult {
  id: string; formula: string; normalized: string; label?: string;
  terms: ({ kind: "dice"; count: number; sides: number; tag?: string;
    dice: { value: number; kept: boolean; exploded: boolean; rerolledFrom?: number[] }[];
    subtotal: number } | { kind: "const"; value: number } | { kind: "ref"; ref: string; value: number })[];
  total: number; byTag: Record<string, number>;
  natural?: number; // first kept d20 of a d20 test
  crit?: boolean; fumble?: boolean;
  seed: number; // uint32 for the 3D animation
  visibility: "public" | "dm" | "blind" | "self"; purpose?: string; manual: boolean;
}
```

Critical hits (house rule *Critical damage*): `double dice` (SRD default: roll all damage dice twice) or `max + roll`.

18.3 Visibility

Mode	Roller	DMs	Other players
Public	full	full	full
Private to DM	full	full	"rolled privately" (no values)
Blind (used by DM roll requests, and by DMs for their own rolls)	a player roller sees "?" (no values)	full	"rolled for the DM" (no values)
Self	full	"rolled for themselves"	nothing

The server sends each client only what its row allows (`roll.result` or `roll.masked`). Masked rolls still animate dice with "?" faces for fun.

18.4 3D dice that land on the server's numbers

- **Worker:** `packages/web/src/dice/physics.worker.ts` loads `@dimforge/rapier3d-deterministic-compat` lazily (prefetched after the table first renders, during idle time).
- **Initial conditions** from the roll's `seed` via `mulberry32`: spawn position along the thrower's edge of the tray, linear velocity toward the centre with jitter, random angular velocity and initial rotation. Because the physics build is deterministic and the seed is shared, every player sees the same tumble.
- **World:** 1 unit = 1 cm, dice ≈ 1.6 units across, gravity -981 units/s² (real gravity at this scale); a floor and four invisible walls matching the visible tray area; restitution 0.3, friction 0.6, damping 0.1/0.1; fixed timestep 1/120 s; `setCanSleep(true)`; step until every body sleeps (cap 1 200 steps; if not asleep by then, tween to the nearest face-up rest pose over 200 ms). Collider: `ColliderDesc.convexHull(vertices)` per die type.
- **Recording:** per step, position and quaternion per die into a transferable `Float32Array`; contact force events (magnitude, die index, step) for sound.
- **Landed face:** for each die type, precompute face normals (local) and labels. Face up = the face whose rotated normal has the largest dot product with +Y. For the **d4**, the result is read at the top vertex: use the face pointing down and map it to the label of the opposite vertex.
- **Remap to the desired value:** precompute the die's rotation symmetry group as quaternions (generate by composing generator rotations until closure: d4 12, d6 24, d8 24, d10 10, d12 60, d20 60 elements). Find `S` in the group with `S · n_desired = n_landed` (vertices for d4). Render the visual mesh with rotation `q_body · S`. Because `S` is a symmetry of the solid, the mesh occupies exactly the same space as the physics body, so contacts look identical, and the face that ends up on top shows the server's number. (The rotation groups act transitively on faces, so `S` always exists.)
- **d10 / d100:** pentagonal trapezohedron; a single d10 labelled 0–9 reads 0 as 10; percentile uses a tens die (00–90) plus a units die (0–9), with 00 + 0 = 100.
- **Geometry and skins:** procedural polyhedra with per-face UVs into a canvas-drawn number atlas per skin (numbers in Fraunces, underline on 6 and 9); a fake bevel via barycentric edge darkening in the shader; materials by skin (`MeshPhysicalMaterial` with transmission for gemstone on High/Ultra, `MeshStandardMaterial` otherwise). A transparent `ShadowMaterial` plane catches dice shadows so they seem to roll over the board.
- **Playback:** main thread interpolates recorded frames; contact events trigger clack sounds (§31); the card's total appears when all dice settle; dice fade out 2.5 s later.

18.5 Roll requests

```
interface RollRequest { id: string; createdBy: string; targets: string[]; // token or actor IDs; cards go to
  their controllers
  type: "check"|"save"|"attack"|"custom"; ability?: Ability; skill?: SkillId; formula?: string;
  dc?: number; showDc: boolean; adv: "none"|"adv"|"dis"; visibility: RollResult["visibility"];
```

```
responses: Record<string, { state: "pending"|"rolled"|"manual"|"skipped"|"dm"; rollId?: string;
                           total?: number; success?: boolean }>; status: "open"|"closed" }
```

The server computes each target's formula from their sheet (e.g. DEX save = `1d20 + @dex.save`), sends `request.card` to each target, and streams `request.status` to DMs.

19. Rules engine and house rules

Pure functions in `packages/shared/src/rules`, unit-tested, used by the server when planning commands and by the client for previews. Rules produce **suggestions and prompts**; commands apply them only after confirmation when the automation level is *Assist* (default).

19.1 Automation levels

Manual (board and dice only; no prompts), *Assist* (default: prompts and previews, DM confirms consequences), *Auto* (deterministic consequences apply immediately; everything stays undoable).

19.2 Damage and healing

```
function applyDamage(t: Combatant, parts: { amount: number; type: DamageType | "untyped" }[],
                  o: { halved?: boolean; crit?: boolean }): DamagePreview {
  // Flat bonuses/penalties are already inside the typed parts (tags, §18.1), so nothing is added twice.
  const resist = new Set(t.resist), immune = new Set(t.immune), vuln = new Set(t.vuln);
  if (t.conditions.includes("petrified")) ALL_DAMAGE_TYPES.forEach((d) => resist.add(d)); // condition-granted
  let total = 0;
  for (const part of parts) { // each part is one damage instance
    let a = part.amount;
    if (o.halved) a = Math.floor(a / 2); // "half damage on a successful save" is a multiplier
    if (part.type !== "untyped") {
      if (immune.has(part.type)) a = 0;
      else {
        if (resist.has(part.type)) a = Math.floor(a / 2); // resistance: halve, round down
        if (vuln.has(part.type)) a = a * 2; // vulnerability: double
      }
    }
    total += Math.max(0, a);
  }
  const fromTemp = Math.min(t.hpTemp, total);
  const rest = total - fromTemp;
  const hp = Math.max(0, t.hp - rest);
  const overflow = Math.max(0, rest - t.hp);
  return { total, fromTemp, hp, overflow,
    massiveDeath: t.isPC && hp === 0 && overflow ≥ t.hpMax,
    concentrationDc: total > 0 && t.concentrating ? Math.min(30, Math.max(10, Math.floor(total / 2))) : null
  };
}
```

Resistance and vulnerability each apply once per damage instance. Healing: `hp = min(hpMax, hp + amount)`; from 0 HP it removes Unconscious (if caused by 0 HP) and resets death saves.

19.3 Conditions metadata (drives hints and some enforcement)

Condition	speedZero	incapacitated	noSight	autoFail STR/DEX saves	Attacks against it	Its attacks	Other
Blinded			✓		advantage	disadvantage	fails sight checks
Charmed						can't target charmer	charmer has adv. on social checks
Deafened							fails hearing checks

Condition	speedZero	incapacitated	noSight	autoFail STR/DEX saves	Attacks against it	Its attacks	Other
Exhaustion (lvl)							d20 tests $-2 \times \text{lvl}$; speed $-5 \times \text{lvl}$ ft; lvl 6 dies
Frightened						disadvantage (source in sight)	disadvantage on ability checks (source in sight); can't move closer to source (hint)
Grappled	✓					disadvantage vs others than grappler	
Incapacitated		✓					no actions/reactions; breaks concentration; disadvantage on initiative
Invisible					disadvantage (unless seen)	advantage (unless seen)	advantage on initiative
Paralyzed	✓	✓		✓	advantage; hits within 5 ft are crits		
Petrified	✓	✓		✓	advantage		resistance to all damage
Poisoned						disadvantage	disadvantage on ability checks
Prone					adv. within 5 ft, else disadvantage	disadvantage	crawl $\times 2$; stand = half speed
Restrained	✓				advantage	disadvantage	disadvantage on DEX saves
Stunned		✓		✓	advantage		(5.1 only: can't move)
Unconscious	✓	✓	✓	✓	advantage; hits within 5 ft are crits		also Prone; drops items

Summaries shown in tooltips come from §34.1. When the campaign uses the SRD 5.1 pack, use the 5.1 variants (Exhaustion table, Stunned "can't move", no Bloodied).

19.4 Speed and budget

```

effectiveSpeed(mode) =
  0                                if any speedZero condition and not ignoreConditionSpeed
  else max(0, (speedOverride ?? sheetSpeed[mode]) - 5 × exhaustion)
turn budget    = effectiveSpeed(mode) × (1 + dashes) + bonusMove    # bonusMove (DM override) is never doubled by Dash
remaining      = max(0, turn budget(current mode) - used)          # switching mode keeps `used`
stand up cost  = floor(effectiveSpeed(walk) / 2)

```

19.5 Other rules

- **Death saves:** state machine `alive` → `atZero(dying)` → `stable` | `dead`, with the transitions in F11. DC 10.
- **Concentration:** DC `min(30, max(10, [damage/2]))` per damage instance; ends on failure, on Incapacitated, on death, or on casting another concentration spell.
- **Initiative:** `1d20 + initMod` where `initMod = dexMod + initBonus`, with advantage/disadvantage from Invisible (adv), Incapacitated (dis), Surprised (dis); fixed initiative = `10 + initMod`, +5 with advantage, -5 with disadvantage; sort per F12.
- **Temporary HP:** don't stack; choose keep or replace.
- **Proficiency bonus:** `2 + floor((totalLevel - 1) / 4)`.
- **Spell save DC** `8 + prof + mod`; **spell attack** `prof + mod`.

19.6 House rules (campaign settings; defaults first)

Rule	Options
Rules pack	SRD 5.2.1 · SRD 5.1
Automation level	Assist · Manual · Auto
Overlong moves	Clamp to max reach · Reject
Move reset	Always during own turn (Xini's choice) · Until an action is used (stops move–attack–reset) · Never
Enforce creature spaces	In combat · Always · Never
Area coverage	Touches · Centre inside
Critical damage	Double dice · Max dice + roll
Initiative ties	Dex mod, then PCs first · DM decides each time
NPC at 0 HP (pre-selected choice in the DM prompt; applied without a prompt under <i>Auto</i>)	Dead · Unconscious · Keep at 0
Default initiative method (used by Quick start)	Players roll, DM rolls NPCs · Roll for everyone · Fixed initiative · Skip rolls
Death saves visible to	Everyone · Owner and DM
Bloodied marker	On · Off
Party vision	Off · On
Player-applied damage	Via DM confirmation · Direct
Hidden combatant count	Hidden · Revealed as "Unknown"
Sheet lock default	Unlocked · Core locked · Fully locked
NPC HP display default	Bar · Exact · Descriptor · Hidden
Squeeze factor	0.4 (0.3–0.5)
Exploration movement	Free · Limited to speed per move

20. Persistence, snapshots and backups

20.1 SQLite

- Open with better-sqlite3; pragmas: `journal_mode = WAL`, `synchronous = FULL` (an accepted change must survive a power cut or OS crash, not just a process crash — P5; the write rate is low enough that the fsync cost doesn't matter), `foreign_keys = ON`, `busy_timeout = 5000`, `temp_store = MEMORY`.
- Migrations: `drizzle-kit generate` produces SQL files committed under `packages/server/drizzle/`; the server applies them at startup with Drizzle's migrator before anything else touches the database.
- JSON documents carry `schemaVersion`; upgrade functions live in `packages/shared/src/schemas/migrations/` and run on read (then the upgraded document is written back lazily).

20.2 Write-through

The command bus (§14.1) writes every accepted change and its history row in one transaction before broadcasting. That is the autosave. The only exception is explored-fog memory, flushed every 5 s and at shutdown (dirty rasters only).

20.3 Snapshots

A snapshot is the full campaign as one JSON document (gzip): campaign row, scenes with walls/lights/zones/effects/tokens, actors and sheets, templates, homebrew content, combat state, fog masks (base64 deflate), handouts, log entries, and the campaign's `assets`

reference rows (name, tags, status, file ID); the files themselves stay in `data/assets`. Format header: `{ "format": "gloam-snapshot", "schemaVersion": 1, "app": "<version>", "createdAt": ... }`. Kinds and retention: `auto` every 10 minutes while the table is open (last 24 kept), `scene` on activation (last 10), `close`, `shutdown`, `manual` (kept until deleted), `pre-restore` (last 5). **Restore** stops the table room's command intake, takes a `pre-restore` snapshot, replaces the campaign's rows in one transaction, reloads the room state and tells clients to resync.

20.4 Backups

Daily at the first opportunity after 04:00 local time (and via **Backup now**): better-sqlite3's online backup API (`db.backup(path)`) to `data/backups/gloam-YYYY-MM-DD.db`; keep 14. Backups run without pausing play.

20.5 Export and import

Export writes `data/exports/<campaign>-<date>.gloam`: a zip (fflate) containing `manifest.json` (format, version, counts, SHA-256 of each file), `campaign.json` (the snapshot document, including asset reference rows) and `assets/<fileId>.<ext>` — for each referenced file, its largest processed output (the `full` variant; GLB or audio as stored), which the importer re-runs through the pipeline to regenerate the other variants. **Import** (Admin only): stream-unzip with limits (total uncompressed ≤ 4 GB, $\leq 10\,000$ entries, reject absolute paths, `..` segments and symlinks), verify hashes, **re-run every asset through the upload pipeline**, and create a new campaign in which **every** ID (campaign, scenes, tokens, actors, walls, lights, zones, effects, assets, handouts, log entries...) is freshly generated and all references are rewritten through one old→new ID map.

21. Asset pipeline

21.1 Upload flow

1. `POST /api/assets` (multipart) with CSRF token and a `purpose` field (`map` | `mini` | `token` | `portrait` | `handout` | `art` | `audio`) that selects the size cap, pixel limit and variants; authenticated and admitted (or still in the lobby: purpose `art`, images only, 20 MB quota). The detected type must match the purpose (e.g. `mini` → GLB).
2. **busboy** streams the single file part to `data/tmp/<random>`, enforcing the per-class byte cap mid-stream and the user's quota; abort and delete on overflow.
3. Compute SHA-256 while streaming, and detect the type (step 4) before anything else. If an `asset_files` row with that hash exists **and** the detected type, purpose and the uploader's permissions allow it, skip processing and just create (or return) this campaign's `assets` reference to it (file-level de-duplication; approval is per reference).
4. Detect the type with `file-type` on the first 4 KB. The detected type must be in the allow-list (§8.16); the extension and client MIME are ignored.
5. Hand the temp file path to the **asset processor**: a separate Node **child process** (`child_process.fork` with `--max-old-space-size=768`), one job at a time, `sharp.concurrency(1)`. Per-job timeouts: image 20 s, GLB 60 s, audio 5 s. On timeout, crash or excessive memory (checked every second via `process.memoryUsage().rss` reported by the child, limit 1.5 GB), kill the process with `SIGKILL` and start a fresh one; the job fails with a readable reason. Worker threads are not enough here: native image and mesh libraries allocate outside the JS heap and a native crash in a thread would take the whole server down.
6. Write outputs to `data/assets/<first two hex chars>/<sha256>-<variant>.<ext>`; insert the `asset_files` row and this campaign's `assets` reference with status `approved` (DM/Admin) or `pending` (players); notify DMs of pending uploads.
7. Delete the temp file in all cases.

21.2 The processor never trusts input

The asset processor gets a path and a job type over IPC, touches nothing else, and returns only metadata and output paths. It doesn't access the database or the network.

21.3 Images

`sharp(input, { failOn: "error", limitInputPixels: <268_435_456 for maps, 50_000_000 otherwise>, animated: false })` → `.rotate()` (EXIF) → strip metadata (default) → variants:

Use	Variants (long edge)	Encoding
Map	8192, 4096, 1024, 256	WebP quality 88 (no alpha)
Token / portrait / drawing	1024, 512, 128	WebP with alpha, quality 90
Handout	2048, 512	WebP quality 88

GIFs contribute their first frame only. Record width, height and dominant colour (for placeholders).

21.4 GLB models

1. `validator.validateBytes(bytes, { format: "glb", maxIssues: 100, writeTimestamp: false })` → reject if `issues.numErrors > 0` or if any resource is stored outside the GLB (external URI).
2. Read with `gltf-transform` `NodeIO` registered with the Khronos extensions and meshopt decoder/encoder (`await MeshoptEncoder.ready`); read from bytes (`io.readBinary`) so no external fetch can happen.
3. **Strip:** `cameras`, `KHR_lights_punctual`, `extras`, and every extension not in the allow-list (`KHR_materials_*`, `KHR_texture_transform`, `KHR_mesh_quantization`, `EXT_meshopt_compression`, `EXT_texture_webp`, `KHR_materials_emissive_strength`). Draco-compressed input (`KHR_draco_mesh_compression`) is decoded on the server with the `draco3dgltf` package (a local WASM decoder, registered as the `draco3d.decoder` dependency) and re-encoded with meshopt, so clients never need a Draco decoder.
4. **Optimise:** `prune()`, `dedup()`, `weLd()`; if triangles > 100 k (minis) or 1.5 M (maps), simplify with the meshoptimizer simplifier toward that budget; `textureCompress({ encoder: sharp, targetFormat: "webp", resize: [1024, 1024] })` for minis (`[2048, 2048]` for maps); `meshopt({ encoder: MeshoptEncoder, level: "medium" })`.
5. Write GLB, **re-validate** (0 errors required), compute stats (triangles, textures, bounding box, animations list) and the normalisation hint (§8.5).
6. The client loads GLBs with three's `GLTFLoader` + `MeshoptDecoder` (from `three/examples/jsm/libs/meshopt_decoder.module.js`), never Draco.

21.5 Audio

Validate the container by magic bytes and a minimal header parse (MP3 frame sync / OGG `OggS` pages / RIFF-WAVE / `ftyp` M4A / `fLaC`); store as-is; duration is measured by the browser on first play and saved via `PATCH /api/assets/:id` (DM).

21.6 Serving

`GET /assets/:assetId/:variant` (`assetId` is the per-campaign `ast_...` reference) → check the session (admitted member of that reference's campaign; pending references only to the uploader and DMs) → resolve the file through `file_id` → `res.sendFile` with `Content-Type` from the database, `Cache-Control: private, max-age=31536000, immutable`, `ETag = hash`, `X-Content-Type-Options: nosniff`, `Content-Security-Policy: default-src 'none'; sandbox`, `Cross-Origin-Resource-Policy: same-origin`, Range support (audio). The client fetches assets with a concurrency limit of 6 (quick-tunnel limit) and relies on the HTTP cache.

22. Security model

22.1 Threats and controls

Threat	Controls
A stranger finds the tunnel URL	Invite code (≈50 bits) + per-IP rate limits + manual approval + ban; the table is closed most of the time

Threat	Controls
A mischievous invited player	Server authority for every change; per-client views (P4); rate limits; upload approval; sheet locks; DM undo/revert
Malicious or broken files	Magic-byte allow-list, size caps, re-encoding, gITF validation, an isolated child process with memory/time limits, sandboxed serving
XSS through names, notes, handouts, spells	React escaping; Markdown rendered without raw HTML; strict CSP; no <code>dangerouslySetInnerHTML</code> except the audited Markdown renderer
CSRF	SameSite cookies, CSRF double-submit token, <code>Origin</code> allow-list for cookie-authenticated requests; bearer-token requests that carry no cookies are exempt (they can't be forged cross-site)
Local network snooping or access	Bind <code>127.0.0.1</code> by default; LAN mode opt-in with warning
Remote access to admin bootstrap, admin login, dangerous settings or API	Local-only guard (§22.3); admin login through the doorway is off by default; the port, LAN, tunnel and cloudflared-path settings can only be changed locally, so a stolen admin session can't make the server run a different program
Denial of service	Message size cap (256 KB), per-message rate limits, formula limits, path limits (≤ 256 points; server path CPU ≤ 50 ms), upload caps, worker timeouts
Supply chain	Pinned versions + lockfile; pnpm build-script allow-list; <code>pnpm audit</code> ; no runtime downloads
Host PC compromise via the app	No shell execution from user input; <code>cloudflared</code> spawned with fixed arguments via <code>spawn</code> (no shell); file paths never derived from user input; data directory permissions

22.2 Cookies and sessions

Cookie	Content	Attributes	Lifetime
<code>gloam_sid</code>	256-bit random session token (stored as SHA-256)	HttpOnly, SameSite=Lax, Path=/, Secure when the request is HTTPS	Players 30 days; Admin 12 h idle, 7 days absolute
<code>gloam_dev</code>	256-bit random device token (stored as SHA-256)	HttpOnly, SameSite=Lax, Secure when HTTPS	400 days
<code>gloam_csrf</code>	random token bound to the session	readable by JS, SameSite=Strict	session

LAN mode is plain HTTP, which browsers treat as an insecure context: `Secure` cookies aren't set, the async Clipboard API and `crypto.randomUUID` are unavailable (use `nanoid`, which relies on `getRandomValues`, and a textarea-select fallback for copy buttons). Localhost and the tunnel are secure contexts.

"HTTPS" = `req.secure`, or `x-forwarded-proto: https` on a loopback request that carries Cloudflare headers. Rotate `gloam_sid` on admission and on admin login. Passwords and PINs use argon2id (`@node-rs/argon2` defaults: m = 19 456 KiB, t = 2, p = 1); admin password ≥ 12 characters.

22.3 Local-only requests

```
function isLocalRequest(req): boolean {
  const peer = req.socket.remoteAddress; // "127.0.0.1" | "::1" | "::ffff:127.0.0.1"
  const viaCloudflare = ["cf-ray", "cf-connecting-ip", "cf-ipcountry", "cf-visitor"]
    .some((h) => req.headers[h] !== undefined);
  const host = (req.headers.host ?? "").split(":")[0];
  return isLoopback(peer) && !viaCloudflare && ["localhost", "127.0.0.1", "::1"].includes(host);
}
```

Local-only: `/setup`, `/admin/magic`, admin password login (unless *Allow admin login through the doorway*), changing the port / LAN / tunnel / cloudflared-path settings, API-token authentication (unless *Allow remote API*), `POST /api/admin/password/reset` (prints a new setup link to the console).

22.4 Headers (every response; helmet + custom)

```
Content-Security-Policy:
  default-src 'self';
```

```
script-src 'self' 'wasm-unsafe-eval';
style-src 'self' 'unsafe-inline';
img-src 'self' blob: data:;
media-src 'self' blob:;
font-src 'self';
connect-src 'self' <ws(s)://current-host>;
worker-src 'self' blob:;
object-src 'none'; base-uri 'none'; frame-ancestors 'none'; form-action 'self'
Referrer-Policy: no-referrer
X-Content-Type-Options: nosniff
Cross-Origin-Opener-Policy: same-origin
Cross-Origin-Resource-Policy: same-origin
Permissions-Policy: camera=(), microphone=(), geolocation=(), payment=(), usb=()
Strict-Transport-Security: max-age=31536000 (HTTPS requests only)
```

Compute `connect-src` per request from the `Host` header (so the tunnel hostname works). In dev only, add `ws://localhost:24678` for Vite HMR and allow the React Fast Refresh preamble with a per-request nonce (Vite's `html.cspNonce`) — never relax the production policy. A CSP violation report endpoint (`/api/csp-report`) logs violations locally; E2E tests fail on any violation.

22.5 Rate limits (in-memory token buckets)

Scope	Limit
Invite code attempts per IP	10 per 10 min → 429 for 10 min
PIN attempts per profile	5 per 15 min
Admin login per IP	5 per 15 min, then exponential backoff
Uploads per user	10 per minute, 200 MB quota (players)
REST per session	60 per 10 s
Room messages	per message type (§13.5)

22.6 Validation and output

Strict zod schemas at every boundary (`.strict()`), bounded numbers and string lengths, arrays with max sizes). Sanitise display names (NFC-normalise, strip control characters, collapse whitespace). Markdown via `react-markdown` without `rehype-raw`; links get `rel="noopener noreferrer nofollow"` and open in a new tab.

22.7 Logging and the security log

pino JSON logs to console (pretty in dev) and daily files. Never log cookies, tokens, codes, PINs, passwords, the tunnel token or request bodies of auth routes. Security events go to the `security_log` table (§8.20).

22.8 Secrets

`data/secret.key` (32 random bytes, mode 0600) encrypts the named-tunnel token at rest (AES-256-GCM) and derives HMAC keys (CSRF). Losing it only invalidates CSRF tokens and the stored tunnel token.

23. Client architecture

23.1 Routes (React Router 7, declarative)

`/` (redirects by session state) · `/join` · `/wait` · `/table` · `/admin/*` (lazy) · `/setup` · `/closed`. The table route mounts the board canvas once and keeps it mounted while panels change.

23.2 State

- **Zustand stores:** `session` (me, role, connection status), `entities` (normalised maps: tokens, walls, lights, zones, effects, combat, presence, scene), `ui` (tool, selection, hover, open panels, drag state, view-as), `settings` (device preferences persisted to `localStorage`, wrapped in try/catch), `feed` (rolls, requests, toasts), `audio`.
- **Render registry:** a mutable `Map<tokenId, { target, path, anim }>` read inside `useFrame` for smooth animation without React re-renders.
- Components subscribe with selectors (`useEntities(s ⇒ s.tokens.get(id))`) to avoid global re-renders.

23.3 Networking layer

`net/client.ts` owns the Colyseus `Client`; `net/table.ts` joins with the shared `Table` schema class, wires `Callbacks.get(room)` into the stores (§13.6), handles reconnection (`room.onDrop`, `room.onReconnect`, SDK automatic retries; after failure, a fresh `joinById`), and exposes a typed `request(name, payload)` wrapper with timeouts and error-to-toast mapping. REST calls use `fetch` with the CSRF header and React Query.

23.4 Workers

`dice/physics.worker.ts` (Rapier), `board/movement/range.worker.ts` (fast marching), `art/paperCutout.worker.ts`, `board/fog/decode.worker.ts` (RLE/deflate explored rasters). Typed `postMessage` wrappers; transferable buffers.

23.5 Code splitting and loading

Lazy: admin console, DM editors, sheet editor, spell builder, drawing pad, dice worker. Prefetch the dice worker and the sheet panel during idle after first render. The table route's initial JS ≤ 1.2 MB gzipped.

23.6 Errors

React error boundaries per panel and around the canvas ("The board hit a snag — Reload board" keeps the rest of the UI alive). Client errors are posted (rate-limited) to `POST /api/client-log` on the local server — never to a third party.

23.7 Test hooks (test builds only)

When `import.meta.env.MODE === "test"`, expose `window.__gloam = { ready(), state(), visibleTokenIds(), camera(set?), stats() }` and set `preserveDrawingBuffer: true`. Production builds must not contain these (checked by a build-output grep in CI).

24. Rendering and VFX

24.1 Scene graph (R3F)

```
<Canvas dpr={tier.dpr} shadows={tier.shadows} gl={{ antialias: false, powerPreference: "high-performance",
  preserveDrawingBuffer: isTest }} camera={{ fov: 40, near: 0.5, far: 4000 }}>
  <PerformanceMonitor .../> <CameraRig/> <Lighting/> <TableSurface/>
  <MapLayer/>                # image plane | GLB map | procedural floor
  <Walls3D/>                 # when enabled
  <ZonesLayer/> <EffectsLayer/> <LightGlows/>
  <FogComposite/>           # render targets + overlay (§15.7)
  <TokensLayer/>            # instantiated where possible; overlays (bars, icons, names)
  <MovementPreview/> <RangeField/> <MeasureLayer/> <PingsLayer/> <VfxLayer/>
  <EditorGizmos/>          # DM tools
  <Hud renderPriority={2}><DiceTray/></Hud> # after the EffectComposer (priority 1); set gl.toneMapping = AgX for this pass only, t
  <PostFX/>
</Canvas>
```

24.2 Lighting rig

- Hemisphere light (sky `#1A2230`, ground `#0A0C10`), intensity by scene ambient (bright 1.0, dim 0.45, dark 0.15).
- Key directional light (warm `#FFD9A8`, 35° elevation), shadows on High/Ultra (2048 map, PCF soft, bias tuned per tier).
- Pooled point lights for the nearest light sources (§15.7), flicker synced with the light map.
- Environment: `RoomEnvironment` (three/examples) through PMREM at startup for subtle reflections — no HDRI downloads.

24.3 Materials

- Image map: unlit (keeps the art's colours) and graded by the fog/light composite.
- Table: procedural wood (`MeshStandardMaterial` + fbm-based colour and roughness in `onBeforeCompile`).
- Standeeds: card with `alphaTest 0.5`, 0.15-ft thickness edge (cardboard colour), slight tilt-free billboard around Y.
- Coins: disc with the image on top, bevelled rim in the ring colour.
- Minis: their own PBR materials, receiving light-map shading.

24.4 Overlays

Name plates use drei `<Text>` with the local Cinzel `.woff` file; HP bars, temp segments, ghost segments and condition icons are instanced quads; icons come from a **texture atlas** built at startup by rasterising the Appendix G SVGs into a 2048² canvas (64 px cells, 2 px padding, mipmapped). Everything faces the camera and scales with zoom within limits.

24.5 VFX system

A small in-house particle engine: instanced quads whose motion is computed in the vertex shader from spawn data and time ($p = p_0 + v_0 \cdot t + \frac{1}{2} \cdot g \cdot t^2$), size and colour over normalised life from small curves passed as uniforms), plus shell meshes (noise-displaced spheres), rings, bolts (procedural midpoint-displacement polylines), and fading decals.

Preset	Cast / impact	Persistent loop
fire	expanding emissive shell + embers + scorch decal (fades 5 s)	licking flames at edges
cold	frost ring + ice shards	drifting frost motes
lightning	forked bolt from caster (flickers 3 frames) + flash	occasional arcs
thunder	shockwave ring with refraction + dust	low pulsing ring
acid	green splash + bubbling puddle decal	bubbles
poison	billowing green-grey clouds	slow churning clouds
necrotic	black-violet tendrils converging	wisps
radiant	golden pillar + motes	soft shimmer
force	translucent blue geometric shell	faint hex lattice
psychic	pink-magenta ripples	ripples
healing	green-gold sparkles rising	—
arcane	runic circle + motes	slow rotating circle

Particle counts scale by tier (§24.6). Budget: two simultaneous cast effects cost ≤ 2 ms GPU on High.

Persistent areas: Fog Cloud (layered soft billboards), Darkness (inky sphere with swirling edge noise; opaque to players), Web (strand pattern decal), Spirit Guardians (orbiting spectral motes around the caster), Moonbeam (pale cylinder of light), Spike Growth (thorny decal), Wall of Fire (flame curtain along the wall line), Silence (faint muted dome).

24.6 Post-processing and tiers

Tier	DPR cap	Shadows	Bloom	AO	SMAA	Point lights	Fog px/ft	Particles
Ultra	2.0	2048 soft	✓	N8AO	✓	8	6	100%
High	1.75	2048	✓	—	✓	4	4	70%
Medium	1.5	1024	✓ (half-res)	—	—	2	3	40%
Low	1.0	blob shadows only	—	—	—	0	2	20%

Chain: tone mapping **AgX** (postprocessing `ToneMappingEffect`), bloom with mipmap blur and a high luminance threshold (only lights, VFX, selection glow and crit sparkles bloom), SMAA, subtle vignette. Guard custom shaders against NaNs (clamp before bloom).

24.7 Shader warm-up

During the intro, render one hidden instance of every material and VFX preset, and call `gl.compileAsync(scene, camera)` so no shader compiles mid-game.

24.8 Resource hygiene

Dispose geometries, materials, textures and render targets on scene switches; cache GLBs by asset ID; clone skinned models with `SkeletonUtils.clone`; log `renderer.info` in the perf overlay (`?perf=1`).

24.9 Forbidden helpers (P1)

No drei `Environment` presets, no `useGLTF` Draco path, no default-font `Text`, no remote textures, no `Stats` from CDNs. The CSP will block them; tests will catch it.

25. Audio engine

25.1 Graph

`AudioContext` → master gain → gentle `DynamicsCompressor` → destination. Channel gains: dice, effects, UI, music, ambience. Each board sound gets a `StereoPannerNode` (pan = screen x mapped to $-0.8 \dots 0.8$) and a distance gain $1 / (1 + d / 60 \text{ ft})$ from the camera target.

25.2 Effects

Generate ZzFX sample buffers with `zzfxG(... params)` once at startup (cache per recipe) and play them as `AudioBufferSourceNode`s in Gloam's own context, so they route through the channel gains. Add $\pm 4\%$ random pitch per play. More complex sounds (dice clacks, door creak, fire whoosh) are small Web Audio graphs (§31).

25.3 Music synchronisation

Server state: `{ kind: "track" | "preset", trackId?, preset?, seed?, startedAtServerMs, paused, pausedAtMs, volume, playlistId?, loop, shuffle }`. Clients: `HTMLAudioElement` → `MediaElementAudioSourceNode` → music gain; position = $(\text{serverNow} - \text{startedAt}) / 1000$ using the clock offset (§13.8); every 2 s measure drift against the server timeline: $> 50 \text{ ms}$ → nudge `playbackRate` to 1.03/0.97 until back under 20 ms; $> 500 \text{ ms}$ → seek. Crossfade 2 s between tracks with two elements.

25.4 Ambience synthesis

Looping 10-s noise buffers (white, pink, brown) generated at startup; per layer: rain = pink noise → high-pass 400 Hz + random short droplet clicks; wind = brown noise → band-pass swept by a slow LFO; fire = brown noise low-pass 900 Hz + random crackle impulses;

water = pink noise band-pass 600–2 000 Hz with slow amplitude modulation; cave drips = sparse sine pings (1.2–2.4 kHz) through a generated-impulse `ConvolverNode` reverb; night insects = high sine chirps with AM bursts. Each layer has a gain the DM controls.

25.5 Generative music presets

A small scheduler (look-ahead 100 ms, `AudioContext.currentTime` clock, seeded PRNG so every client plays the same notes) drives oscillator/noise voices through a shared bus with a generated-impulse reverb and a gentle limiter. Each preset is a set of layers whose note choices follow a scale and a slowly changing chord progression, with phrase lengths that avoid obvious repetition:

Preset	Scale / tempo	Layers
Dungeon drone	D minor (Aeolian), free time	two detuned saw pads through a slowly swept low-pass; sub pulse every 6–10 s; occasional distant bell (sine + inharmonic partials)
Tavern	G major pentatonic, 96 BPM	plucked voice (Karplus–Strong from a short noise burst) playing arpeggios; soft hand-drum (filtered noise + sine thump) on a 6/8 pattern; low bass on chord roots
Battle	E Phrygian, 110 BPM	ostinato on a filtered square; low tom pattern; string-like swell (layered saws with slow attack) every 8 bars
Wonder	A Lydian, 72 BPM	airy pad (sine stack, slow attack); glassy bell motifs; high shimmer noise

The DM's music state carries `{ preset, seed, startedAtServerMs }`; each client renders the same timeline from the seed (clock offset §13.8), so the table hears the same music. Presets and uploaded tracks crossfade over 2 s.

26. Local API and MCP server

26.1 REST v1 (`/api/v1`)

Method & path	Scope	Purpose
<code>GET /schemas/{name}.json</code>	none (admitted)	JSON Schemas (spell, character, monster, item, handout, campaign-log-entry)
<code>GET /campaigns</code>	<code>campaign:read</code>	List campaigns (id, name, active scene)
<code>GET /campaigns/:id/summary</code>	<code>campaign:read</code>	Party, scenes, session number
<code>GET /content/spells?query=&level=&pack=</code>	<code>content:read</code>	Search spells
<code>POST /content/spells:import?campaignId=&dryRun=true&onConflict=skip\ overwrite\ rename</code>	<code>content:write</code>	Import homebrew spells into a campaign (array or <code>{ spells: [] }</code>)
<code>POST /content/monsters:import?campaignId=&dryRun=</code>	<code>content:write</code>	Import homebrew monsters into a campaign
<code>POST /actors:import?campaignId=&dryRun=</code>	<code>sheets:write</code>	Import characters
<code>GET /actors/:id</code>	<code>sheets:read</code>	Read a sheet
<code>GET /campaigns/:id/log?sinceSession=</code>	<code>log:read</code>	Read the campaign log
<code>POST /campaigns/:id/log</code>	<code>log:write</code>	Append a log entry (e.g. an AI-written recap)

Responses: JSON `{ data, errors?, report? }`. Import reports list `created`, `updated`, `skipped`, `invalid[{ index, path, message }]`.

26.2 MCP tools (`packages/mcp`, `@modelcontextprotocol/server v2`, `stdio`)

Tool	Input (zod)	Does
<code>get_schema</code>	<code>{ kind }</code>	Returns the JSON Schema so the model can produce valid JSON

Tool	Input (zod)	Does
list_campaigns	{}	Campaign list
search_spells	{ query?, level?, pack? }	Spell search
import_spells	{ campaignId, spells: unknown[], dryRun?: boolean, onConflict? }	Import homebrew spells
import_monsters	{ campaignId, monsters: unknown[], dryRun?, onConflict? }	Import homebrew monsters
import_character	{ campaignId, sheet: unknown, dryRun? }	Import a character sheet
get_campaign_log	{ campaignId, sinceSession? }	Read the log (for recaps)
add_log_entry	{ campaignId, text, kind? }	Write a recap or note

Configuration via env: `GLOAM_URL` (default `http://127.0.0.1:4747`), `GLOAM_TOKEN`. Logs go to stderr only (stdout is the protocol). Setup lines for the Connect Claude page:

```
claude mcp add gloam --env GLOAM_URL=http://127.0.0.1:4747 --env GLOAM_TOKEN=<token> \
-- node <repo>/packages/mcp/src/index.ts
```

```
{ "mcpServers": { "gloam": { "command": "node",
  "args": ["<repo>/packages/mcp/src/index.ts"],
  "env": { "GLOAM_URL": "http://127.0.0.1:4747", "GLOAM_TOKEN": "<token>" } } } }
```

Part D — Design system

27. Art direction and tokens

27.1 Direction: "the candlelit war table"

A heavy oak table at night under a brass lamp; ink-stained maps; wax seals; minis casting long shadows. The **UI chrome** is dark ink with brass details and bone-white type. **Documents** (sheets, spell cards, handouts, the campaign log) are parchment surfaces that feel like paper laid on the table. The **board** is lit warmly from above, with deep, soft darkness at the edges. Motion is weighty and tactile: things drop, settle and glide rather than blink. Sound is wooden, papery and metallic, never "app-like".

Keep one committed aesthetic everywhere. Dominant ink and brass with sharp, meaningful accents (ember for danger, verdigris for success, arcane blue for magic) beats a timid, evenly spread palette.

27.2 Colour tokens

Define every colour as a CSS custom property in `packages/web/src/styles/tokens.css` and expose them to Tailwind 4 through `@theme`. No other file may contain raw hex values (enforced by `tools/check-tokens.mjs`, with an allow-list for shader code in `packages/web/src/board/**` which imports the same values from `packages/shared/src/constants.ts`).

```
:root {
  /* ink - chrome surfaces */
  --ink-950: #07090C; --ink-900: #0D1117; --ink-850: #121821; --ink-800: #18202B;
  --ink-700: #243041; --ink-600: #334155; --ink-500: #4A5A70;
  /* type */
  --bone-100: #EDE6D6; --fog-300: #A9B4C2; --fog-400: #8492A6;
  /* accents */
  --brass-300: #E6C98B; --brass-400: #D4AF6A; --brass-600: #9C7A3C; --brass-800: #5E4822;
  --ember-400: #F08A4B; --blood-500: #C8413B; --verdigris-400: #5FBF9A; --arcane-400: #7FA7E8;
  --hex-400: #B07FE0; --ice-300: #9FDCF0;
  /* documents */
  --parchment-100: #EFE5CC; --parchment-200: #E3D5B3; --parchment-400: #C4AE7E;
  --parchment-ink: #2B2118; --parchment-ink-muted: #6B5A45; --wax-500: #9E2B25;

  /* semantic */
  --bg: var(--ink-950); --surface: var(--ink-900); --panel: var(--ink-850); --raised: var(--ink-800);
  --border: var(--ink-700); --text: var(--bone-100); --text-muted: var(--fog-300);
  --accent: var(--brass-400); --accent-strong: var(--brass-300); --focus: var(--brass-300);
  --danger: var(--blood-500); --warning: var(--ember-400); --success: var(--verdigris-400);
  --magic: var(--arcane-400); --mental: var(--hex-400);
  --path-ok: var(--verdigris-400); --path-over: var(--blood-500);
  --hp-high: var(--verdigris-400); --hp-mid: var(--brass-400); --hp-low: var(--ember-400);
  --hp-temp: var(--ice-300); --hp-ghost: #F4E9D8;
}
```

Colour-blind palette (swap when enabled):

--path-ok: #56B4E9; --path-over: #E69F00; --hp-high: #009E73; --hp-mid: #F0E442; --hp-low: #D55E00; --disp-hostile: #D55E00; --disp-friendly: #009E73; --disp-neutral: #F0E442, plus patterns (dashed out-of-range path, striped low-HP fill).

Dispositions: party = owner's player colour; friendly #5FBF9A; neutral #D4AF6A; hostile #E0584B.

Player colours (12; chosen at join; unique per campaign when possible): amber #E6B450, sky #5FB3E6, coral #E0605C, mint #6CC98A, orchid #C77DDB, citrine #F2E266, teal #4FD1C5, rose #F08FB0, periwinkle #9AA7FF, copper #D08B4F, lime #B5D264, silver #D9DEE6.

Damage types: acid #B5D33D, bludgeoning #B8B2A7, cold #7FD3F5, fire #FF7A2F, force #8FA8FF, lightning #F5E663, necrotic #8C6BB1, piercing #D9D3C7, poison #5FAE6B, psychic #F07BC8, radiant #FFD66B, slashing #E4E6EA, thunder #6FA3C9; healing #5FBF9A.

Icon badge categories: senses (Blinded, Deafened, Invisible) `#4E7BC4` ; mind (Charmed, Frightened) `#8E5CC8` ; body (Grappled, Restrained, Paralyzed, Petrified, Prone) `#A67C3D` ; incapacity (Incapacitated, Stunned, Unconscious) `#C8643B` ; affliction (Poisoned, Exhaustion, Burning) `#5E9A4E` ; vital (Bloodied, Death saves, Dead) `#B43A36` ; boon (Blessed, Hasted, Heroic Inspiration, Dodging, Stable) `#3F9C78` ; tactical (Concentrating, Hidden, Readied, Disengaged, Dashing, Flying, Surprised, Banned, Slowed) `#56657A` .

27.3 Typography

Role	Family (package)	Use
Display	Fraunces Variable (<code>@fontsource-variable/fraunces</code>), <code>opsz auto</code> , <code>S0FT 50</code> , weight 600–800	Headings, scene titles, HP numbers, dice totals, big moments
UI	Alegreya Sans (<code>@fontsource/alegreya-sans</code>) 400/500/700/800	Body, labels, buttons, tables
Engraved caps	Cinzel (<code>@fontsource/cinzel</code>) 600/700, letter-spacing 0.08em	Token name plates, turn-tracker names, small eyebrow labels, the "DM" seal
Mono	JetBrains Mono Variable (<code>@fontsource-variable/jetbrains-mono</code>)	Dice formulas and invite codes only

Scale (px): 12 · 13 · 14 · 16 · 18 · 22 · 28 · 36 · 48 · 64. Line heights 1.25 (display) / 1.45 (body). Stats use

`font-variant-numeric: tabular-nums lining-nums` . Minimum HUD text 12 px (13 px on touch).

27.4 Space, shape, depth, texture

- Spacing on a 4-px grid: 4 · 8 · 12 · 16 · 20 · 24 · 32 · 40 · 48 · 64.
- Radii: 4 (chips, name plates), 6 (buttons, inputs), 10 (panels, cards). Circles only for portraits, pips and the dice button. **No pill buttons.**
- Borders: 1 px `--border` ; selected/focused elements get a 1-px brass hairline plus a 2-px outer glow `rgba(212,175,106,.35)` .
- Panel shadow: `0 12px 32px rgba(0,0,0,.45)` , `inset 0 1px 0 rgba(255,255,255,.04)` .
- Texture: a 128 × 128 noise tile generated at startup (canvas) overlaid at 4% on panels; parchment documents use a baked `feTurbulence` fibre texture plus a faint vignette and a deckled top edge on handouts.
- Ornaments (SVG, brass): engraved divider (hairline with a small diamond), corner filigree for modals and the parchment cards, a wax-seal badge for the DM role, a brass pin for pinned items.

27.5 Motion

Token	Value
Durations	instant 80 ms · fast 140 · base 220 · panel 320 · scene 600 · cinematic 1200
Easing	out <code>cubic-bezier(.2,.8,.2,1)</code> ; in-out <code>cubic-bezier(.6,0,.2,1)</code>
Springs (motion)	panel { <code>stiffness: 380</code> , <code>damping: 34</code> }; pop { <code>stiffness: 520</code> , <code>damping: 22</code> }

Signature moments: the **first-load sequence** (black → a candle flame ignites in the centre (400 ms) → the board fades up from darkness (900 ms) → HUD elements stagger in 60 ms apart; click to skip); **dice** tumble and settle; **HP ghost drain**; **turn start** (the active portrait swells, a brass ring sweeps around it, chime); **scene travel** (fade through black with the scene name); **handout reveal** (parchment unfurls). Reduced motion replaces all of these with short fades.

27.6 Banned patterns

The visual critic flags any of these:

- Purple or blue gradient backgrounds; neon glows on UI chrome; frosted-glass (glassmorphism) as the default panel style.
- Cream or off-white **app backgrounds** (parchment is only for document surfaces).
- Pill-shaped buttons as the default button; fully rounded inputs.
- Generic SaaS cards: white cards with big soft shadows, dashboard grids of identical tiles.
- Inter, Roboto, Arial, system-ui or Space Grotesk as visible fonts.

- Emoji as UI icons (emoji appear only as emotes).
- "01 / 02 / 03" numbered section labels; monospace text used as decoration; italic accent words in headings.
- Stock icon-library glyphs for game concepts (conditions, actions, dice, spell schools) — those use the custom set.
- Lorem ipsum, placeholder data, dead buttons.
- Toasts piled in the middle of the board; modal dialogs for things that can be done inline.

28. Components

All in `packages/web/src/ui`, built on the tokens, accessible (focus rings, ARIA), and themed for the dark chrome and the parchment document variant.

Component	Notes
Button	Variants: primary (brass fill <code>--accent</code> , ink text), secondary (ink raised, brass hairline), ghost , danger (blood outline → fill on hover); sizes S/M/L; loading state with a small spinning d20 glyph
IconButton	36 px (44 px on touch), tooltip with shortcut hint
Toggle, Checkbox, Radio, Slider	Brass thumb, ink track, value bubble for sliders
NumberStepper	For HP and values: <code>-/+</code> buttons, type-in, hold to repeat, <code>Shift = ×5</code>
Tabs	Brass underline indicator animating between tabs
Dock / Panel	Right-side dock with icon rail; resizable (320–520 px); panels remember size
BottomSheet	Phones: snap points 30/60/95%, drag handle, momentum
Dialog	Ink or parchment variant; corner filigree; focus trap; <code>Esc</code> closes
Toast	Top-right stack (max 4), typed (info/success/warning/danger/knock), action button, auto-dismiss 5 s
Tooltip / HoverCard	400 ms delay; hover cards for tokens, conditions, spells
RadialMenu	6–8 slices around the pointer, icons + labels, keyboard numbers, long-press on touch
CommandPalette	<code>Ctrl/Cmd+K</code> : search commands, DM sections, spells, tokens
Portrait	Circular image with ring (player/disposition colour), active-turn state, hand-raise badge, HP arc option
HPBar	Current / temp / ghost segments, 50% tick, display modes, colour-blind patterns
ConditionBadge	Icon on category-coloured rounded square (4 px radius), exhaustion level digit, duration pip
Pips	Slots, death saves (hearts/skulls), action economy shapes (● ▲ ◆ ■)
RollCard	Roller, label, formula (mono), die chips, total (display face), crit/fumble treatments, manual hand icon, expand
SpellCard / ItemCard	Parchment card with header band coloured by school or damage type, structured strip, attribution line
ResolutionCard	Stepper layout (Targets → Rolls → Outcome → Apply) with per-target rows
DiceTray	Dice buttons, formula field (mono, syntax colours), adv/dis toggles, visibility select
SegmentedCodeInput	Invite code / PIN entry, paste-aware
ColorSwatchPicker	Player colours with names (not colour-only)
EmptyState	Small line illustration (procedural SVG: a candle, a map, a die), one sentence, one action
Skeleton	Shimmer that looks like candle light moving across the placeholder
KeyHint	Small engraved keycap for shortcuts

29. Screens and wireframes

The wireframes fix **layout and hierarchy**; Part D's tokens fix the look. Dimensions are for 1440 × 900 unless noted.

29.1 Join — code step

(blurred, slowly panning render of a candle-lit board in the background)

+ GLOAM A table awaits.	Fraunces 36 Alegreya 16
Invite code [7][K][2][Q][H] — [9][X][M][4][D]	segmented input
[Knock on the door] Got the code from your DM?	primary button muted 13

Identity step replaces the card body with two tabs (**New here / I've played before**), name field, colour swatches (with names), optional PIN (segmented 4–8), and **Continue**.

29.2 Waiting room

▒ a heavy wooden door, lit by ▒
a single animated candle

Waiting for the DM to let you in... (Fraunces 28)
Dave · amber · returning ✓ (Cinzel 13)

[🗑 Draw your character] [♦ Choose your dice] [🎵 Test sound]

Leave the lobby

29.3 Table — desktop HUD

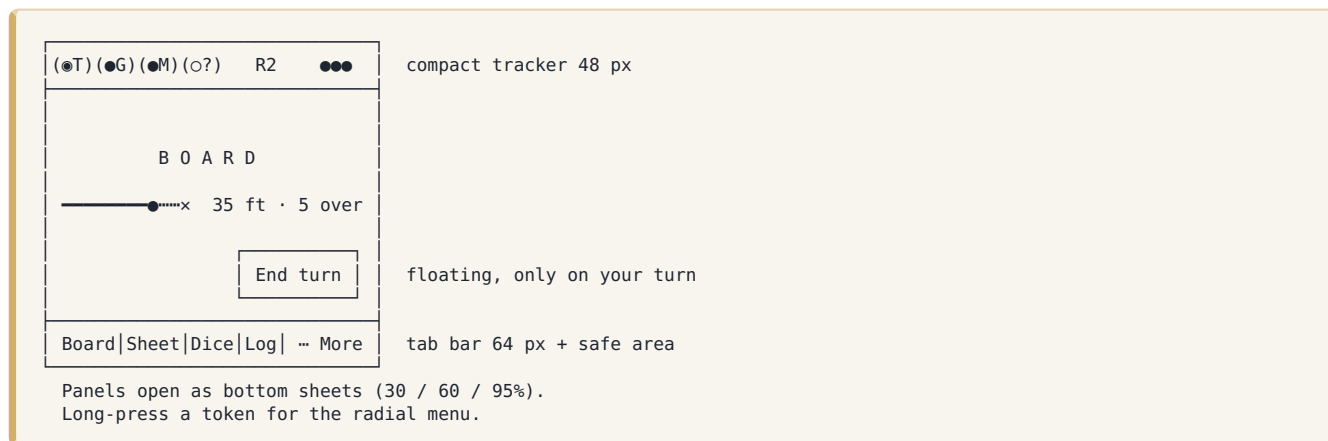
+ Lantern Crypt ▾		(●T 18)(●G 15)(●M 14)(○ ?)(●P 9)	ROUND 2	♪ ⦿ ●●●●
▶	B O A R D (full-bleed, behind the HUD) path —————●—————x "35 ft · 5 over" DM tools (walls, fog, lights, zones) appear below the divider for DMs only			Party Sheet ▾
◆				Spells
↔				Log
⊙				DM +
+				dock
—				320–520 px
■				
▲				
●				
Roll feed (last 3)		(T) Thorin ▒ 24/30 ● ▲ ◆ ■ move ▒▒▒▒		toasts (top-right)
Mira 17 · Thorin 26		[Dash] [Reset move] [1][2][3][4] (d20) [End turn]		

left toolbar 52 px: Select, Pan, Measure, Ping, Target · DM: Walls, Fog, Lights, Zones, Effects
top bar 56 px · bottom action bar 96 px, centred; shows the selected or active creature you control

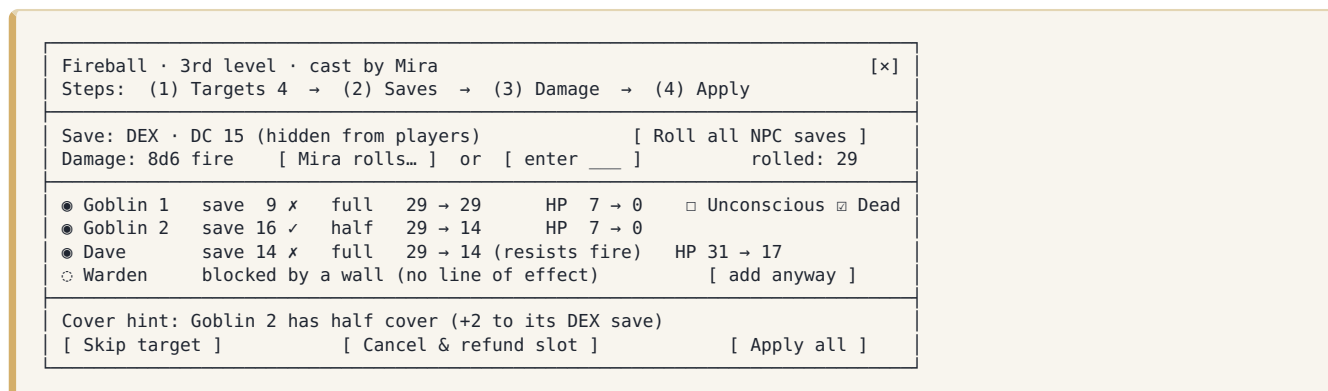


Figure 2 — Table HUD, desktop (1440 × 900)

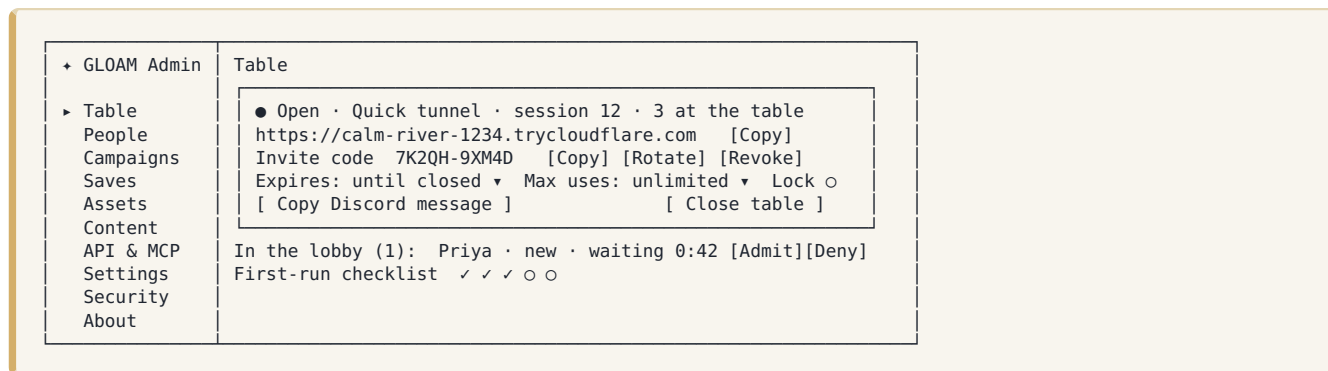
29.4 Table — phone (390 × 844)



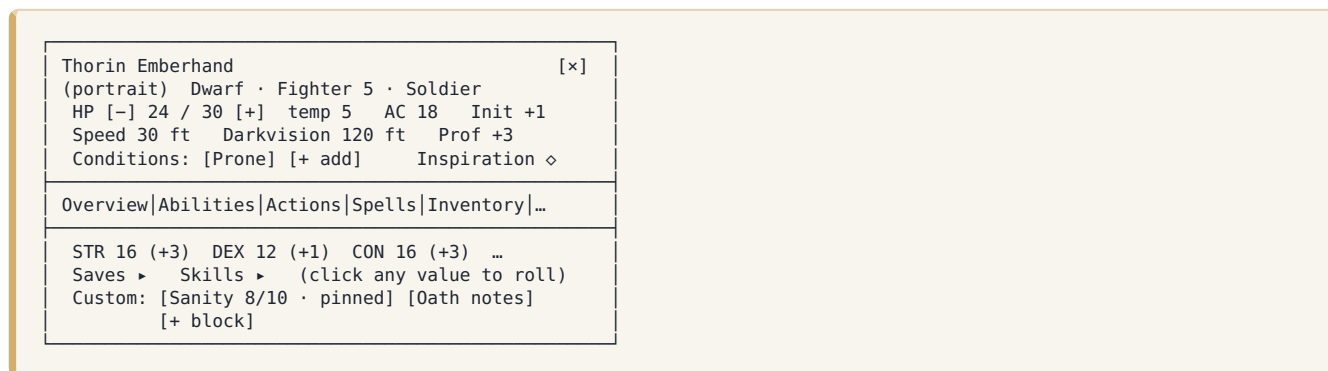
29.5 Resolution card (DM view)



29.6 Admin console — Table page



29.7 Character sheet (desktop side panel, parchment)



30. Icons

30.1 Grammar

- 24 × 24 viewBox, drawn for 16–24 px display; strokes 1.75 px (scaled), round caps and joins; filled shapes where a silhouette reads better.
- Glyphs use `currentColor` and sit on a **category-coloured badge** (rounded square, 4 px radius at 24 px) when shown on tokens; in the DOM they can appear bare.
- Every icon must be distinguishable **by shape alone** (colour-blind safe) at 16 px.
- React components in `packages/web/src/icons/`; the same SVG strings feed the WebGL atlas (§24.4).
- The reference drawings in **Appendix G** are the starting point: implement them faithfully, refine proportions at 16 px, keep the concepts.

30.2 Set

ID	Name	Concept
blinded	Blinded	An eye crossed by a heavy diagonal bar
charmed	Charmed	A heart with a small spiral inside
deafened	Deafened	An ear with a diagonal bar
exhaustion	Exhaustion	An hourglass; the level digit (1–6) in a corner notch
frightened	Frightened	A ghost silhouette with hollow eyes
grappled	Grappled	A gripping hand (fingers curled over a bar)
incapacitated	Incapacitated	A circle-slash over a small lightning spark (no actions)

ID	Name	Concept
invisible	Invisible	A dashed outline of a figure
paralyzed	Paralyzed	A rigid figure with vibration ticks
petrified	Petrified	A cracked stone block
poisoned	Poisoned	A round-bottomed flask with bubbles (distinct from the Bloodied drop)
prone	Prone	A figure lying horizontally on a ground line
restrained	Restrained	Two interlocked chain links
stunned	Stunned	A spiral with two small stars
unconscious	Unconscious	"Z z" letters rising
bloodied	Bloodied	A blood drop with a highlight
concentrating	Concentrating	Three concentric rings (focus)
death saves	Death saves	A heart split by a crack
stable	Stable	A heart with a plus
dead	Dead	A skull
hidden	Hidden	A hooded head in profile
surprised	Surprised	An exclamation mark in a burst
dodging	Dodging	A shield with motion lines
disengaged	Disengaged	An arrow leaving a circle
dashing	Dashing	Speed lines behind double chevrons
inspiration	Heroic Inspiration	A four-point star over a small d20 outline
blessed	Blessed	A sun disc with rays
baned	Baned	An inverted triangle (a d4) with a minus sign
hasted	Hasted	A lightning bolt with speed lines
slowed	Slowed	A snail shell spiral
burning	Burning	A flame
flying	Flying	A single wing
readied	Readied	A crosshair
custom	Custom marker	A blank banner shape for a letter or glyph

31. Sound design

All sounds are synthesized at runtime (§25). "ZzFX" means a `zzfxG` parameter recipe tuned in the `/dev/sounds` audition page (Phase 11); "WA" means a small Web Audio graph. Values are starting points; tune by ear with the visual critic's screenshots of the audition page as evidence of coverage.

Event	Channel	Character	Recipe (starting point)
Die hits tray (per contact)	Dice	Woody/resin clack, bright transient	WA: 12–25 ms noise burst → band-pass 1.8–3.2 kHz (random) → fast exp decay; gain ∝ impulse; metal skin adds a 2.5 kHz sine ring (60 ms)
Die rolls/slides	Dice	Soft rumble while angular speed high	WA: brown-noise loop, low-pass 700 Hz, gain ∝ angular speed
Dice settle	Dice	Small tick, then silence	ZzFX short blip (≈ 1.4 kHz, 30 ms)
Natural 20	Dice	Rising shimmer + chime	ZzFX arpeggio up (C–E–G–C), sparkle noise tail

Event	Channel	Character	Recipe (starting point)
Natural 1	Dice	Deflating "womp"	ZzFX downward pitch slide, low-pass
Your turn	UI	Warm bell	WA: two sines (660 Hz + 990 Hz) with 1.2 s decay + soft reverb
Turn passes (others)	UI	Quiet wooden tick	ZzFX tick
Knock (lobby)	UI	Two knocks on wood	WA: two low thumps (120 Hz sine + noise click), 180 ms apart
Admitted	UI	Door creak + latch	WA: filtered noise with pitch-swept band-pass (creak) + click
Token pick up / put down	Effects	Felt tap	ZzFX soft low click
Footsteps (every 5 ft)	Effects	Muffled thump	ZzFX low thump, random pitch
Door open / close	Effects	Creak / thud	WA creak (as above) / low thump
Locked door rattle	Effects	Metallic rattle	WA: 3 quick metallic clicks (band-pass 3 kHz)
Melee hit	Effects	Punchy impact	ZzFX noise + low sine thump
Damage taken	Effects	Short crunch	ZzFX noise burst with pitch drop
Heal	Effects	Soft rising chime	ZzFX sine up-sweep with shimmer
Down (0 HP)	Effects	Heavy low thud + reverb	WA low sine drop 90 → 40 Hz
Death save success / fail	UI	Heartbeat / hollow knock	WA double thump / single hollow knock
Condition applied	UI	Glassy tick coloured by category	ZzFX blip, pitch per category
Spell cast (generic)	Effects	Whoosh	WA noise sweep band-pass 300 → 3 kHz
Fire	Effects	Roaring whoosh + crackle	WA brown noise swell + crackle impulses
Cold	Effects	Crystalline tinkle	ZzFX high blips cluster
Lightning	Effects	Crack + buzz	WA noise crack + 60 Hz sawtooth buzz fading
Thunder	Effects	Boom	WA low noise + 50 Hz sine, long decay
Acid / poison	Effects	Hiss / bubbling	WA filtered noise hiss / random low blips
Necrotic	Effects	Dark swell	WA low detuned saws swell and fade
Radiant	Effects	Choir-like shimmer	WA stacked sines (fifths) with slow attack
Force	Effects	Hum + thump	WA 110 Hz sine hum + thump
Psychic	Effects	Warble	WA sine with fast vibrato
Emote pop	UI	Bubbly pop	ZzFX pop
Ping	UI	Sonar ping	WA sine 880 Hz with delay echo
Hand raised	UI	Soft chime (DM only)	ZzFX chime
Handout reveal	UI	Paper unfurl	WA filtered noise swish with rustle
Initiative start	Effects	War drum hit	WA low membrane (sine 70 Hz + noise), two hits
Scene travel	UI	Low whoosh	WA noise sweep down
Error / not allowed	UI	Muted wooden clunk	ZzFX low short click

Part E — Rules and content

32. Research phase (Phase 0 tasks)

Xini asked for the build to *research the D&D rules itself* so that nothing is missed. The facts in §34 were verified against the official SRD 5.2.1 PDF on 26 September 2026, but you must re-verify them and extract the full data sets yourself. Dispatch these research subagents **in parallel** at the start of Phase 0 (definitions in Appendix C). Each writes its output file and returns a short summary. They must cite sources (URL and page) and say plainly when something could not be verified.

Task	Agent	Output	Scope
R1 Rules verification	rules-researcher	docs/research/rules-5.2.1.md , packages/content/overlays/ conditions.json	Verify every table in §34 against the SRD 5.2.1 PDF (hash-checked); list any discrepancy with this spec; write the condition tooltip summaries (≤ 160 characters each, paraphrased, never long verbatim passages) and the glossary summaries for status markers
R2 Spell data	spell-data-engineer	packages/content/scripts/* , packages/content/packs/srd-5.2.1/ spells.json , docs/research/spells-report.md	Build and run the content pipeline in §33 end to end; assert the counts; report every spell lacking structured area/save/damage data and how it was resolved
R3 Stack verification	stack-verifier	docs/research/stack.md	Re-check each version and API claim in §9; flag breaking changes; produce minimal code snippets for the spikes below
R4 Sound research	general-purpose (leaf)	docs/research/sound.md	For each event in §31: what makes it satisfying in tabletop and game UIs, and a concrete ZzFX parameter array or Web Audio graph to start from; plus design notes for the four generative music presets (§25.5) and a short list of reputable CCO music packs a DM could import (none are bundled)
R5 UX patterns	general-purpose (leaf)	docs/research/ux-patterns.md	Movement preview, turn portraits, opportunity-attack warnings (Divinity: Original Sin 2, Baldur's Gate 3); walls/doors/lights/fog tools (Foundry VTT, Owlbear Rodeo); 3D table feel (TaleSpire). Concrete, buildable takeaways; no copying of assets or trade dress

Then, sequentially yourself, run the **integration spikes** (throwaway code under `spikes/`, deleted at the end of Phase 0 once their findings are in `docs/research/stack.md`):

- **S1** Colyseus 0.18 via `defineServer` + Express 5 + Vite middleware on one origin; `onAuth` reading the `gloam_sid` cookie from a real browser's matchmaking request; `StateView` add/remove of a map item and a tagged field observed in the browser.
- **S2** R3F canvas at the pinned versions with one render target composited over a textured plane (the fog pattern), plus drei `<Text>` with a local `.woff` font and zero network requests outside the origin.
- **S3** Rapier deterministic-compat in a Web Worker inside a Vite **production** build; a d20 simulated from a seed on two separate page loads produces identical frames; the symmetry remap shows a chosen number.
- **S4** Asset processor: sharp + file-type + gltf-validator + gltf-transform + meshoptimizer (+ `draco3dgltf` for Draco input) running inside a forked child process under pnpm 12 with the `allowBuilds` config from §9.4.
- **S5** better-sqlite3 + Drizzle migrations running under Node 24 type stripping; `db.backup()` works while writes happen.
- **S6** Tunnel manager against a fake `cloudflared` script that prints the quick-tunnel banner to stderr and serves `/quicktunnel` and `/ready` on the metrics port (and, if the real binary is installed on this machine, one real run).

33. Content pipeline (SRD 5.2.1)

33.1 Pinned sources

Source	Pin	Use
Official SRD 5.2.1 PDF	<code>https://media.dndbeyond.com/compendium-images/srd/5.2/SRD_CC_v5.2.1.pdf</code> — SHA-256 <code>8974902d109d6e63672d7c490bde9ccf052410503d9cfa768237154fbc5e3d87</code> , 6 031 375 bytes, 364 pages	Canonical list and text of all spells; conditions; rules text for R1
Foundry VTT dnd5e system data	Tag <code>release-6.0.5</code> , commit <code>3ee48de02f8f6f7b2638c9f6cf3e9540c9c181cc</code> ; files <code>packs/_source/spells24/<dir>/<slug>.yml</code> where <code><dir> ∈ cantrips, 1st-level ... 9th-level</code> (each directory also has a <code>_folder.yml</code>)	Structured mechanics: area template type/size/width, save ability, damage parts, types and scaling, attack type, concentration/ritual flags, materials, range and duration units. MIT code, CC-BY-4.0 content
Open5e API v2 data	<code>https://raw.githubusercontent.com/open5e/open5e-api/staging/data/v2/wizards-of-the-coast/srd-2024/Spell.json</code> (document key <code>srd-2024</code>)	Cross-check of header fields, saves and per-slot damage
5e-bits database	<code>https://raw.githubusercontent.com/5e-bits/5e-srd-api/main/packages/5e-database/src/2024/en/5e-SRD-Spells.json</code>	Cross-check of header fields (reflects 5.2, not 5.2.1; its structured save/area fields are empty)

Downloads happen **only** when running `pnpm content:build`; outputs are committed, so runtime never touches the network. Cache raw downloads under `packages/content/.cache/` (gitignored) and verify hashes/commits before use.

33.2 Pipeline

- Fetch** the PDF (verify SHA-256), the Foundry files at the pinned commit (sparse checkout or per-file raw URLs), Open5e and 5e-bits JSON.
- Parse the PDF spell chapter** (use `pdfjs-dist` text extraction with positional data). Spell entries follow a regular header: name; "Level N School (Class, Class...)" or "School Cantrip (...)" ; Casting Time (may say "or Ritual") ; Range; Components (V, S, M with material text, cost "worth N+ GP" and whether consumed); Duration (including "Concentration, up to ...") ; effect text; optional "Using a Higher-Level Spell Slot." (109 spells) or "Cantrip Upgrade." (15 cantrips). Produce the **canonical list** with text in Markdown (keep tables and bullet lists).
- Import Foundry mechanics** by slug, dropping anything not in the canonical list (notably `2nd-level/arcane-vigor.yml`, which is not in the SRD). Resolve formula sizes (e.g. Fog Cloud radius `20 * @item.level` → base 20 with +20 per slot level above 1st). Map Foundry template types to §17.1 shapes.
- Fill gaps from prose** with patterns such as `(\d+)-foot(?:-radius)? (Sphere|Cube|Cone|Line|Cylinder|Emanation)`, `(\d+) feet long and (\d+) feet wide`, `(\d+)-foot-radius`, `(\d+)-foot-high Cylinder`, and `Bright Light in a (\d+)-foot radius and Dim Light for an additional (\d+) feet`. Record which fields came from prose.
- Apply overlays** from `packages/content/overlays/` (hand-authored, reviewed by R1): light, obscurement and magical darkness (§34.4); cylinder heights and emanations (§33.4); persistent-effect behaviours (attachment, movement, drift, triggers) for the 16 effects in F13; VFX preset overrides.
- Validate** every spell with the zod schema (Appendix F.2); **assert** 339 spells and per-level counts `27 / 57 / 57 / 42 / 34 / 38 / 31 / 20 / 17 / 16`; cross-check header fields against Open5e and 5e-bits and saves/per-slot dice against Open5e; write disagreements to the report.
- Write** `packs/srd-5.2.1/spells.json`, `conditions.json`, `light-sources.json`, `MANIFEST.json` (source pins and hashes, counts, generation time, pipeline version) and `ATTRIBUTION.md` (Appendix I), plus `docs/research/spells-report.md`.

33.3 Normalised spell record

See Appendix F.2 for the full schema. Key fields: `id` (slug), `name`, `level` (0–9), `school`, `classes[]`, `castingTime` { `amount`, `unit`: `action|bonus|reaction|minute|hour`, `reactionTrigger?` }, `ritual`, `range` { `kind`: `self|touch|ranged|sight|unlimited|special`, `ft?` }, `components` { `v`, `s`, `m`, `material?`, `costGp?`, `consumed?` }, `duration` { `kind`: `instantaneous|timed|until-dispelled|special`, `amount?`, `unit?`, `concentration` }, `text` (Markdown), `higherLevels?`, `cantripUpgrade?`, `area?` (§17.1 shape + sizes + scaling), `targeting` { `kind`: `area|creatures|self|point|object`, `count?`, `countPerSlot?` }, `attack?` { `kind`: `melee|ranged` }, `save?` { `ability`, `onSuccess`: `half|none|special` }, `damage?` [{ `formula`, `type`, `scaling?` }], `healing?`, `conditions?`, `effect?` (persistent-effect template: props and triggers), `light?`, `obscurement?`, `vfx`, `source` { `pack`, `page` }, `provenance` (which fields came from which source).

33.4 Special shapes to handle

Cylinders (radius × height, ft): Call Lightning 60 × 10 (the storm cloud; each bolt hits creatures within 5 ft of a chosen point), Conjure Celestial 10 × 40 (fills with Bright Light; moves 30 ft when you move), Flame Strike 10 × 40, Ice Storm 20 × 40, Magic Circle 10 × 20, Moonbeam 5 × 40, Reverse Gravity 50 × 100, Sleet Storm 20 × 40.

Emanations (move with the caster unless noted): 10 ft — Antilife Shell, Antimagic Field, Conjure Woodland Beings, Globe of Invulnerability (immobile), Tiny Hut (stationary); 15 ft — Conjure Minor Elementals, Spirit Guardians, and Darkness *when cast on an object* (moves with the object; otherwise a 15-ft-radius Sphere); 30 ft — Aura of Life, Holy Aura, Pass without Trace, Speak with Plants (immobile); 60 ft — Daylight *when cast on an object* (otherwise a 60-ft-radius Sphere).

Walls (the `wall` shape, §17.1): spells that create walls — for example Wall of Fire, Wall of Force, Wall of Ice, Wall of Stone, Wall of Thorns, Wind Wall, Blade Barrier and Prismatic Wall; R2 confirms which are in SRD 5.2.1 and their exact dimensions, ring options, damaging sides, opacity and whether they block movement.

33.5 Optional packs (after all required ACs pass)

- **SRD 5.1** (2014 rules) for groups that still use them: same pipeline with Open5e `srd-2014` / 5e-bits 2014 data and the 5.1 PDF; 319 spells (24 / 49 / 54 / 42 / 31 / 37 / 31 / 20 / 16 / 15). Rules differences switch with the campaign's rules pack (§34.8).
- **SRD creatures** for the DM's Bestiary (quick-spawn stat blocks: size, AC, HP formula, speeds, senses, saves, resistances, a text stat block) from Foundry `actors24` or Open5e `Creature.json` at the same pins.

33.6 Licence handling

SRD content is CC-BY-4.0. Show the attribution statement (Appendix I) in About & Credits, in `ATTRIBUTION.md` of each pack, and as a small line on SRD spell and condition cards. The SRD text itself mentions "D&D" in a couple of places (pp. 5 and 24); don't import those passages into the app's UI, and note any edits of SRD text in `ATTRIBUTION.md` as CC-BY requires. Never use "Dungeons & Dragons", "D&D" or other Wizards of the Coast trademarks in the app's name, logo or branding; the phrases "compatible with fifth edition" and "5E compatible" are permitted by the SRD's own terms.

34. Verified rules reference (SRD 5.2.1)

Page numbers refer to the official SRD 5.2.1 PDF. Conditions don't stack except Exhaustion.

34.1 Conditions

Condition	Summary (paraphrased)	Page
Blinded	Can't see; automatically fails checks that need sight; attack rolls against it have advantage; its attack rolls have disadvantage	177
Charmed	Can't attack the charmer or target it with damaging effects; the charmer has advantage on social checks against it	178
Deafened	Can't hear; automatically fails checks that need hearing	181

Condition	Summary (paraphrased)	Page
Exhaustion	Cumulative levels: D20 Tests –2 × level; Speed –5 ft × level; level 6 = death; a Long Rest removes one level	181
Frightened	Disadvantage on ability checks and attack rolls while the source is in line of sight; can't willingly move closer to the source	182
Grappled	Speed 0; disadvantage on attacks against anyone but the grappler; the grappler can drag it (+1 ft per ft, unless it's Tiny or two+ sizes smaller); escape: action, Athletics or Acrobatics vs DC 8 + grappler's Str mod + PB	182, 190
Incapacitated	No action, bonus action or reaction; concentration breaks; can't speak; disadvantage on initiative	184
Invisible	Advantage on initiative; unaffected by effects that require seeing it (unless the creator can); gear hidden too; attacks against it have disadvantage and its attacks have advantage, unless the other creature can see it	184
Paralyzed	Incapacitated; Speed 0; auto-fails Str and Dex saves; attacks against it have advantage; hits from within 5 ft are critical hits	186
Petrified	As Paralyzed but without the automatic crits; resistance to all damage; immune to Poisoned; weight ×10	186
Poisoned	Disadvantage on attack rolls and ability checks	186
Prone	Can only crawl or spend half its Speed to stand (not at Speed 0); its attacks have disadvantage; attacks against it have advantage within 5 ft, otherwise disadvantage; dropping prone is free	186, 14
Restrained	Speed 0; attacks against it have advantage; its attacks have disadvantage; disadvantage on Dex saves	187
Stunned	Incapacitated; auto-fails Str and Dex saves; attacks against it have advantage (no Speed 0 in 5.2.1)	189
Unconscious	Incapacitated and Prone (stays prone when it ends); drops what it holds; Speed 0; attacks against it have advantage; auto-fails Str and Dex saves; hits from within 5 ft are critical hits; unaware of surroundings	191

34.2 Status-related glossary

- **Bloodied** (p. 177): at half its HP or fewer; no effect on its own (p. 16). Not in 5.1.
- **Concentration** (p. 179): ends when starting another concentration effect, becoming Incapacitated, or dying; taking damage requires a Con save, DC = max(10, half the damage), maximum 30.
- **Heroic Inspiration** (p. 183): spend to reroll any die just rolled (must use the new roll); hold at most one.
- **Surprise** (pp. 13, 189): disadvantage on the initiative roll. (5.1: no move or action on the first turn and no reaction until it ends.)
- **Hide** (p. 183): DC 15 Dex (Stealth) while heavily obscured or behind three-quarters or total cover and out of enemies' line of sight; success grants the Invisible condition while hidden, and the check total becomes the Perception DC to find you; ends on noise louder than a whisper, being found, attacking, or casting a spell with a Verbal component.
- **0 HP** (pp. 17–18): monsters die unless the GM decides otherwise; player characters fall Unconscious and make death saves. Stable = 0 HP, no death saves, still Unconscious; regains 1 HP after 1d4 hours. A revived creature loses its attunements and returns with one fewer Exhaustion level.
- **Temporary HP** (p. 18): lost first; don't stack (choose which to keep); last until used or a Long Rest; aren't healing; don't restore consciousness.
- **Burning** (hazard, p. 178): 1d4 fire damage at the start of each turn; an action and going prone puts it out.
- **Rests** (glossary; R1 re-verifies the pages): a **Short Rest** lets a creature spend Hit Dice, rolling each die and adding its Con modifier to regain HP. A **Long Rest** restores all lost HP and all spent Hit Dice, returns a reduced HP maximum to normal, and removes one Exhaustion level (SRD 5.1: regain spent Hit Dice up to half your total). Features state their own recharge (short or long rest).

34.3 Vision, light and senses

- **Light** (p. 11): bright light is normal; dim light makes an area lightly obscured; darkness (including magical darkness) makes it heavily obscured.
- **Obscurement** (pp. 182, 184): lightly obscured → disadvantage on sight-based Perception checks; heavily obscured → Blinded when trying to see something there.
- **Blindsight** (p. 177): within range, sees anything not behind total cover, even while Blinded or in darkness, including Invisible creatures.
- **Darkvision** (p. 180): within range, dim light counts as bright and darkness as dim; only shades of grey in darkness.
- **Tremorsense** (p. 190): pinpoints creatures and moving objects within range touching the same surface or liquid; can't detect anything in the air; doesn't count as sight.

- **Truesight** (p. 190): within range, sees through normal and magical darkness, sees Invisible creatures and objects, sees visual illusions as transparent, sees true forms, sees into the Ethereal Plane.

Light source	Bright	+ Dim	Duration	Notes	Page
Candle	5 ft	5 ft	1 hour		96
Torch	20 ft	20 ft	1 hour		100
Lamp	15 ft	30 ft	6 h per flask of oil		98–99
Hooded Lantern	30 ft	30 ft	6 h per flask	Bonus action: lower the hood → dim light 5 ft only	98
Bullseye Lantern	60-ft cone	60 ft	6 h per flask	Cone $\approx 53.13^\circ$	98

Lighting a candle, lamp, lantern or torch with a Tinderbox is a bonus action (p. 100).

34.4 Light, darkness, obscurement and sense spells

Spell (page)	Lvl	Bright / +Dim (ft)	Obscurement / sense effect	Shape and size	Moves?	Conc.
Light (144)	0	20 / 20	—	touched object (Large or smaller)	with object	no (1 h)
Dancing Lights (121)	0	dim 10 per light, up to 4 lights	—	lights within 20 ft of each other, within 120 ft	bonus action: 60 ft	yes (1 min)
Starry Wisp (165)	0	dim 10 on target	target can't be Invisible	target	with target	—
Produce Flame (156)	0	20 / 20	—	in hand	with caster	no (10 min)
Continual Flame (119)	2	20 / 20	—	touched object	with object	no (until dispelled)
Flame Blade (132)	2	10 / 10	—	in hand	with caster	yes (10 min)
Flaming Sphere (132)	2	20 / 20	—	5-ft sphere	bonus action: 30 ft	yes
Shining Smite (162)	2	bright 5 on target	target can't be Invisible	target	with target	—
Faerie Fire (129)	1	dim 10 around each target	targets can't benefit from Invisible	20-ft cube	with targets	yes (1 min)
Moonbeam (150)	2	dim light fills area	—	cylinder r 5, h 40	Magic action: 60 ft	yes (1 min)
Daylight (122)	3	bright in area, +60 dim	dispels spell Darkness $\leq$ 3rd overlapping	sphere r 60, or 60- ft emanation from an object	object version moves	no (1 h)
Fire Shield (132)	4	10 / 10	—	self	with caster	no (10 min)
Wall of Fire (172)	4	not stated	wall is opaque	up to 60 × 20 × 1 ft, or ring 20 ft across × 20 × 1	fixed	yes (1 min)
Conjure Celestial	7	bright light fills area	—	cylinder r 10, h 40	moves 30 ft when you move	yes
Sunbeam (166)	6	30 / 30 (sunlight, from a mote above you)	—	line 60 × 5	mote with caster	yes (1 min)

Spell (page)	Lvl	Bright / +Dim (ft)	Obscurement /sense effect	Shape and size	Moves?	Conc.
Sunburst (167)	8	—	dispels spell-created Darkness in its area	sphere r 60	instantaneous	no
Prismatic Wall (155)	9	bright within 100, +100 dim	wall is opaque	wall	fixed	—
Darkness (122)	2	—	magical darkness (heavy); darkvision can't see through; nonmagical light can't light it; dispels spell light $\leq$ 2nd overlapping	sphere r 15, or 15-ft emanation from an object	object version moves	yes (10 min)
Fog Cloud (133)	1	—	heavily obscured	sphere r 20 (+20 per slot above 1st)	fixed; strong wind disperses	yes (1 h)
Sleet Storm (163)	3	extinguishes exposed flames	heavily obscured; difficult terrain	cylinder r 20, h 40	fixed	yes (1 min)
Stinking Cloud (165)	3	—	heavily obscured	sphere r 20	fixed; wind disperses	yes (1 min)
Cloudkill (116)	5	—	heavily obscured	sphere r 20	drifts 10 ft away from you at the start of each of your turns	yes (10 min)
Incendiary Cloud (142)	8	—	heavily obscured	sphere r 20	drifts 10 ft each turn in a direction you choose	yes (1 min)
Web (174)	2	—	lightly obscured; difficult terrain	20-ft cube	fixed	yes
Insect Plague (143)	5	—	lightly obscured	sphere r 20	fixed	yes
Tiny Hut (169)	3	interior can be dim or dark	opaque from outside	10-ft emanation (stationary)	fixed	no
Silence (162)	2 (ritual)	—	no sound; Deafened inside; immune to thunder; no Verbal spells	sphere r 20	fixed	yes (10 min)
Darkvision (122)	2	—	grants darkvision 150 ft (5.1: 60 ft)	touch	on target	no (8 h)
See Invisibility (160)	2	—	see Invisible creatures and into the Ethereal Plane	self	on caster	no (1 h)
True Seeing (171)	6	—	grants truesight 120 ft	touch	on target	no (1 h)
Invisibility (143)	2	—	Invisible; ends when the target attacks, deals damage or casts; +1 target per slot above 2nd	touch	on target	yes (1 h)
Greater Invisibility (137)	4	—	Invisible, doesn't end early	touch	on target	yes (1 min)

Hunger of Hadar is **not** in the SRD (neither 5.1 nor 5.2.1). Also relevant: Arcane Eye's eye has darkvision 30 ft (p. 110); Hallow has Darkness and Daylight options (p. 138); Gust of Wind and Wind Wall disperse fog and gas (pp. 138, 174).

34.5 Areas of effect (p. 177 onward)

- **Line of effect:** every area has a point of origin; a location is excluded if every straight line from the origin to it is blocked, and only total cover blocks a line. If you choose an unseen point with a wall between you and it, the origin appears on your side of the wall. Targeting a spell needs a clear path (nothing behind total cover, p. 106).
- **Cone** (p. 179): width at any point equals that point's distance from the origin; the effect sets the length; origin excluded unless the creator chooses.
- **Cube** (p. 179): origin anywhere on one face; size is the side length; origin excluded unless the creator chooses.

- **Cylinder** (p. 180): origin at the centre of the top or bottom circle; the effect sets radius and height; origin included.
- **Emanation** (p. 181): extends in all directions from a creature or object; moves with it unless instantaneous or stationary; the origin creature/object is excluded unless the creator chooses.
- **Line** (p. 184): the effect sets length and width; origin excluded unless the creator chooses.
- **Sphere** (p. 188): radius from the origin; origin included.
- 5.1 has the same total-cover rule and no Emanation shape.

34.6 Movement and combat

- **Your turn** (pp. 13–14): move up to your Speed and take one action, splitting movement around it; one free object interaction.
- **Difficult terrain** (pp. 14, 181): +1 ft per ft; never cumulative.
- **Climbing, crawling, swimming** (pp. 178, 179, 189): +1 ft per ft (+2 in difficult terrain); a climb or swim speed removes the extra cost for that mode.
- **Dash** (p. 180): extra movement equal to your Speed (a special speed can be used instead).
- **Jumps** (pp. 183–185): long jump up to Str score in feet with a 10-ft run-up (half standing); high jump 3 + Str mod feet (min 0; half standing); every foot jumped costs a foot of movement.
- **Moving through creatures** (p. 14): you can pass through an ally's space, an Incapacitated creature's, a Tiny creature's, or one two or more sizes larger or smaller. A space holding a creature that isn't Tiny and isn't your ally is difficult terrain. You can't willingly end your move in another creature's space; if you end your turn there you fall Prone unless you are Tiny or larger than it. (5.1: pass through any non-hostile creature; a hostile one only if two sizes different.)
- **Size and space** (p. 14): Tiny $2\frac{1}{2} \times 2\frac{1}{2}$ ft; Small and Medium 5×5 ; Large 10×10 ; Huge 15×15 ; Gargantuan 20×20 .
- **Initiative** (p. 13): a Dexterity check; one roll per group of identical creatures; optional fixed score = 10 + Dex mod (± 5 for advantage/disadvantage, p. 184).
- **Actions** (pp. 9–10): Attack, Dash, Disengage, Dodge, Help (also stabilises: DC 10 Wis (Medicine)), Hide, Influence (DC max(15, target's Int)), Magic, Ready (a readied spell needs concentration), Search, Study, Utilize.
- **Bonus action** (p. 10): only when a feature grants one; at most one per turn. **Reaction** (p. 10): one until the start of your next turn; resolves right after its trigger.
- **Opportunity attack** (pp. 15, 185): when a creature you can see leaves your reach using its action, bonus action, reaction or a speed, you may use your reaction for one melee attack just before it leaves. Disengage, teleporting and being moved without using your own movement don't provoke.
- **Cover** (p. 15): half +2 AC and Dex saves; three-quarters +5; total — can't be targeted directly. Only the best degree applies.
- **Death saving throws** (pp. 17–18): d20, 10+ succeeds; 3 successes → Stable; 3 failures → dead; natural 1 = two failures; natural 20 = regain 1 HP; damage at 0 HP = one failure (two from a critical hit); damage at 0 HP $\geq$ HP maximum → death.
- **Massive damage**: when damage reduces you to 0 HP and the remaining damage $\geq$ your HP maximum, you die. **Critical hit**: roll all the attack's damage dice twice (p. 16).
- **Damage types** (p. 180): acid, bludgeoning, cold, fire, force, lightning, necrotic, piercing, poison, psychic, radiant, slashing, thunder.
- **Order of adjustments** (p. 17): bonuses, penalties and multipliers first; then resistance (halve, round down); then vulnerability (double); each at most once per instance; immunity = no damage.

34.7 Spell counts

- **SRD 5.2.1: 339 spells** — cantrips 27; level 1: 57; 2: 57; 3: 42; 4: 34; 5: 38; 6: 31; 7: 20; 8: 17; 9: 16.
- **SRD 5.1: 319 spells** — 24; 49; 54; 42; 31; 37; 31; 20; 16; 15.
- New in 5.2.1 (22): Aura of Life, Befuddlement, Charm Monster, Chromatic Orb, Dissonant Whispers, Divine Smite, Dragon's Breath, Elementalism, Ensnaring Strike, Hex, Ice Knife, Mind Spike, Phantasmal Force, Power Word Heal, Ray of Sickness, Searing Smite, Shining Smite, Sorcerous Burst, Starry Wisp, Summon Dragon, Tsunami, Vitriolic Sphere. Dropped from 5.1: Branding Smite, Feeblemind.

34.8 Differences when a campaign uses SRD 5.1

Exhaustion is the 6-step table (disadvantage on checks → speed halved → disadvantage on attacks and saves → HP maximum halved → speed 0 → death); Stunned creatures can't move; Surprise skips the first turn; no Bloodied; no Emanation shape; Blindsight doesn't mention invisibility or total cover; Tremorsense can't detect flying or incorporeal creatures; Darkvision (spell) grants 60 ft; Produce Flame 10/+10; Continual Flame "as bright as a torch"; Holy Aura sheds dim light 5 ft; Darkness and Fog Cloud "spread around corners"; lighting a torch or lowering a lantern hood takes an action.

Part F — Delivery

35. Phase plan

Work through the phases in order. Each phase ends with a **gate**: its listed acceptance criteria pass with evidence in `docs/FEATURES.json`, `pnpm check` is green, the phase's E2E journeys pass, screenshots are reviewed by the visual critic (from Phase 2 on), and `PROGRESS.md` is updated. Commit at least once per feature group. If a later phase reveals a flaw in an earlier one, fix it at once and re-run that phase's tests.

Phase	Goal	Delivers	Gate: ACs mapped to the phase in Appendix E
P0 — Bootstrap, research, spikes	A verified foundation	Monorepo scaffold (§10), tooling, CLAUDE.md, <code>.claude/</code> config (Appendices B–D), <code>tools/extract-features.mjs</code> → <code>FEATURES.json</code> , research tasks R1–R5 in parallel, spikes S1–S6, SRD content pack built and verified	<code>pnpm check</code> green on an empty app; <code>pnpm content:build</code> asserts 339 spells; research docs exist; spikes' findings in <code>docs/research/stack.md</code> ; <code>FEATURES.json</code> has one entry per AC in this document
P1 — Server foundations	Identity, lobby, persistence, commands	Config, DB schema + migrations, secret key, admin setup/login/magic link, sessions, CSRF, rate limits, security log, invite codes, profiles/PINs/devices, lobby and table rooms with auth, command bus + history + permissions skeleton, snapshots, backups, <code>closeTable()</code> / <code>shutdown()</code> , crash hygiene, tunnel manager (quick/named/LAN/local)	HOST, AUTH (not AUTH-07), PER-01/03/04, SEC-01...06, SEC-08
P2 — The board	Beautiful, fast 3D table	Design tokens and UI kit (§27–28), HUD shell, R3F scene, camera, table environment, image/GLB/procedural maps, calibration, tokens (3 modes, linked/unlinked), bases, overlays (name, HP bar), selection, radial menu, Quick Unit, asset pipeline (images, GLB) with approvals and library, performance tiers, post-FX, first-load sequence, scene activation and transitions, DM prep view	BRD, TOK (not TOK-04/07/12/13), SCN (not SCN-06/07), AST, AUTH-07, DS-01/04/06
P3 — Walls and movement	Gridless movement	Walls/doors/windows/curtains/secret doors, zones, editor tools, 3D walls, shared pathfinding, routed and freehand drags, click-to-move with waypoints, range field worker, measurement tools, pings; exploration-mode movement end to end	WAL, MOV-02/03/08/11–14/17, TOK-07, FUN-02
P4 — Vision, light, fog	Light and senses matter	Lights and presets, senses, visibility polygons, perception rules, server vision service and per-client views, painted and dynamic fog with <code>fog.*</code> messages, explored memory, DM tools, view-as, sensed markers, partial-move clipping, client fog composite and grading	VIS (not VIS-06/07/08/15), SCN-07
P5 — Dice	Fair, visible, delightful dice	Parser/evaluator, server rolls, feed, visibility modes, manual entry, 3D dice worker with remap, skins, dice sounds (basic), roll requests	DICE (not DICE-06/11)
P6 — Character sheets	Sheets that bend to homebrew	Sheet schema and UI, derived values and overrides, custom blocks, templates, locks and proposals, JSON import/export + AI prompt, drawing pad, paper cutout, linked-token linkage, party spawn	SHEET, SCN-06, DICE-06, TOK-13
P7 — HP, conditions, death	Health and status	Damage/heal pipeline, temp HP, the icon set (Appendix G) in DOM and atlas, conditions and markers UI, exhaustion, bloodied, concentration prompts, death saves (outside combat via DM request), massive damage, NPC O-HP prompt, rests, feedback animations, condition roll hints	HP (not HP-08), TOK-04, TOK-12, VIS-15, DICE-11, DS-03

Phase	Goal	Delivers	Gate: ACs mapped to the phase in Appendix E
P8 — Combat	Structured turns	Quick start and start dialog, initiative collection, tracker (<code>combat.view</code>), turn flow with start/end processing, budgets, reset/undo segment, Dash/stand/crawl, creature spaces, opportunity-attack warnings, movement enforcement, action pips, DM controls, stop and summary, round-based durations	CMB, MOV-01/04/05/07/09/10/15/16/18, HP-08
P9 — Spells and effects	Magic on the board	Spell browser and cards, casting flow, templates and targeting, resolution cards, slots, concentration links, persistent effects (16 named, including wall shapes), obscurement/darkness/invisibility into vision, VFX presets, homebrew builder, import (UI/REST), weapon attacks through the same flow, cover hint	SPL, VIS-06/07/08
P10 — Undo and saves	Mistakes are cheap	Undo/redo with conflicts, history panel, revert/restore-to-here, snapshots UI, restore, export/import <code>.gloam</code> , reconnect banner, crash-recovery E2E	UNDO, PER-02/05/06/07, MOV-06
P11 — Audio and flavour	The table feels alive	Audio engine, all §31 sounds, <code>/dev/sounds</code> audition page, music player and sync, generative music presets, ambience layers, mixer, emotes/phrases, hand raise, handouts and secret notes, campaign log	AUD, FUN-01/03/04/05
P12 — DM panel, admin console, demo	Everything two clicks away	All DM sections, overrides, act-as, approvals inbox, command palette, admin sections, people management, settings, security log viewer, first-run checklist, About & Credits, the Lantern Crypt demo campaign	DMP, ADM, DEMO
P13 — Local API and MCP	Connect Claude	REST v1, API tokens and scopes, JSON Schemas endpoint, MCP server and tools, Connect Claude page	API
P14 — Responsive and touch	Phones and tablets (the last feature phase, as Xini asked)	Breakpoint layouts, bottom sheets, gestures, iOS/Android quirks, Low-tier verification, accessibility pass	RSP, A11Y
P15 — Hardening and polish	Ready for game night	Security review and fixes, upload fuzzing, full frame inspection for hidden info, performance profiling and fixes, final visual-critic rounds, <code>docs/HOSTING.md</code> , README, final full E2E run	SEC-07, SEC-09, PERF, DS-02, DS-05 — and every AC re-verified

Although phones and tablets get their dedicated pass in P14, build every screen from P2 onward with fluid layouts, tokens-based spacing and touch-sized hit areas in mind, so P14 adapts rather than rewrites.

After P15, if time remains: the optional content packs (§33.5), then quality improvements the critics flagged as nice-to-have.

36. Testing and verification

36.1 Layers

Layer	Tool	Scope
Unit	Vitest	<code>packages/shared</code> : geometry (property tests vs brute force), visibility, pathfinding (cost parity client/server), AoE inclusion and line of effect, cover hint, dice grammar and evaluation (including limits and distributions), damage pipeline (≥ 30 cases), death saves, concentration DC, speed/budget, initiative sort, schema round-trips, JSON Schema generation
Integration	Vitest + in-process server	Start the server on a random port with a temp <code>DATA_DIR</code> ; connect <code>@colyseus/sdk</code> clients with cookies from the REST join flow; exercise commands; assert state per client (views), DB rows, history, snapshots; restart the server and assert persistence

Layer	Tool	Scope
End-to-end	Playwright (Chromium; WebKit for the mobile subset)	The journeys J1–J10 (§7) with multiple contexts (Admin/DM, Player A, Player B, Spectator); touch emulation for RSP; axe-core checks; WebSocket frame inspection for hidden-info leaks; CSP violation monitoring (fail on any)
Visual	<code>pnpm shots</code> + <code>visual-critic</code> subagent	Key screens at 1440×900, 1024×768 and 390×844 (and 844×390 for landscape); the critic scores against Part D and the banned list
Performance	<code>pnpm bench</code>	Benchmark scene (§37) in headed Chromium on the host GPU (headless WebGL is software-rendered and meaningless for fps), frame-time sampling via <code>requestAnimationFrame</code> deltas, server timings from pino metrics
Security	<code>tests</code> + <code>security-reviewer</code>	Upload fuzz corpus, auth/rate-limit tests, header assertions, local-only guard tests, frame inspection
Rules	<code>rules-auditor</code> subagent	Reads the rules code and tests, compares with §34 and the R1 research, reports discrepancies

36.2 Conventions

- Tests live next to code (`foo.ts` → `foo.test.ts`) in `packages/*` ; E2E in `e2e/journeys/` .
- **Never delete or weaken a test to make it pass.** If a test is wrong, fix it and explain in the commit message.
- Deterministic: test mode seeds dice (`GLOAM_TEST_SEED`), fixes the clock where needed, disables animations via `reduced-motion` (except in visual shots), and uses the fake `cloudflare` script.
- Evidence: when marking an AC as passing, set `evidence` to the test file and name (for example `e2e/journeys/j4-darkness.spec.ts > B never receives goblin`) or the screenshot path.

36.3 The visual critic loop

After each phase from P2 on: run `pnpm shots` , then dispatch `visual-critic` with the list of new screenshots and the relevant spec sections. It returns a scored list: **blocking** (breaks the spec or a banned pattern; must fix), **important** (clearly hurts quality; fix if cheap), **nice** (optional). Fix blocking and cheap important items, re-shoot, and run the critic once more at most.

37. Performance budgets

Benchmark scene (`pnpm bench` , also seeded as a hidden demo scene): 200 × 150 ft image map; 40 tokens (10 GLB minis ≤ 20 k triangles each sharing 3 models, 30 standees/coins); 300 walls with 12 doors; 20 lights (8 animated); 3 persistent effects (fog cloud, darkness, spirit guardians); dynamic fog with 4 player viewers; combat active.

Metric	Target
Desktop, High tier, 1080p, headed Chromium on the host (target class: Apple M1 / Intel Iris Xe)	≥ 60 fps median, p95 frame ≤ 20 ms
Tablet (iPad 9th gen class), Medium tier	≥ 30 fps
Phone (iPhone 12 / Pixel 6 class), Low tier; proxy: headed Chromium on the host GPU, DPR 1, 4× CPU throttle	≥ 30 fps
Initial table-route JS	≤ 1.2 MB gzipped
Time to board for a joining player (cached app, 20 Mbps)	≤ 4 s
Server command handling	p95 < 5 ms
Vision recomputation (8 players)	p95 < 10 ms
Path preview query (500 walls)	< 4 ms
Range field (60 ft budget)	< 30 ms in worker
Dice simulation (10 dice)	< 60 ms in worker
Memory (desktop tab, benchmark scene)	< 800 MB

38. Definition of done

The build is done when all of the following hold:

1. Every entry in `docs/FEATURES.json` has `"passes": true` with evidence, or `"disputed"` with a reason Xini would accept (logged in DECISIONS.md).
2. `pnpm check`, `pnpm test:e2e` and `pnpm bench` pass on a clean clone (`git clone`, `corepack enable`, `pnpm install`, `pnpm build`).
3. A fresh `pnpm start` on an empty data directory gets a new user from setup to a playable demo with a second (private-window) player in under 3 minutes.
4. The final visual-critic pass has no blocking items; the security reviewer has no open high-severity items; the rules auditor has no open discrepancies against §34.
5. `docs/HOSTING.md` explains install, first run, opening the table, quick vs named tunnels, LAN mode, backups, updating, and troubleshooting (cloudflared missing, port in use, quick tunnel won't start because of `config.yaml`, players can't connect).
6. No TODOs, placeholders or dead controls in shipped code paths (`grep -R "TODO\|FIXME\|lorem" packages/*/src` is empty or each hit is justified).

39. Host runbook (becomes `docs/HOSTING.md`)

1. **Install once:** Node.js 24 LTS; Git; `corepack enable`; then `git clone` (or use the build folder), `pnpm install`, `pnpm build`.
2. **Start:** `pnpm start`. The terminal shows the local address and a one-time admin link. First run: open the setup link, choose a password (≥ 12 characters), optionally create the demo campaign.
3. **Install cloudflared** (for friends outside your home): macOS `brew install cloudflared`; Windows `winget install --id Cloudflare.cloudflared`; Linux: Cloudflare's package repository (`.deb` / `.rpm`) or the binary. Check with `cloudflared --version`.
4. **Game night:** Admin → Table → **Open table** (Quick tunnel). Copy the Discord message; admit people as they knock; appoint the DM (Admin → People) if it isn't you; activate the scene.
5. **Stable link (optional):** create a free Cloudflare account, add your domain, create a tunnel in the Cloudflare dashboard (Zero Trust → Networks → Tunnels) with a public hostname pointing to `http://127.0.0.1:4747`, copy its token into Admin → Settings → Tunnel (Named), and choose Named mode.
6. **End: Close table** (or Ctrl+C in the terminal). Everything is already saved.
7. **Backups:** automatic daily in `data/backups/`; copy the whole `data/` folder elsewhere occasionally. Export campaigns as `.gloom` files to move them between machines.
8. **Update:** `git pull`, `pnpm install`, `pnpm build`, `pnpm start` (migrations run automatically; a database backup is taken first).
9. **Troubleshooting:** port 4747 busy → `PORT=4750 pnpm start`; quick tunnel won't start → rename `~/.cloudflared/config.yaml`; a player sees "can't join" → check the invite code, lock toggle and ban list; audio silent on iPhone → tap the speaker chip; slow on a phone → Settings → Graphics → Low.

40. Risks and fallbacks

Risk	Fallback
Colyseus cookie not sent with matchmaking requests in some browser	Short-lived signed join ticket via <code>POST /api/table/ticket</code> , passed in join options, verified in <code>onAuth</code>
A Colyseus 0.18 API differs from this spec	Follow the official docs; keep the behaviour; record in DECISIONS.md
Node type stripping blocks something	Run the server with <code>tsx</code> (allow <code>esbuild</code> builds in pnpm)
TypeScript 7 incompatibility in some tool	Pin <code>typescript@6.0.3</code> for that package

Risk	Fallback
SRD PDF download blocked	Use a mirror only if its SHA-256 matches the pinned hash; otherwise build the canonical list from Foundry + Open5e at the pinned versions, run the same count assertions, and flag for R1 review
Foundry data layout changed	Use the pinned commit (never <code>main</code>); if unavailable, use Open5e for mechanics and prose extraction for areas
Rapier deterministic build too slow on phones	Use <code>@dimforge/rapier3d-compat</code> (same API) on the Low tier; results still correct via remapping
<code>StateView</code> performance issue	Batch view updates per patch; reduce recompute frequency during drags; move vision to a worker thread
WebGL context loss (mobile)	Handle <code>webglcontextlost/restored</code> : rebuild render targets, re-upload textures, show "Restoring board..."
cloudflared not installable	Local/LAN modes; document alternatives in <code>HOSTING.md</code>
<code>draco3dgltf</code> can't decode a particular Draco GLB	Reject that file with a clear message and instructions to re-export without Draco compression
Very large maps exceed mobile GPU limits	Variant selection (§21.3); if needed tile the map into 4096 ² chunks

41. Out of scope (don't build)

- Text or voice chat (the group uses Discord); video.
- Any third-party account, sign-in provider, cloud storage, analytics or AI API inside the app.
- Payment, marketplace, public sharing of campaigns.
- Grids (square or hex) — Gloam is deliberately gridless.
- Full character-builder automation (class progression, feat prerequisites, multiclass rules) beyond the derived values in F10.
- Automatic parsing of arbitrary uploaded sheet PDFs inside the app (use the AI prompt or MCP flow instead).
- Splitting the party across different active scenes at once.
- Automatic dragging of grappled creatures, jumping automation, falling damage automation, encumbrance automation, and a calendar/time-of-day tracker (the DM handles these manually).
- Non-SRD copyrighted content of any kind.

Appendices

Appendix A — Kickoff prompt and goal

Before pasting: the repository folder contains `docs/SPEC.md` (this document) and nothing else; Claude Code is running Opus 5.5; effort `high` (`/effort high`, or launch with `claude --effort high`); permission mode **auto**. If the run stops (usage limits, a crash, a closed terminal), resume with `claude --continue`: an active goal is restored automatically.

Kickoff prompt (paste as your first message):

```
Build Gloam — a self-hosted, account-free 3D virtual tabletop for my D&D group — from the
specification in docs/SPEC.md.

1. Read docs/SPEC.md completely before doing anything else. It is the source of truth.
   Part A §4 says how to work; follow it exactly, including the standing instruction about
   how turns end.
2. Phase 0 (§35): scaffold the monorepo (§10); create CLAUDE.md, .claude/settings.json,
   .claude/agents/* and .claude/hooks/* from Appendices B–D; write tools/extract-features.mjs
   and tools/features-status.mjs (Appendix E) and generate docs/FEATURES.json; dispatch the
   research subagents R1–R5 in parallel (§32); run spikes S1–S6 yourself; build and verify
   the SRD content pack (§33).
3. Then work through phases P1–P15 in order. Do not ask me questions: decide, log the
   decision in docs/DECISIONS.md, and continue. Keep docs/PROGRESS.md current and commit
   after every feature group.
4. The quality bar is a crafted indie game, not a CRUD app (§1, Part D). Implement every
   feature and interaction the spec describes, completely — no stubs, placeholders or dead
   buttons. Prove each acceptance criterion with tests or screenshots before marking it.
5. Use subagents for research and independent review only; build coupled systems yourself,
   sequentially (§4.5).
6. Your context will be compacted automatically as it fills, so never stop early to save
   tokens. Keep state in the docs files and re-read CLAUDE.md, the tail of PROGRESS.md and
   the current phase after any compaction.

Start now with Phase 0.
```

Goal (run right after the kickoff prompt; it keeps the session working across turns):

```
/goal Every entry in docs/FEATURES.json has "passes": true with a non-empty evidence string
(or a "disputed" reason logged in docs/DECISIONS.md), and the most recent run of
`pnpm features:status --verify && pnpm check && pnpm test:e2e && pnpm bench`, shown in this
conversation, exits 0 with 0 failing and the bench within its budgets. Acceptance criteria and tests must not be deleted or weakened
there.
```

Appendix B — CLAUDE.md

Keep it short; it is loaded every session. Create it in Phase 0 with this content (update the commands if they change):

```
# Gloam — project memory

Self-hosted 3D virtual tabletop. **Source of truth: docs/SPEC.md** (read the relevant
section before working on anything). Progress: docs/PROGRESS.md. Decisions: docs/DECISIONS.md.
Acceptance criteria: docs/FEATURES.json (never delete or reword entries; only flip `passes`
with evidence).

## Commands
- pnpm dev · pnpm build · pnpm start
- pnpm check (typecheck + lint + tests; must pass before every commit) · pnpm check:fast
- pnpm test · pnpm test:e2e · pnpm shots · pnpm bench
- pnpm content:build · pnpm features:extract · pnpm features:status --verify

## Rules that are easy to forget
- Server authority: every mutation goes through the command bus (SPEC §14); hidden info
  never reaches a player's client (SPEC §13.4, §15).
- No network at runtime: no CDNs, no Google Fonts, no drei Environment presets, no Draco
```

- decoder on the client (the server decodes Draco uploads), drei <Text> always with a local .woff font (SPEC §9.4, §24.9).
- Server TS runs natively on Node 24: erasableSyntaxOnly, .ts extensions in relative imports, no enums/decorators/namespaces.
- Colyseus: use @colyseus/core + ws-transport + tools + schema (not the `colyseus` meta-package); schema() builder; ≤ 63 fields per schema; view tags are powers of two; .view() collections are undefined until the first add.
- pnpm 12: build scripts need `allowBuilds` in pnpm-workspace.yaml.
- Colours, type, spacing and motion only from design tokens (SPEC §27). No banned patterns (SPEC §27.6).
- Every automation offers Skip/manual override (SPEC §2 P2).
- Never weaken or delete a test to make it pass.

When compacting

Preserve: current phase and its open ACs, files modified since the last commit, failing tests and their error messages, and any decision not yet written to DECISIONS.md.

Appendix C — Subagents (.claude/agents/*.md)

```
---
name: rules-researcher
description: Verifies D&D SRD 5.2.1 rules facts against the official PDF and writes cited research notes. Use for Phase 0 task R1 and any later rules question.
tools: Read, Grep, Glob, Write, Bash, WebFetch, WebSearch
model: inherit
---
```

You verify tabletop rules facts for the Gloam build. Sources, in priority order: the official SRD 5.2.1 PDF (download from the pinned URL in docs/SPEC.md §33.1 and check its SHA-256), then the SRD 5.1 PDF for differences. Cite every fact with a page number. Never guess: if you cannot verify something, say so. Write paraphrased summaries (≤ 160 characters each) – never long verbatim passages. Output: docs/research/rules-5.2.1.md (every table of SPEC §34 re-verified, with a "Discrepancies with SPEC" section) and packages/content/overlays/conditions.json (id, name, summary, page, automation flags per SPEC §19.3). Return a 10-line summary of what you verified and any discrepancies.

```
---
name: spell-data-engineer
description: Builds and runs the SRD spell content pipeline (download, parse, merge, overlay, validate, assert counts). Use for Phase 0 task R2.
tools: Read, Write, Edit, Bash, Grep, Glob, WebFetch
model: inherit
---
```

Implement docs/SPEC.md §33 exactly in packages/content/scripts, using the pinned sources and hashes. The output must validate against the spell zod schema in packages/shared (Appendix F.2 of the spec; create the schema there if it doesn't exist yet) and must contain exactly 339 spells with per-level counts 27/57/57/42/34/38/31/20/17/16. Never include non-SRD content (drop Foundry's arcane-vigor). Write docs/research/spells-report.md listing, per spell, missing structured fields and how they were resolved (source, prose pattern, overlay), plus every cross-check disagreement. Commit nothing; return a summary with the counts and the number of spells needing manual review.

```
---
name: stack-verifier
description: Re-checks library versions and APIs used by the Gloam spec and reports breaking changes with minimal verified snippets. Use for Phase 0 task R3 and when an API behaves unexpectedly.
tools: Read, Write, Bash, WebFetch, WebSearch, Grep, Glob
model: inherit
---
```

For every row of docs/SPEC.md §9 and every gotcha in §9.4: confirm the current stable version on npm, read the official docs/changelog for breaking changes since the version named, and, where the spec shows code, verify it by running a minimal script in a temporary directory. Output docs/research/stack.md with: version table (spec vs current), verified snippets, deviations the builder must know, and recommended pins. Be concrete and brief.

```
---
name: visual-critic
description: Harsh, independent reviewer of Gloam screenshots against the design system. Read-only. Use after
```

every phase from P2 on.
tools: Read, Glob, Grep

model: inherit

You are a demanding art director for a premium indie game. You did not build this and you owe it nothing. Review the screenshots you are given (open each PNG with Read) against docs/SPEC.md Part D (§27–§31), the relevant feature sections, and the banned-pattern list in §27.6. Judge: hierarchy and readability of the board, typography, colour discipline (tokens only), spacing, alignment, states (hover/selected/empty/loading/error), motion evidence where visible, mobile layouts, and whether it looks crafted rather than generic. Return a list of findings, each tagged BLOCKING (violates the spec or a banned pattern), IMPORTANT (clearly hurts quality), or NICE, with the screenshot file, the exact element, and a concrete fix. End with a 1–10 score. Do not suggest features outside the spec.

name: rules-auditor

description: Independent audit of rules automation code and tests against the verified rules reference. Use at the end of phases P4, P9 and P15.

tools: Read, Grep, Glob, Bash

model: inherit

Compare packages/shared/src/{rules,vision,movement,aoe,dice} and their tests with docs/SPEC.md §15–§19 and §34 and with docs/research/rules-5.2.1.md. Run the test suites (`pnpm test`) and read failures. Report only discrepancies that affect correctness or the spec (wrong DC formula, wrong rounding order, missing condition flag, wrong shape geometry, off-by-one in death saves, etc.), each with file, line, the rule and page reference, and a suggested fix.

name: security-reviewer

description: Independent security review of the Gloam server and client. Use at the end of phases P4, P9 and P15.

tools: Read, Grep, Glob, Bash

model: inherit

Review against docs/SPEC.md §22 and the SEC acceptance criteria: auth and sessions, invite codes and rate limits, local-only guard, CSRF and Origin checks, CSP and headers, room message validation and rate limits, per-client views and hidden-information leaks (tokens, walls, lights, rolls, notes), upload pipeline and worker isolation, path traversal, command injection (cloudflared spawn), logging of secrets, dependency audit. Run the security-related tests and `pnpm audit --prod`. Report findings as HIGH/MEDIUM/LOW with file, line, exploit scenario and fix. Don't report style issues.

Appendix D — `.claude/settings.json` and hook scripts

```
{
  "permissions": {
    "allow": [
      "Bash(pnpm *)", "Bash(node *)", "Bash(npx playwright *)", "Bash(git *)",
      "Bash(corepack *)", "Bash(cloudflared --version)",
      "WebSearch",
      "WebFetch(domain:media.dndbeyond.com)", "WebFetch(domain:www.dndbeyond.com)",
      "WebFetch(domain:raw.githubusercontent.com)", "WebFetch(domain:github.com)",
      "WebFetch(domain:api.open5e.com)", "WebFetch(domain:open5e.com)",
      "WebFetch(domain:docs.colyseus.io)", "WebFetch(domain:www.npmjs.com)",
      "WebFetch(domain:registry.npmjs.org)", "WebFetch(domain:threejs.org)",
      "WebFetch(domain:r3f.docs.pmnd.rs)", "WebFetch(domain:drei.docs.pmnd.rs)",
      "WebFetch(domain:orm.drizzle.team)", "WebFetch(domain:zod.dev)", "WebFetch(domain:vite.dev)",
      "WebFetch(domain:developers.cloudflare.com)", "WebFetch(domain:modelcontextprotocol.io)",
      "WebFetch(domain:www.redblobgames.com)", "WebFetch(domain:gltf-transform.dev)",
      "WebFetch(domain:rapier.rs)", "WebFetch(domain:sharp.pixelpilumbing.com)"
    ],
    "deny": [
      "Bash(git push *)",
      "Read(./data/secret.key)"
    ]
  },
  "hooks": {
    "PostToolUse": [
      { "matcher": "Edit|Write",
```

```

    "hooks": [ { "type": "command",
                  "command": "node \"${CLAUDE_PROJECT_DIR}/.claude/hooks/post-edit.mjs\"",
                  "timeout": 30 } ] },
  ],
  "Stop": [
    { "hooks": [ { "type": "command",
                    "command": "node \"${CLAUDE_PROJECT_DIR}/.claude/hooks/stop-check.mjs\"",
                    "timeout": 300 } ] }
  ]
}
}

```

`.claude/hooks/post-edit.mjs` — reads the hook JSON from stdin, takes `tool_input.file_path`, and if it is a `.ts`, `.tsx`, `.json` or `.css` file inside the repo (not under `docs/`), runs `pnpm exec biome format --write <file>`. Always exits 0 (formatting only; never blocks).

`.claude/hooks/stop-check.mjs` — reads the hook JSON from stdin; if `stop_hook_active` is true, exits 0. Otherwise runs `pnpm -s check:fast` and `node tools/features-status.mjs --verify`. If either fails, it writes a short summary (the first 40 lines of errors) to **stderr** and exits **2**, which prevents the turn from ending so the errors get fixed; otherwise exits 0. Must work on Windows, macOS and Linux (Node only, no shell features).

Appendix E — `docs/FEATURES.json` and tools

Format:

```

{
  "specVersion": "1.0",
  "generatedFrom": "docs/SPEC.md",
  "features": [
    { "id": "AC-MOV-02", "feature": "F06", "code": "MOV", "phase": "P3",
      "text": "The client preview cost equals the server's authoritative cost within 0.05 ft...",
      "passes": false, "evidence": null, "disputed": null, "updatedAt": null }
  ]
}

```

`tools/extract-features.mjs`: read `docs/SPEC.md`; match lines `^- \ (AC-[A-Z0-9]+\d{2})`(.+)$`; derive code from the ID and feature / phase from the maps below; merge with the existing file (preserve `passes`, `evidence`, `disputed`, `updatedAt` for IDs that still exist); fail on duplicate IDs; write pretty JSON; print counts per phase.

`tools/features-status.mjs`: print `PASSING n/N · DISPUTED d · FAILING f` and a per-phase table. With `--verify`: fail (exit 1) if any `passes: true` entry has empty `evidence`, or if the ID set differs from the spec's.

Code → feature → default phase:

Code	Feature	Default phase	Exceptions
HOST	F01	P1	
AUTH	F02	P1	AUTH-07 → P2
SCN	F03	P2	SCN-06 → P6; SCN-07 → P4
BRD	F04	P2	
TOK	F05	P2	TOK-04, TOK-12 → P7; TOK-07 → P3; TOK-13 → P6
MOV	F06	P3	MOV-01, 04, 05, 07, 09, 10, 15, 16, 18 → P8; MOV-06 → P10
WAL	F07	P3	
VIS	F08	P4	VIS-06, 07, 08 → P9; VIS-15 → P7
DICE	F09	P5	DICE-06 → P6; DICE-11 → P7

Code	Feature	Default phase	Exceptions
SHEET	F10	P6	
HP	F11	P7	HP-08 → P8
CMB	F12	P8	
SPL	F13	P9	
UNDO	F14	P10	
PER	F15	P10	PER-01, 03, 04 → P1
AST	F16	P2	
AUD	F17	P11	
FUN	F18	P11	FUN-02 → P3
DMP	F19	P12	
ADM	F20	P12	
RSP	F21	P14	
A11Y	F22	P14	
API	F23	P13	
DEMO	F24	P12	
SEC	cross-cutting	P1	SEC-07, SEC-09 → P15 (SEC-07's frame inspection starts in P4 and grows each phase)
PERF	cross-cutting	P15	
DS	cross-cutting	P2	DS-03 → P7; DS-02, DS-05 → P15

This table is the single source of truth for phase gates: `tools/extract-features.mjs` assigns `phase` from it, and a phase's gate is "every AC whose phase is this phase passes".

Appendix F — Import formats and AI conversion prompts

F.1 Rules shared by all imports

- UTF-8 JSON. Distances in feet. Damage types from the 13 SRD types. Dice in the §18.1 grammar.
- Validation is strict: unknown fields are rejected, so the AI prompt includes the exact JSON Schema.
- The prompts below contain `{{SPELL_SCHEMA_JSON}}`-style markers. **Copy AI prompt** replaces each marker with the pretty-printed schema served at `/api/v1/schemas/<kind>.json`, so the text on the clipboard is complete and never contains a marker.
- Every import supports a dry run with a report and a conflict strategy (skip / overwrite / rename).

F.2 Spell

```
{
  "id": "poison-ball",
  "name": "Poison Ball",
  "level": 3,
  "school": "evocation",
  "classes": ["sorcerer", "wizard"],
  "castingTime": { "amount": 1, "unit": "action" },
  "ritual": false,
  "range": { "kind": "ranged", "ft": 150 },
  "components": { "v": true, "s": true, "m": true, "material": "a pinch of bitter moss" },
  "duration": { "kind": "instantaneous", "concentration": false },
  "text": "A hissing green orb bursts at a point you choose within range...",
  "higherLevels": "The damage increases by 1d6 for each slot level above 3.",
  "targeting": { "kind": "area" },
  "area": { "shape": "sphere", "radius": 20 },
}
```

```

"save": { "ability": "con", "onSuccess": "half" },
"damage": [ { "formula": "8d6", "type": "poison", "scaling": { "mode": "slot", "perLevel": "1d6" } } ],
"conditions": [ { "id": "poisoned", "onFailedSave": true, "duration": { "rounds": 10 } } ],
"effect": null,
"light": null,
"obscurement": null,
"vfx": "poison",
"source": { "pack": "homebrew" }
}

```

Field notes: `school` ∈ abjuration, conjuration, divination, enchantment, evocation, illusion, necromancy, transmutation.

`castingTime.unit` ∈ action, bonus, reaction, minute, hour (`reactionTrigger` text for reactions). `range.kind` ∈ self, touch, ranged, sight, unlimited, special. `duration.kind` ∈ instantaneous, timed (`amount` + `unit` ∈ round, minute, hour, day), until-dispelled, special. `area.shape` ∈ sphere (`radius`), cylinder (`radius`, `height`), cone (`length`), cube (`size`), line (`length`, `width`), emanation (`distance`), wall (`length`, `height`, `thickness`, optional `ring` diameter, `opaque`, `blocksMove`, `damagingSide`), each with optional `scaling` of size per slot level. `damage[].scaling.mode` ∈ slot (perLevel) or cantrip (`atLevels`: { "5": "2d10", "11": "3d10", "17": "4d10" }). `effect` (persistent area) = { props: {...}, triggers: [...], attach: "caster"|"object"|"point", movement: {...} } as in §12.3. `vfx` ∈ the 12 presets.

AI prompt (copied by "Copy AI prompt" in Import spells):

Convert the spells I give you into a JSON array for the Gloam virtual tabletop.
Output ONLY the JSON array – no prose, no code fences.

Rules:

1. One object per spell, validating against the JSON Schema below. Unknown fields are not allowed.
2. "id" is the kebab-case of the name. Cantrips have level 0.
3. Use only information present in my text. Never invent numbers. Omit optional fields you can't determine.
4. Areas: sphere (radius), cylinder (radius, height), cone (length), cube (size), line (length, width; default width 5), emanation (distance), wall (length, height, thickness, optional ring diameter, opaque, blocksMove, damagingSide). All distances in feet (convert metres: 1.5 m = 5 ft).
5. Damage: dice notation like "8d6", type from: acid, bludgeoning, cold, fire, force, lightning, necrotic, piercing, poison, psychic, radiant, slashing, thunder. Upcasting → `scaling { "mode": "slot", "perLevel": "1d6" }`; cantrip scaling → `{ "mode": "cantrip", "atLevels": {...} }`.
6. Saves: ability str|dex|con|int|wis|cha; onSuccess half|none|special.
7. Choose "vfx" from: fire, cold, lightning, thunder, acid, poison, necrotic, radiant, force, psychic, healing, arcane.
8. Set "source": { "pack": "homebrew" }.

JSON Schema:

```
{SPELL_SCHEMA_JSON}
```

My spells:

F.3 Character sheet

```

{
  "schemaVersion": 1,
  "core": {
    "name": "Thorin Emberhand",
    "species": "Dwarf", "background": "Soldier", "alignment": "Lawful Good",
    "classes": [ { "name": "Fighter", "level": 5, "subclass": "Champion" } ],
    "abilities": { "str": 16, "dex": 12, "con": 16, "int": 10, "wis": 13, "cha": 8 },
    "saves": { "str": { "proficient": true }, "con": { "proficient": true } },
    "skills": { "athletics": { "prof": "proficient" }, "perception": { "prof": "proficient" } },
    "ac": { "value": 18, "note": "chain mail + shield" },
    "speeds": { "walk": 30 },
    "size": "medium",
    "hp": { "max": 44, "current": 44, "temp": 0 },
    "hitDice": [ { "die": "d10", "total": 5, "used": 0 } ],
    "senses": { "darkvision": 120 },
    "resistances": [ "poison" ],
    "attacks": [ { "name": "Warhammer", "attack": "1d20 + @str + @prof", "damage": "1d8 + @str [bludgeoning]", "range": "5 ft" } ],
    "spellcasting": null,
    "inventory": [ { "name": "Torch", "qty": 5, "light": "torch" } ],
    "features": [ { "name": "Second Wind", "text": "...", "uses": { "max": 2, "used": 0, "recharge": "short" } } ],
    "languages": [ "Common", "Dwarvish" ],
  }
}

```

```

    "notes": ""
  },
  "custom": [
    { "id": "b1", "type": "counter", "title": "Sanity", "value": 8, "max": 10, "pinToToken": true },
    { "id": "b2", "type": "text", "title": "Oath of the Forge", "markdown": "..." }
  ],
  "importNotes": ["Portrait not included in source"]
}

```

Attack and damage formulas may end with a damage-type tag in brackets, which types every untagged term (§18.1). Custom block types: `text` (`markdown`), `number` (`value`), `counter` (`value`, `max`, `pinToToken`), `checkList` (`items`[`{ label, done }`]), `table` (`columns`[], `rows`[][]), `keyValue` (`entries`[`{ key, value }`]), `image` (`assetId`).

AI prompt (copied by "Import with AI..."):

You are converting a tabletop RPG character sheet into JSON for the Gloam virtual tabletop.
Output ONLY one JSON object that validates against the JSON Schema below – no prose, no fences.

Rules:

1. Use only information present in the sheet I provide (text, photo or PDF). Never invent values. If something is unknown, omit it.
2. Put standard fifth-edition fields in "core". Put everything that doesn't fit – homebrew stats, custom resources, unusual sections – in "custom" blocks, choosing the closest type: `text`, `number`, `counter` (`value`/`max`), `checklist`, `table`, `keyValue`.
3. Distances in feet (1.5 m = 5 ft). Ability scores are the scores, not modifiers.
4. Attack and damage formulas use dice notation; you may use `@str` `@dex` `@con` `@int` `@wis` `@cha` `@prof` references. Tag damage types in brackets, e.g. "1d8 + `@str` [slashing]".
5. Keep names of spells, features and items exactly as written.
6. List anything you were unsure about in "importNotes".

JSON Schema:

```
{{CHARACTER_SCHEMA_JSON}}
```

The sheet:

F.4 Monster (homebrew bestiary entry)

```

{
  "id": "bog-lurker", "name": "Bog Lurker", "size": "large", "type": "monstrosity",
  "ac": 13, "hp": { "average": 59, "formula": "7d10 + 21" },
  "speeds": { "walk": 20, "swim": 40 }, "senses": { "darkvision": 60, "tremorsense": 30 },
  "abilities": { "str": 18, "dex": 12, "con": 16, "int": 3, "wis": 12, "cha": 5 },
  "saves": { "con": 5 }, "resistances": ["cold"], "immunities": [], "vulnerabilities": [],
  "conditionImmunities": ["prone"], "cr": "3",
  "statBlockMarkdown": "...", "tokenAssetId": null, "source": { "pack": "homebrew" }
}

```

Appendix G — Icon reference drawings

Reference SVGs for every condition and status icon (§30). Each is a 24 × 24 glyph drawn in `currentColor`; on tokens it sits on a category-coloured badge. Implement them as React components and in the WebGL atlas, refining proportions for 16 px while keeping the concept.

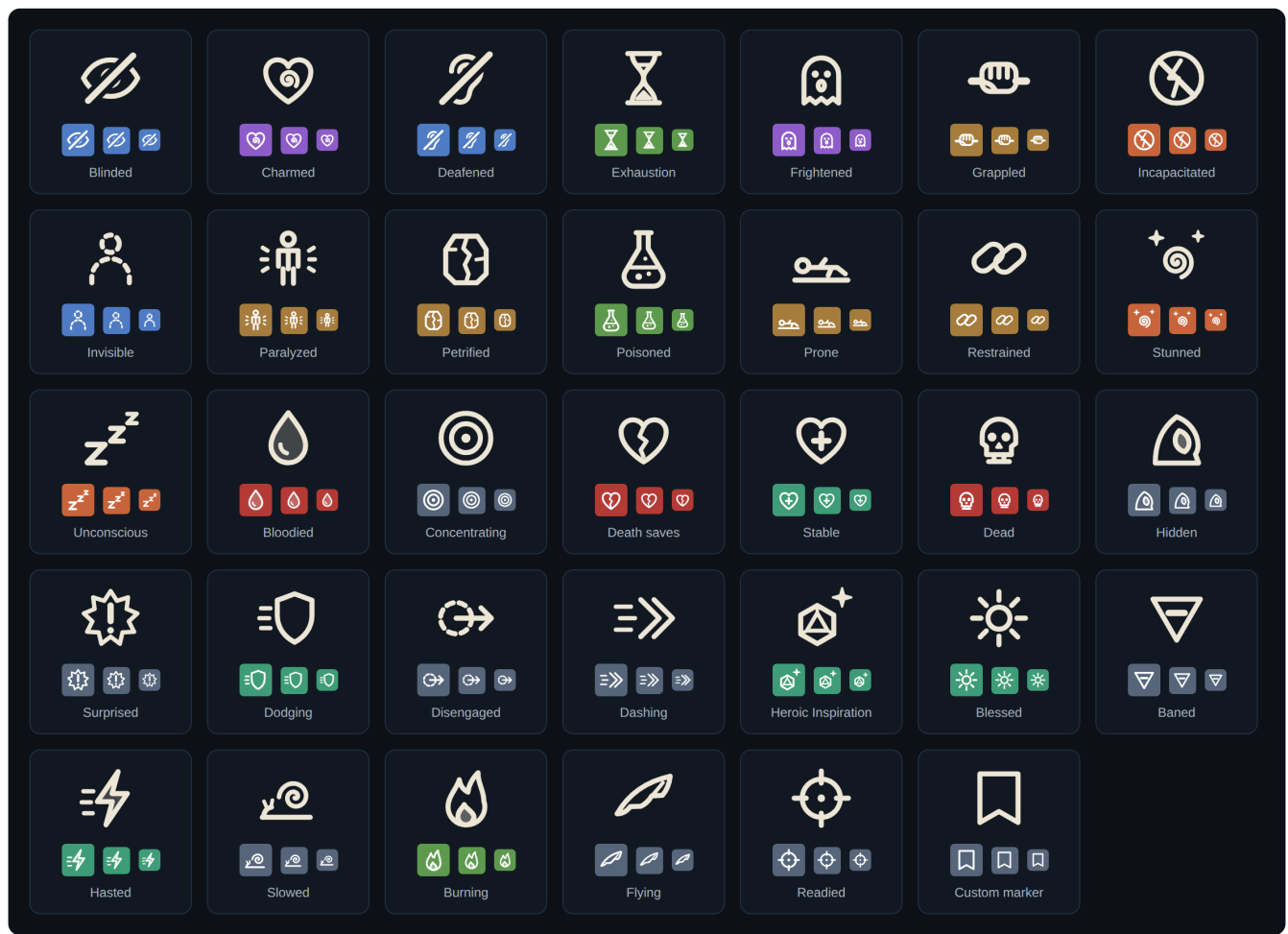


Figure 3 — Reference icon sheet: each glyph large, then on its category badge at 24, 20 and 16 px.

Badge categories

Category	Badge colour	Icons
Senses	#4E7BC4	Blinded, Deafened, Invisible
Mind	#8E5CC8	Charmed, Frightened
Affliction	#5E9A4E	Exhaustion, Poisoned, Burning
Body	#A67C3D	Grappled, Paralyzed, Petrified, Prone, Restrained
Incapacity	#C8643B	Incapacitated, Stunned, Unconscious
Vital	#B43A36	Bloodied, Death saves, Dead
Tactical	#56657A	Concentrating, Hidden, Surprised, Disengaged, Dashing, Banned, Slowed, Flying, Readied, Custom marker
Boon	#3F9C78	Stable, Dodging, Heroic Inspiration, Blessed, Hasted

Source (each `<svg>` is one icon; `id` in the comment is the icon ID used in code; mask IDs are prefixed `gx-` and must stay unique when several icons are inlined on one page):

```

<!-- blinded · Blinded · senses -->
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24" width="24" height="24" fill="none"
stroke="currentColor" stroke-width="1.75" stroke-linecap="round" stroke-linejoin="round"><mask id="gx-
blinded"><rect width="24" height="24" fill="#fff"/><path d="M4.2 19.8L19.8 4.2" stroke="#000" stroke-width="5.4"/
></mask><g mask="url(#gx-blinded)"><path d="M2.6 12c2.3-4.2 5.5-6.3 9.4-6.3s7.1 2.1 9.4 6.3c-2.3 4.2-5.5 6.3-9.4
6.3S4.9 16.2 2.6 12z"/><circle cx="12" cy="12" r="2.6"/></g><path d="M4.2 19.8L19.8 4.2" stroke-width="2.2"/></
svg>
<!-- charmed · Charmed · mind -->
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24" width="24" height="24" fill="none"
stroke="currentColor" stroke-width="1.75" stroke-linecap="round" stroke-linejoin="round"><path d="M12 20.2 C12

```

```
20.2 4.2 15.6 4.2 9.9 A4.1 4.1 0 0 1 12 8 A4.1 4.1 0 0 1 19.8 9.9 C19.8 15.6 12 20.2 12 20.2 Z"/><path d="M11.81
12.53 L11.77 12.46 L11.76 12.37 L11.77 12.27 L11.81 12.17 L11.89 12.08 L12 12 L12.13 11.95 L12.29 11.92 L12.45
11.94 L12.63 12 L12.79 12.1 L12.94 12.25 L13.05 12.44 L13.13 12.65 L13.16 12.9 L13.14 13.16 L13.06 13.41 L12.91
13.66 L12.71 13.88 L12.46 14.06 L12.16 14.19 L11.83 14.26 L11.48 14.25 L11.13 14.17 L10.78 14.02 L10.47 13.78
L10.2 13.48 L10 13.12 L9.87 12.71 L9.83 12.27 L9.88 11.81 L10.02 11.36 L10.26 10.94 L10.59 10.57 L11.01 10.27
L11.48 10.05 L12.01 9.93 L12.56 9.92 L13.11 10.03 L13.64 10.25 L14.13 10.59 L14.55 11.02 L14.88 11.54 L15.1 12.14
L15.2 12.77 L15.16 13.43 L14.99 14.08 L14.68 14.69" stroke-width="1.5"/></svg>
<!-- deafened · Deafened · senses -->
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24" width="24" height="24" fill="none"
stroke="currentColor" stroke-width="1.75" stroke-linecap="round" stroke-linejoin="round"><mask id="gx-
deafened"><rect width="24" height="24" fill="#fff"/><path d="M3.8 20.2L20.2 3.8" stroke="#000" stroke-width="5.4"/
></mask><g mask="url(#gx-deafened)"><path d="M7 9.6a5.1 5.1 0 0 1 10.2 0c0 2.3-1.2 3.5-2.4 4.6-1 .9-1.4 1.9-1.6
3.1-.3 1.8-1.5 3-3.2 3-1.4 0-2.5-.8-2.9-2"/><path d="M9.8 9.8a2.3 2.3 0 1 1 3.9 1.6c-.7-1.5 1.2-1.6 2.3"/></
g><path d="M3.8 20.2L20.2 3.8" stroke-width="2.2"/></svg>
<!-- exhaustion · Exhaustion · affliction -->
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24" width="24" height="24" fill="none"
stroke="currentColor" stroke-width="1.75" stroke-linecap="round" stroke-linejoin="round"><path d="M6.5 3.5h11M6.5
20.5h11"/><path d="M8 3.5c0 4.2 4 5.3 4 8.5s-4 4.3-4 8.5M16 3.5c0 4.2-4 5.3-4 8.5s4 4.3 4 8.5"/><path d="M9.6
19.1h4.8L12 16.6z" fill="currentColor" stroke-width="1"/><path d="M10.2 7.4h3.6" stroke-width="1.4"/></svg>
<!-- frightened · Frightened · mind -->
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24" width="24" height="24" fill="none"
stroke="currentColor" stroke-width="1.75" stroke-linecap="round" stroke-linejoin="round"><path d="M6 20.6V11a6 6 0
0 1 12 0v9.6L-2-1.6-2 1.6-2-1.6-2 1.6-2-1.6z"/><circle cx="9.9" cy="10.6" r="1.25" fill="currentColor"
stroke="none"/><circle cx="14.1" cy="10.6" r="1.25" fill="currentColor" stroke="none"/><ellipse cx="12" cy="14.6"
rx="1.1" ry="1.5"/></svg>
<!-- grappled · Grappled · body -->
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24" width="24" height="24" fill="none"
stroke="currentColor" stroke-width="1.75" stroke-linecap="round" stroke-linejoin="round"><path d="M2.5 13h3.6M17.9
13h3.6" stroke-width="2.6"/><rect x="6.1" y="7.2" width="11.8" height="9.4" rx="3"/><path d="M9.1 7.3v4.4M12
7.3v4.4M14.9 7.3v4.4"/><path d="M6.3 13.4c2 .1 3.6 1.1 4.6 3.1"/></svg>
<!-- incapacitated · Incapacitated · incapacity -->
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24" width="24" height="24" fill="none"
stroke="currentColor" stroke-width="1.75" stroke-linecap="round" stroke-linejoin="round"><circle cx="12" cy="12"
r="8.6"/><path d="M5.9 5.9L12.2 12.2" stroke-width="2"/><path d="M13.4 5.9L-4 6.1h3.3L-1.9 6.1" stroke-
width="1.6"/></svg>
<!-- invisible · Invisible · senses -->
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24" width="24" height="24" fill="none"
stroke="currentColor" stroke-width="1.75" stroke-linecap="round" stroke-linejoin="round"><circle cx="12" cy="7"
r="3" stroke-dasharray="2 2.1"/><path d="M5.5 20.5v-1.8a6.5 6.5 0 0 1 13 0v1.8" stroke-dasharray="2.2 2.2"/></svg>
<!-- paralyzed · Paralyzed · body -->
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24" width="24" height="24" fill="none"
stroke="currentColor" stroke-width="1.75" stroke-linecap="round" stroke-linejoin="round"><circle cx="12" cy="5.4"
r="2.1"/><path d="M8.6 9.4h6.8M12 9.4v6.3M8.6 9.4v5.4M15.4 9.4v5.4M10.2 15.7v5M13.8 15.7v5"/><path d="M3.2 8.4L1.6
1M2.8 12.4h1.9M3.2 16.4L1.6-1M20.8 8.4L-1.6 1M21.2 12.4h-1.9M20.8 16.4L-1.6-1" stroke-width="1.5"/></svg>
<!-- petrified · Petrified · body -->
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24" width="24" height="24" fill="none"
stroke="currentColor" stroke-width="1.75" stroke-linecap="round" stroke-linejoin="round"><path d="M5 9L3-4.6h8L19
9v8.4L-3.2 3.1H8.2L5 17.4z"/><path d="M12.4 4.5L-1.3 4.1 2.2 2.3-1.7 3.4 1.3 3.2-.9 3" stroke-width="1.5"/><path
d="M5 9h3.4M15.6 12.5L19 12" stroke-width="1.3"/></svg>
<!-- poisoned · Poisoned · affliction -->
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24" width="24" height="24" fill="none"
stroke="currentColor" stroke-width="1.75" stroke-linecap="round" stroke-linejoin="round"><path d="M9.4
3.2h5.2M10.4 3.4v5.3L5.7 16.6a3.4 3.4 0 0 0 2.9 5.1h6.8a3.4 3.4 0 0 0 2.9-5.1L-4.7-7.9V3.4"/><path d="M7.5 15.2h9"
stroke-width="1.4"/><circle cx="10.3" cy="18.4" r="1.15" fill="currentColor" stroke="none"/><circle cx="13.8"
cy="17.4" r="1.8" fill="currentColor" stroke="none"/><circle cx="12.6" cy="12.2" r="1.7" fill="currentColor"
stroke="none"/></svg>
<!-- prone · Prone · body -->
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24" width="24" height="24" fill="none"
stroke="currentColor" stroke-width="1.75" stroke-linecap="round" stroke-linejoin="round"><path d="M2.8 19.6h18.4"/
><circle cx="5.6" cy="14.6" r="2.1"/><path d="M7.9 15.2h9.6L2.8 2.2M11.4 15.2L2-2.7M15.6 15.2L-1.2 2.8"/></svg>
<!-- restrained · Restrained · body -->
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24" width="24" height="24" fill="none"
stroke="currentColor" stroke-width="1.75" stroke-linecap="round" stroke-linejoin="round"><rect x="2.8" y="8.7"
width="11" height="6.2" rx="3.1" transform="rotate(-40 8.3 11.8)"/><rect x="10.2" y="9.1" width="11" height="6.2"
rx="3.1" transform="rotate(-40 15.7 12.2)"/></svg>
<!-- stunned · Stunned · incapacity -->
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24" width="24" height="24" fill="none"
stroke="currentColor" stroke-width="1.75" stroke-linecap="round" stroke-linejoin="round"><path d="M12.3 13.2
12.36 13.26 L12.41 13.35 L12.43 13.45 L12.44 13.56 L12.41 13.68 L12.36 13.8 L12.27 13.91 L12.16 14.02 L12.01 14.1
L11.85 14.15 L11.67 14.17 L11.47 14.16 L11.27 14.11 L11.08 14.01 L10.9 13.88 L10.74 13.7 L10.6 13.49 L10.51 13.25
L10.46 12.98 L10.45 12.7 L10.51 12.41 L10.61 12.12 L10.77 11.85 L10.99 11.6 L11.25 11.39 L11.56 11.23 L11.9 11.11
L12.27 11.06 L12.65 11.08 L13.04 11.16 L13.41 11.32 L13.76 11.54 L14.08 11.83 L14.34 12.19 L14.55 12.59 L14.68
13.03 L14.73 13.5 L14.71 13.99 L14.59 14.47 L14.39 14.93 L14.1 15.37 L13.73 15.75 L13.3 16.07 L12.8 16.31 L12.26
16.47 L11.68 16.53 L11.1 16.49 L10.52 16.35 L9.96 16.1 L9.45 15.75 L8.99 15.32 L8.61 14.79 L8.32 14.21 L8.14 13.56
L8.07 12.89 L8.12 12.2 L8.28 11.52 L8.57 10.87 L8.98 10.27 L9.49 9.74 L10.09 9.3 L10.77 8.97 L11.51 8.75 L12.29
8.67 L13.08 8.72 L13.86 8.91 L14.6 9.24 L15.29 9.7 L15.9 10.28 L16.4 10.96 L16.78 11.73 L17.03 12.56 L17.13 13.44
L17.08 14.33 L16.87 15.22 L16.5 16.06 L15.99 16.84 L15.35 17.52 L14.59 18.09 L13.73 18.53" stroke-width="1.6"/
```

```

<path d="M5 2.6 L5.57 4.43 L7.4 5 L5.57 5.57 L5 7.4 L4.43 5.57 L2.6 5 L4.43 4.43 Z" fill="currentColor" stroke-
width="0.8"/><path d="M19.4 2.6 L19.82 3.98 L21.2 4.4 L19.82 4.82 L19.4 6.2 L18.98 4.82 L17.6 4.4 L18.98 3.98 Z"
fill="currentColor" stroke-width="0.8"/></svg>
<!-- unconscious · Unconscious · incapacity -->
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24" width="24" height="24" fill="none"
stroke="currentColor" stroke-width="1.75" stroke-linecap="round" stroke-linejoin="round"><path d="M4 14.2h6l-6
6.2h6" stroke-width="2"/><path d="M12 8.6h4.4L12 13h4.4" stroke-width="1.8"/><path d="M17.8 3.6h3.2l-3.2 3.2h3.2"
stroke-width="1.5"/></svg>
<!-- bloodied · Bloodied · vital -->
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24" width="24" height="24" fill="none"
stroke="currentColor" stroke-width="1.75" stroke-linecap="round" stroke-linejoin="round"><path d="M12 3.2c3.5 4.6
6 8 6 11.3a6 6 0 0 1-12 0c0-3.3 2.5-6.7 6-11.3z" fill="currentColor" fill-opacity=".22"/><path d="M9.3 14.2a2.9
2.9 0 0 0 2.2 3" stroke-width="1.5"/></svg>
<!-- concentrating · Concentrating · tactical -->
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24" width="24" height="24" fill="none"
stroke="currentColor" stroke-width="1.75" stroke-linecap="round" stroke-linejoin="round"><circle cx="12" cy="12"
r="8.6"/><circle cx="12" cy="12" r="5"/><circle cx="12" cy="12" r="1.6" fill="currentColor" stroke="none"/></svg>
<!-- deathsaves · Death saves · vital -->
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24" width="24" height="24" fill="none"
stroke="currentColor" stroke-width="1.75" stroke-linecap="round" stroke-linejoin="round"><path d="M12 20.2 C12
20.2 4.2 15.6 4.2 9.9 A4.1 4.1 0 0 1 12 8 A4.1 4.1 0 0 1 19.8 9.9 C19.8 15.6 12 20.2 12 20.2 Z"/><path d="M12
8.1l-1.4 3.2 2.4 1.8-2 3 .9 2.2" stroke-width="1.5"/></svg>
<!-- stable · Stable · boon -->
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24" width="24" height="24" fill="none"
stroke="currentColor" stroke-width="1.75" stroke-linecap="round" stroke-linejoin="round"><path d="M12 20.2 C12
20.2 4.2 15.6 4.2 9.9 A4.1 4.1 0 0 1 12 8 A4.1 4.1 0 0 1 19.8 9.9 C19.8 15.6 12 20.2 12 20.2 Z"/><path d="M12
10.3v5.2M9.4 12.9h5.2" stroke-width="2"/></svg>
<!-- dead · Dead · vital -->
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24" width="24" height="24" fill="none"
stroke="currentColor" stroke-width="1.75" stroke-linecap="round" stroke-linejoin="round"><path d="M6.2 11.2a5.8
5.8 0 0 1 11.6 0v2.6l-1.5 1.1v2.8H7.7v-2.8l-1.5-1.1z"/><circle cx="9.6" cy="11.6" r="1.55" fill="currentColor"
stroke="none"/><circle cx="14.4" cy="11.6" r="1.55" fill="currentColor" stroke="none"/><path d="M12 13.9l-.8
1.3h1.6z" fill="currentColor" stroke-width="1"/><path d="M10.3 17.7v2.4M13.7 17.7v2.4M8.4 20.1h7.2"/></svg>
<!-- hidden · Hidden · tactical -->
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24" width="24" height="24" fill="none"
stroke="currentColor" stroke-width="1.75" stroke-linecap="round" stroke-linejoin="round"><path d="M4.6 20.6c0-6.1
1.7-11.7 7.4-16 4.4 1.6 7.4 5.6 7.4 10 0 1.1-.3 2.1-.9 3l1.1 3z"/><path d="M12.9 9.4c2 .6 3.4 2.4 3.4 4.4 0 1.2-.6
2.2-1.5 2.9-1.9-.4-3.3-2-3.3-4 0-1.2.5-2.4 1.4-3.3z" fill="currentColor" fill-opacity=".35"/></svg>
<!-- surprised · Surprised · tactical -->
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24" width="24" height="24" fill="none"
stroke="currentColor" stroke-width="1.75" stroke-linecap="round" stroke-linejoin="round"><path d="M12 2.6 L14.19
5.99 L18.04 4.8 L17.54 8.8 L21.26 10.37 L18.3 13.11 L20.14 16.7 L16.11 16.9 L15.21 20.83 L12 18.4 L8.79 20.83
L7.89 16.9 L3.86 16.7 L5.7 13.11 L2.74 10.37 L6.46 8.8 L5.96 4.8 L9.81 5.99 Z" stroke-width="1.5"/><path d="M12
7.6v5.4" stroke-width="2.2"/><circle cx="12" cy="16.2" r="1.15" fill="currentColor" stroke="none"/></svg>
<!-- dodging · Dodging · boon -->
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24" width="24" height="24" fill="none"
stroke="currentColor" stroke-width="1.75" stroke-linecap="round" stroke-linejoin="round"><path d="M14.2 3.6l6
2.2v5.2c0 4.3-2.6 7.6-6 9.3-3.4-1.7-6.5-6-9.3V5.8z"/><path d="M2.6 8.6h3.2M2 12h3.4M2.6 15.4h3.2" stroke-
width="1.5"/></svg>
<!-- disengaged · Disengaged · tactical -->
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24" width="24" height="24" fill="none"
stroke="currentColor" stroke-width="1.75" stroke-linecap="round" stroke-linejoin="round"><circle cx="8.6" cy="12"
r="5.4" stroke-dasharray="2.4 2"/><path d="M8.6 12h12.2M17.2 8.4l3.6 3.6-3.6 3.6"/></svg>
<!-- dashing · Dashing · tactical -->
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24" width="24" height="24" fill="none"
stroke="currentColor" stroke-width="1.75" stroke-linecap="round" stroke-linejoin="round"><path d="M2.4 8h5.2M3.8
12h5.2M2.4 16h5.2" stroke-width="1.5"/><path d="M11 5.6l6.4 6.4-6.4 6.4M15.6 5.6l6.4 6.4-6.4 6.4"/></svg>
<!-- inspiration · Heroic Inspiration · boon -->
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24" width="24" height="24" fill="none"
stroke="currentColor" stroke-width="1.75" stroke-linecap="round" stroke-linejoin="round"><path d="M10.6 7.6 L16.32
10.9 L16.32 17.5 L10.6 20.8 L4.88 17.5 L4.88 10.9 Z"/><path d="M10.6 10.2 L14.2 16.3 L7 16.3 Z" stroke-
width="1.4"/><path d="M10.6 7.6v2.6M4.9 17.5l2.1-1.2M16.3 17.5l-2.1-1.2" stroke-width="1.2"/><path d="M19.2 1.6
L19.91 4.09 L22.4 4.8 L19.91 5.51 L19.2 8 L18.49 5.51 L16 4.8 L18.49 4.09 Z" fill="currentColor" stroke-
width="0.8"/></svg>
<!-- blessed · Blessed · boon -->
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24" width="24" height="24" fill="none"
stroke="currentColor" stroke-width="1.75" stroke-linecap="round" stroke-linejoin="round"><circle cx="12" cy="12"
r="3.8"/><path d="M18.2 12 L21.2 12 M16.38 16.38 L18.51 18.51 M12 18.2 L12 21.2 M7.62 16.38 L5.49 18.51 M5.8 12
L2.8 12 M7.62 7.62 L5.49 5.49 M12 5.8 L12 2.8 M16.38 7.62 L18.51 5.49"/></svg>
<!-- baned · Baned · tactical -->
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24" width="24" height="24" fill="none"
stroke="currentColor" stroke-width="1.75" stroke-linecap="round" stroke-linejoin="round"><path d="M3.8 5.4h16.4L12
19.6z"/><path d="M9.2 10.2h5.6" stroke-width="2.2"/></svg>
<!-- hasted · Hasted · boon -->
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24" width="24" height="24" fill="none"
stroke="currentColor" stroke-width="1.75" stroke-linecap="round" stroke-linejoin="round"><path d="M14.6 2.8L7.4
13.2h5.2l-2.2 8 7.4-10.8h-5.3z" fill="currentColor" fill-opacity=".22"/><path d="M2.6 9.6h3.4M2 13.4h3.2M2.8

```

```

17.2h3" stroke-width="1.5"/></svg>
<!-- slowed · Slowed · tactical -->
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24" width="24" height="24" fill="none"
stroke="currentColor" stroke-width="1.75" stroke-linecap="round" stroke-linejoin="round"><path d="M14.2 11.9
L14.26 11.97 L14.35 12.03 L14.47 12.07 L14.59 12.08 L14.73 12.06 L14.87 12 L15 11.91 L15.12 11.78 L15.22 11.62
L15.29 11.44 L15.32 11.23 L15.31 11.01 L15.25 10.78 L15.15 10.55 L15 10.34 L14.8 10.14 L14.56 9.99 L14.29 9.87
L13.98 9.8 L13.65 9.79 L13.31 9.84 L12.98 9.95 L12.66 10.13 L12.36 10.37 L12.1 10.67 L11.9 11.02 L11.76 11.42
L11.68 11.85 L11.68 12.3 L11.77 12.75 L11.94 13.2 L12.18 13.62 L12.51 14 L12.91 14.32 L13.38 14.58 L13.89 14.75
L14.44 14.84 L15.01 14.83 L15.58 14.72 L16.14 14.5 L16.66 14.18 L17.13 13.77 L17.53 13.28 L17.84 12.7 L18.06 12.08
L18.16 11.41 L18.15 10.71 L18.01 10.02 L17.75 9.35 L17.37 8.72 L16.88 8.16 L16.29 7.68 L15.62 7.3 L14.88 7.05
L14.09 6.92 L13.27 6.93 L12.46 7.08 L11.67 7.38 L10.93 7.81 L10.27 8.38 L9.71 9.06 L9.27 9.83 L8.96 10.69 L8.8
11.6" stroke-width="1.6"/><path d="M3 18.6h16.6"/><path d="M3.4 18.6c.2-2 1.2-3.2 2.8-3.5"/><path d="M4.6
15.4l-1-2.3M6 15.2l.5-2.5" stroke-width="1.3"/></svg>
<!-- burning · Burning · affliction -->
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24" width="24" height="24" fill="none"
stroke="currentColor" stroke-width="1.75" stroke-linecap="round" stroke-linejoin="round"><path d="M12 21.2a6.2 6.2
0 0 1-6.2-6.2c0-3.6 2.7-5.8 3.9-9.4 4.2 5 1.8 3.7 3 4.3-2.5 1.5-4.7 3.5-6.7-.2 3.7 2 5.8 2 9.8a6.2 6.2 0 0 1-6.2
8.2z"/><path d="M12 21.2a2.8 2.8 0 0 1-2.8-2.8c0-1.9 1.4-2.8 2.2-4.2 7 1.4 2.4 2.1 3.2 3.6-3.6 2 1.2-.1 1.7a2.8
2.8 0 0 1-2.5 1.7z" fill="currentColor" fill-opacity=".35"/></svg>
<!-- flying · Flying · tactical -->
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24" width="24" height="24" fill="none"
stroke="currentColor" stroke-width="1.75" stroke-linecap="round" stroke-linejoin="round"><path d="M2.8 17.4C6.9
10.3 12.9 6.2 21.2 4.4c-.3 2.7-1.2 4.9-2.7 6.6l-3.7-2c-.9 1.6-2.2 2.9-3.8 3.6l-3.6-1c-1.2 1.6-2.7 2.5-4.6 2.5z"/
><path d="M14.8 11.2c1.4-1.5 2.7-3 3.9-4.6M11 14.8c1.9-1.4 3.6-3 5.1-4.8" stroke-width="1.3"/></svg>
<!-- readied · Readied · tactical -->
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24" width="24" height="24" fill="none"
stroke="currentColor" stroke-width="1.75" stroke-linecap="round" stroke-linejoin="round"><circle cx="12" cy="12"
r="6.6"/><path d="M12 2.6v4.2M12 17.2v4.2M2.6 12h4.2M17.2 12h4.2"/><circle cx="12" cy="12" r="1.3"
fill="currentColor" stroke="none"/></svg>
<!-- custom · Custom marker · tactical -->
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24" width="24" height="24" fill="none"
stroke="currentColor" stroke-width="1.75" stroke-linecap="round" stroke-linejoin="round"><path d="M5.4
3.6h13.2v16.8L12 16.6l-6.6 3.8z"/></svg>

```

Appendix H — Keyboard and gesture map

Action	Desktop	Touch
Select / deselect	Click / Esc; DM multi-select: Shift+click, Shift+drag on empty board (box)	Tap
Move token	Drag (Alt = freehand); or click-to-move: click floor to move, Ctrl/Cmd+click adds waypoints, then click or Enter commits	Drag with one finger
Reset move / undo last segment	Backspace / Ctrl+Z	Buttons on the action bar
Pan	Left-drag on empty board, Space+drag, middle-drag	One-finger drag on empty board
Orbit / tilt	Right-drag	Two-finger twist / two-finger vertical drag
Zoom	Wheel (toward cursor)	Pinch
Camera presets	Shift+1 top-down · Shift+2 tabletop · Shift+3 low · T toggle top-down/tabletop · O orthographic	Camera button menu
Focus / follow selected	F / Shift+F	Double-tap
Radial menu	Right-click token	Long-press token
Action hotbar	1–9	Tap the slot
Elevation (flyers)	Alt+wheel over the token, or the HUD stepper	HUD stepper
Rotate a spell template	[/] or wheel while placing	Rotate handle
DM copy / paste tokens	Ctrl/Cmd+C, Ctrl/Cmd+V (at the cursor)	—
Wall drawing	Click points; double-click or Esc ends; Backspace removes last segment; Shift snaps angle to 15°; Ctrl/Cmd disables snapping	Tablet: tap points, done button
Ping / DM spotlight	Alt+click / Alt+Shift+click	Long-press empty board

Action	Desktop	Touch
Measure	M (then click points; Esc clears)	Measure button
Movement range overlay	G	Toggle button
Dice tray	D	Dice tab
Emote wheel	E	Long-press own portrait, or Emote in your token's radial menu
Hand raise	H	Party list button
Character sheet	C	Sheet tab
Log	L	Log tab
End turn	Ctrl+Enter	End turn button
Command palette	Ctrl/Cmd+K	—
Undo / redo	Ctrl/Cmd+Z / Ctrl/Cmd+Shift+Z (or Ctrl+Y)	Buttons in the ⋮ menu
Shortcut sheet	?	—
DM tools	V select · W walls · Shift+W doors · I lights · Z zones · B fog brush · R reveal room · X effects	Tools menu (tablet)
DM combat	Ctrl/Cmd+Shift+C quick start / stop (toggle) · N next · P previous	Combat panel
Advantage / disadvantage on a roll	Alt+click / Ctrl(Cmd)+click	Long-press → menu

Appendix I — Attribution and trademarks

SRD 5.2.1 attribution (verbatim, required):

This work includes material from the System Reference Document 5.2.1 ("SRD 5.2.1") by Wizards of the Coast LLC, available at <https://www.dndbeyond.com/srd>. The SRD 5.2.1 is licensed under the Creative Commons Attribution 4.0 International License, available at <https://creativecommons.org/licenses/by/4.0/legalcode>.

SRD 5.1 attribution (only if the optional 5.1 pack is used, verbatim):

This work includes material taken from the System Reference Document 5.1 ("SRD 5.1") by Wizards of the Coast LLC and available at <https://dnd.wizards.com/resources/systems-reference-document>. The SRD 5.1 is licensed under the Creative Commons Attribution 4.0 International License available at <https://creativecommons.org/licenses/by/4.0/legalcode>.

- The CC-BY licence does not license trademarks. Keep "Dungeons & Dragons", "D&D", "D&D Beyond", "Wizards of the Coast", the dragon ampersand and Wizards' product and setting names out of the app's name, logo and branding. The SRD's own terms permit saying the work is "compatible with fifth edition" or "5E compatible"; no other attribution to Wizards is allowed.
- Foundry VTT `dnd5e` data is used only at build time as a structured source for SRD mechanics (MIT code, CC-BY-4.0 content); credit it in About & Credits.
- Fonts: Fraunces, Alegreya Sans, Cinzel, JetBrains Mono — SIL Open Font License 1.1 (include the licence texts).
- ZzFX — MIT. Other libraries — list licences in About & Credits (generate from `pnpm licenses list --prod`).

Appendix J — Configuration reference

Variable	Default	Meaning
PORT	4747	HTTP + WebSocket port

Variable	Default	Meaning
HOST	127.0.0.1	Bind address (LAN mode sets 0.0.0.0 via settings, not env)
DATA_DIR	./data	Data directory
METRICS_PORT	4748	cloudflared metrics port (loopback)
CLOUDFLARED_PATH	c:\cloudflared	Path to the cloudflared binary
PUBLIC_URL	—	Public URL for named tunnels (also settable in the UI)
(TUNNEL_TOKEN)	—	Not read by Gloam: Gloam passes the stored named-tunnel token to the cloudflared child process through this variable (never argv)
LOG_LEVEL	info	pino level
NODE_ENV	development	production for pnpm start
GLOAM_TEST_SEED	—	Test builds only: seeds dice

UI-managed settings (table `settings`): tunnel mode, named-tunnel token (encrypted), LAN mode, cloudflared path, port (the last four only from the host PC), auto-admit returning players, DMs can admit, auto-approve player images, allow admin login through the doorway, allow remote API, defaults for new campaigns (rules pack, units, house rules).

Errata

Clarifications discovered during the build are appended here by the builder (date, section, clarification), mirrored in `docs/DECISIONS.md`.